

I/O from Python to the operating system — PB158 Lecture 2

Contents

1 The claim

2 1. What does one byte cost?

2.1 Why your laptop will disagree

3 2. Where does the difference come from?

3.1 Why a crossing is expensive

3.2 The tool that watches the boundary

3.3 Reading a trace

3.4 Counting, and the honest limit on the count

3.5 What a buffer does, and when it stops paying

3.6 Trusting the instrument

3.7 Windows counts too, and differently

3.8 Counting on Windows: ask the kernel

4 3. It is not one buffer — it is a stack

4.1 One rule instead of four facts

4.2 One program, one buffer — C and Python take different paths

4.3 The bottom level, and the limit on the thesis

4.4 The demonstration, and what it does not prove

4.5 `ltrace` : the same idea, one layer up

4.6 Which boundaries a tool can actually watch

5 4. The same write, at three levels

5.1 The demo program

5.2 `fwrite` takes a size and a count

6 5. Two operating systems, side by side

6.1 Process Monitor: what the kernel did

6.2 NtTrace: a tracer for the Windows Native API

6.3 Why NtTrace and not Process Monitor

6.4 Why the boundaries sit where they do

6.5 One program, two kernels

7 6. What we have not looked at

7.1 Buffering moves your errors

7.2 Networking and IPC

7.3 Honest limits

8 What follows

8.1 Assignment 1

9 References

10 Appendix — sources from the author's working notes

10.1 V1 — The cost structure of a system call

10.2 V2 — the true cost of a system call

10.3 V3 — System-call count as the first-order I/O cost driver

10.4 V4 — The Non-Durability of a Successful write() or WriteFile() Call

10.5 V5 — The Non-Guarantee Behind a Successful write() or WriteFile() Call

10.6 V6 — Destination-dependent buffering of standard output

10.7 V7 — Stdio buffer sizing on regular files

10.8 V8 — The layer at which Windows and Linux stabilize the syscall boundary

10.9 V9 — Where Windows and Linux place the stable syscall boundary

10.10 V10 — The System Call Boundary

10.11 V11 — The System Call Boundary

10.12 V12 — The Syscall Boundary's Silent Scope Clause

10.13 V13 — The trustworthiness gap between strace's call counts and its timings

10.14 V14 — Leaky OS Abstractions at the Syscall Boundary

10.15 V15 — The Leaky Boundary Between Application Abstractions and the Operating System

10.16 V16 — Leaky Operating-System Abstractions and How to Observe Them

10.17 V17 — Mechanisms for Observing a Running System

10.18 V18 — The Memory Hierarchy from a Python Object to a Physical Page

10.19 V19 — The cache layers behind a benchmark number

10.20 V20 — File I/O and durability

10.21 V21 — NtTrace, a debug-interface tracer for the Windows Native API

10.22 V22 — NtTrace, a user-mode Windows native-API tracer

10.23 V23 — NtTrace's published sample capture

10.24 V24 — WiredTiger's fsync() retry-to-panic fix

10.25 V25 — The production-unsuitability of syscall-boundary tracing (ptrace / strace)

10.26 V26 — The System Call Boundary

10.27 V27 — The system call as a distinct hardware mechanism, not an expensive function call

10.28 V28 — The Kernel-Object Model Behind Sockets, and Where Each Platform Pins Its Contract

10.29 V29 — The kernel-object model behind sockets on Linux and Windows

10.30 V30 — The gap between a completed write and durable storage on Linux and Windows

10.31 V31 — Mechanisms for observing a running system

10.32 V32 — Virtual memory and the page cache as a fault-driven mechanism

10.33 V33 — File I/O durability across the OS buffering stack

- [10.34 V34 – Virtual Memory and the Page Cache](#)
- [10.35 V36 – The placement of the stable syscall boundary across operating systems](#)
- [10.36 V37 – The four-layer buffering-and-durability boundary model for file I/O](#)
- [10.37 V38 – the layer at which Windows and Linux fix their syscall contract](#)
- [10.38 V39 – The Trade-off Between Observation Cost and Blind Spots in Tracing Mechanisms](#)
- [10.39 V40 – PostgreSQL's fsync\(\) Surprise](#)
- [10.40 V41 – PostgreSQL's fsync\(\) Error-Reporting Failure](#)
- [10.41 V43 – The differing layer at which Windows and Linux fix the stable syscall boundary](#)
- [10.42 V44 – The dated hardware behind the FlexSC eviction measurements](#)
- [10.43 V45 – The system call as a per-crossing cost mechanism, not a per-byte one](#)
- [10.44 V46 – Why a system call's per-crossing cost cannot be quoted as one number](#)
- [10.45 V47 – NtTrace, a user-mode Windows native-API tracer](#)
- [10.46 V48 – NtTrace, a user-mode Windows native-API tracer](#)
- [10.47 V49 – NtTrace, a debugger-based native API tracer for Windows](#)

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

Reading this document: *[S<n>] cites the numbered source list at the end. [inference] marks a claim that follows from those sources but is not stated by them. [speculative] marks the author's judgement, offered as judgement.*

This document stands on its own. You do not need the slides to read it, and it goes further in several places than I went in the lecture. [V<n>] entries are notes from the author's research vault. The appendix at the end summarises each one and lists every external source it draws on.

1 The claim

The call you write is not the call the operating system runs [inference]. `fflush()` flushes only the C library's buffer, and the kernel buffer needs a second call [S3]. Four boundaries sit below your code. A different tool watches each of the first three:

- the application calling into the library [V10]
- the library calling into the kernel [S1]
- the kernel writing to the device [V30]

The fourth is where data is lost [V20], and no tool in this lecture sees it [V5]. I argue it in three steps. First I measure. Then I count. Then I look.

Four claims should survive a week.

1. Buffering is fast when writes are adjacent [inference]. It buys little when they are not. Lampson gave the batching half in 1983 [S12].
2. A library call is not a system call [S3]. Several levels sit between the call you type and the bytes on a medium.
3. The boundary is observable [S1]. A tracer shows which system call your source line became, and how many times it ran.
4. Linux and Windows nominate different levels as the stable boundary [V8].

2 1. What does one byte cost?

Two Python programs write the same mebibyte — 1,048,576 bytes — to a file. The first opens the file with `buffering=0`. The second takes the default. Python documents the argument: "Pass 0 to switch buffering off (only allowed in binary mode)" [S6].

The two programs hold everything else fixed:

- the loop
- the interpreter
- the iteration count
- the bytes
- the filesystem

That design matters. It eliminates "Python is slow" by construction rather than by argument, because the interpreter does the same work in both runs [inference].

The author prints no measured ratio, in the lecture or in this document. A benchmark the reader did not watch being produced is exactly the kind of claim this course teaches students to distrust [speculative]. The figure also differs between the lecture machine and a student laptop, for reasons covered below. Run it. Write down both wall-clock times. Record the Python version and `io.DEFAULT_BUFFER_SIZE` beside them [inference].

One caveat belongs on the measurement. A one-mebibyte file does not reach a disk in any detectable way. It lands in the operating system's own memory and stays there. The honest rule is to "state which cache layer your number includes, because you cannot eliminate all of them" [V19]. So this measures the cost of asking the operating system to do something, not the cost of a disk [inference].

2.1 Why your laptop will disagree

The size of Python's default buffer is not a constant. It was 8192 bytes for sixteen years. The CPython issue that changed it records that the value "was set to 8192 since 16 years ago" [S7]. The issue gives the reason for that choice. In 2008 "you'd run a 32 bits operating system with less than 2 GB memory available" [S7].

The maintainers then raised it on a measured "3 to 5 times write performance improvement" [S7]. **State the baseline with that multiplier.** It compares larger buffers against the 8192-byte default, on Python 3.11. The benchmark sweeps the buffer from 0 bytes up to 1 MiB [S7]. The workload writes a file streamed over HTTP and times only the write calls. The reporter ran it on local Linux storage, on an NFS home directory and on

Windows [S7]. The figure is the reporter's own measurement, not an independent reproduction [inference]. A core developer confirmed the direction in the same thread and posted no figures [S7]. Python now documents that "On most systems, the buffer will typically be 128 kilobytes long" [S6].

The practical consequence for assignment 1 is direct. A cohort split between Python 3.13 and 3.14 produces crossing counts that differ by a factor of sixteen [inference]. That reason has nothing to do with what the assignment measures [inference]. Report the version and the buffer size with every number.

The general habit is the point. Never quote a remembered constant about a system. Measure the machine in front of you. Say which machine it was [speculative].

3 2. Where does the difference come from?

The difference is the number of times each program crosses into the operating system. You can already define a system call. The author names what he counts instead. Every **crossing** is one system call: "a privilege transition, not a jump" — the processor changes protection domain and your process keeps running [V15].

The arithmetic is simple. The unbuffered program makes one call per byte, so one mebibyte costs 1,048,576 calls. The buffered program makes one call per buffer fill, so one mebibyte divided by the buffer this machine allocated. At 8 KiB that is 128 calls. At 128 KiB it is 8 [inference]. **Print the size before you divide.** Python allocates `max(min(blocksize, 8 MiB), DEFAULT_BUFFER_SIZE)` when the device block size is available [S6]. So the divisor depends on the filesystem as well as on the Python version [inference].

3.1 Why a crossing is expensive

Soares and Stumm measured the processor footprint of individual Linux system calls in FlexSC, at OSDI 2010 [S16]. A single `stat` call "retires 4,972 instructions in 13,585 cycles — an IPC of 0.37 — while evicting 32 L1 instruction-cache lines, 186 L1 data-cache lines, 660 L2 lines, 2,559 L3 lines, and 21 data-TLB entries" [V2]. **Those are 2010 figures, and the counts are not current.** The note that carries them says so: "the specific eviction counts should not be quoted as current" [V44]. What carries over is the regime distinction below, not the numbers [V44].

An IPC of 0.37 is roughly one finished instruction every three cycles [inference]. Straight-line code on the same processor finishes more than one per cycle, so this is slow work rather than ordinary work [speculative].

The instruction count is not the story. The paper counts the evicted lines and does not say whose data they held, and your program was the code running [inference]. Every one of those lines your code touches again is a miss it pays for after the call returns [inference]. The paper separates two regimes, and the distinction matters here. The indirect cost dominates "for medium frequencies of system call invocation (once per 2,000 to 100,000 instructions)", and at higher rates the direct transition cost takes over [S16]. Program A makes one system call for every loop iteration, which plausibly puts it in that high-frequency regime [speculative]. That placement is the author's [speculative]. The paper measures system calls rather than Python programs, and no count of the interpreter's instructions per call exists here [speculative]. Either way the cost is largely fixed per call. Doubling the bytes in one call is nearly free. Doubling the calls is not [inference].

3.2 The tool that watches the boundary

Reading a trace needs the tool that produces one. On Linux that tool is `strace`. Its manual page states what it does: it "intercepts and records the system calls made by a process and the signals a process receives" [S1]. The same page says where the output goes. It states that "The name of each system call, its arguments, and its return value are printed to standard error or to the file specified with the `-o` option" [S1]. So a shell redirect of standard output does not capture a trace [inference].

`strace` intercepts each call and records it. It does not sample [inference]. Two limits on the counts arrive later in this section. The counting run below uses a filter, and some Linux calls never reach the kernel for `strace` to intercept [V26].

The cost is speed. The manual page lists it under BUGS: "A traced process runs more slowly than a non-traced one" [S1]. Measured factors for one worst-case workload appear under *Trusting the instrument* below [V17]. Never benchmark a program under the tracer.

Where the tool comes from. Somebody built this tool, and you can read what they built. The manual page's HISTORY section records the authors: "The original `strace` was written by Paul Kranenburg for SunOS and was inspired by its trace utility. The SunOS version of `strace` was ported to Linux and enhanced by Branko Lankester" [S1]. The project publishes the source on GitHub and GitLab under the Lesser GPL, version 2.1 or later [S30]. It also names the mechanism: `strace` works through "the kernel feature known as `ptrace`" [S30]. That is the same kernel interface a debugger uses [inference].

This lecture uses three forms:

```

strace ./iodemo          # one line per system call
strace -c ./iodemo       # a table: calls, errors, time per call
strace -e trace=write ./iodemo # only the writes

```

Without `-c` the tracer prints one line per call, which the next subsection reads. With `-c` it "Counts time, calls, and errors for each system call and report a summary on program exit" [S1]. Read that time column with care. By default it "shows system time (CPU time spent in the kernel), which is independent of wall clock time" [S1]. That is a default and not a limit: the same page documents a `-w` option that "Summarizes the wall clock time for each system call" [S1]. I do not time a run under the tracer, and the reason is the overhead measured below, not this column [inference].

`strace` runs on Linux only. Support for non-Linux operating systems "was removed" in 2012 [S1]. The Windows tracer used in this course is NtTrace, and section 5 uses it [S10]. For the Windows counts in this section the demo programs ask Windows instead, through `GetProcessIoCounters` [S24]. `strace` is already installed in the course dev container, so nobody installs it. In the lab you run it yourself for the first time. In the lecture you only read its output.

3.3 Reading a trace

A **trace** records the system calls a program actually made, in order, with arguments and return values. The `strace` manual page documents the format and uses this example line [S1]:

```
open("/dev/null", O_RDONLY) = 3
```

Name, then arguments in parentheses, then the return value after the equals sign. Every line looks like this. The `3` here is a file descriptor, defined in section 4 below.

A trace is not a log. A log carries what the program's author chose to print. A trace carries what the program did [inference].

3.4 Counting, and the honest limit on the count

Running the two programs again under `strace -c -e trace=write python3 a.py` counts the calls. The counts match the arithmetic above, and that is the point of the exercise [inference]. You derive a number from a documented buffer size, then confirm it with an independent tool [inference].

The count ratio is not the time ratio, and the author says so explicitly in the lecture.

Total cost "is better modelled as $N_{\text{crossings}} \times \text{fixed_cost} + \text{bytes} \times \text{marginal_cost}$ than as a function of bytes alone. For small payloads the first term dominates" [V1]. Both programs write the same mebibyte through the same million-iteration loop [inference]. The second term and all the interpreter work form a floor under both [inference]. Under that model the count ratio over-predicts the time ratio [inference]. A count ratio of a hundred thousand against a time ratio of forty is not a mistake [inference]. Those two figures are an illustration rather than a measurement from this room [inference]. The source note grades the model **Confident** rather than Verified, a grade it reserves for a conclusion no source states outright [V45]. The per-crossing cost is also not one number. The same note reports "a 76–668 cycle spread for `close` and raw `syscall` invocations measured across fifteen different hosts" [V46]. Treat the model as a shape rather than a formula [inference].

3.5 What a buffer does, and when it stops paying

A buffer collects **adjacent** bytes and hands them down in one call. The saving sits entirely in the call count. The same bytes are copied either way [inference].

Now the condition. A buffer holds bytes destined for consecutive positions, because a block write carries one offset and one length. Seek somewhere else and those bytes cannot stay in it [inference]. They belong at the old offset, so the library must hand them over before it can move [inference]. With one byte written per seek the buffer never holds more than that byte, and the buffered run converges on the unbuffered one [inference].

So the rule is not "buffering is fast". It is *buffering is fast when writes are adjacent, and buys little when they are not* [inference]. Lampson stated the batching half in 1983: "Use batch processing if possible. Doing things incrementally almost always costs more, even aside from the fact that disks and tapes work much better when accessed sequentially" [S12]. He writes about physical media and sets that point aside from the batching advice. The claim that a seek empties a userspace buffer is mine, and it follows from the mechanism above [inference].

Two limits, both load-bearing. Adjacency is necessary and not sufficient: a program already writing a mebibyte in one call has nothing left to collect [inference]. And the seek itself costs a crossing. `pread` and `pwrite` "combine[s] file access and seek operations into a single command... thereby reducing the number of system calls issued from the user environment" [V3]. The call count is the measured part. For identical data volumes the reported result is "a relatively reduced load in terms of overall duration" [V27]. That is the source's own wording. It is weaker than a measured runtime

figure, and the author does not upgrade it [inference]. **Why those calls were added is a different question, and the source does not answer it.** The documented rationale is an atomic offset-and-access for threads that share a descriptor [speculative]. An unbuffered random-access run that writes one byte per seek makes two crossings per byte, not one [inference].

3.6 Trusting the instrument

The tracer stops the traced program twice per system call — once before the call runs, once after the return value is set. Both of the tool's properties follow from that single mechanism [inference].

Because it stops every call that reaches the kernel, it counts every one of them. Nothing is sampled. It cannot count what never crosses: Linux serves some calls from a page shared with the process, with no privilege transition, and "strace cannot see them, because there is nothing to intercept" [V26]. Because it stops every call, it also makes the program dramatically slower. The worst case on record is Brendan Gregg's: "`dd if=/dev/zero of=/dev/null bs=1 count=500k`", a program doing nothing but syscalls — runs 442× slower under `strace`, and does so even when tracing a syscall the program never makes, because the stop is paid regardless" [V17]. Re-runs of that same `dd` loop report "roughly 37× to 136×" on different hardware, with different processor security mitigations in place [V31]. **The later range does not contain Gregg's figure, and that is the point rather than a contradiction.** Overhead is proportional to the system-call rate, so each figure measures one machine running one workload rather than the tool [inference].

The rule for this course: count under the tracer, time without it. That "is why the *counts* are trustworthy and the absolute times are not" [V13]. A benchmark run under a tracer reports nonsense [inference].

3.7 Windows counts too, and differently

Neither platform needs a tracer to answer the question the benchmark asks. Both kernels already keep a per-process write count, and reading it costs one file read on Linux or one call on Windows [inference]. The counting summary is a feature of `strace` alone. NtTrace, the tracer I use on Windows, is a different design and prints no equivalent [speculative].

On Windows, `GetProcessIoCounters` "Retrieves accounting information for all I/O operations performed by the specified process" and fills an `IO_COUNTERS` structure [S24]. The member that answers the benchmark's question is `WriteOperationCount`. Microsoft

documents it as "The number of write operations performed" [S23]. One process may also read another process's count. The handle must carry the `PROCESS_QUERY_INFORMATION` or `PROCESS_QUERY_LIMITED_INFORMATION` access right [S24]. That is why a ready-made process viewer can display the same counts with no code of your own [inference]. Task Manager can add I/O columns on its Details tab — check it on your own machine rather than taking my word for it.

Linux keeps a similar count in `/proc/pid/io`, a file that "contains I/O statistics for the process and its waited-for children" [S25]. The manual page heads the field "syscw: write syscalls". It defines the field as the number of "file write" system calls — "those from the `write(2)` family, `sendfile(2)`, `copy_file_range(2)`, and `ioctl(2)` `BTRFS_IOC_ENCODED_WRITE[_32]`" [S25]. A program reads its own count through `/proc/self/io`, which is what `demo/iodemo-python/iodemo.py` does. On Windows the same program calls `GetProcessIoCounters` through `ctypes`. Both `demo/iodemo-python/iodemo.py` and `demo/iodemo-c/iodemo.c` print the count when given `--count`. A third program, `demo/iocount-win32/iocount_win32.py`, reads the same structure through the `PyWin32` package, which the course does not install.

One measurement, from one source file on both systems. On Windows `iodemo.exe --count` printed "write operations counted by the kernel: 1" [S37]. The same program in the Linux container printed 7 [S37]. The one write is the same write on both systems [inference]. The other six belong to the Linux loader, before `main` ran. The README states it as "the program's one write plus the loader's" [S37]. The two totals are also not one quantity measured twice. Linux counts the `write(2)` family through `syscw` [S25]. Microsoft counts "write operations performed" and defines it no further [S23]. That is the concrete reason the course tells you to compare two runs and never to read one [inference].

Three limits, and all three belong beside the count.

First, the two fields are not one measurement. The Linux page states what it counts: "file write" system calls [S25]. Microsoft states "The number of write operations performed" and stops there [S23]. So the Linux count is a count of crossings, and the Windows count cannot be described that way on the evidence the vendor publishes [inference].

Second, these are counts of I/O operations rather than of crossings. A seek is a crossing, and it is not a write. The Linux field counts calls from the write family, so it does not count a seek [inference]. The random-access program seeks once per write. Its count is therefore roughly half its crossings [inference]. The sequential program never seeks, so for that one the two agree [inference]. Microsoft defines a separate member as "The number of I/O operations performed, other than read and write operations" [S23]. The page does not say that a seek increments it, and I will not say so either.

Third, the count covers the whole process rather than your loop. The interpreter writes before your first line runs [inference]. So take the count for one program, take it for the other, and report the ratio rather than either absolute count [inference]. The Linux file widens the scope once more, to the process "and its waited-for children" [S25]. The same page adds a caveat about reading the value at all. The counters are not atomic. On a machine where a 64-bit read can tear, a concurrent update can yield an intermediate value that was never the count [S25].

All three limits leave the count useful. It answers the benchmark's question — did the buffered program make fewer calls — and it needs no tracer to do it [inference]. The platform comparison with a published Windows trace is still Part V [S10]. Equal footing between the platforms is a course-level promise, not a per-lecture one.

3.8 Counting on Windows: ask the kernel

In section 2 I counted on Linux with `strace -c`. This is the Windows path, in a program short enough to read on a slide. It is `demo/iocount-win32/iocount_win32.py`, cut to the lines that matter:

```
import win32api
import win32process

def write_operations():
    handle = win32api.GetCurrentProcess()
    return win32process.GetProcessIoCounters(handle)["WriteOperationCount"]

before = write_operations()
with open(path, "wb") as f:
    f.write(b"one\n")
    f.write(b"two\n")
    f.write(b"three\n")
after = write_operations()
print(f"the three writes cost:    {after - before} operation(s)")
```

Why not a tracer. There is no `strace -c` on Windows. NtTrace lists the calls it traces and prints no counting summary [speculative]. So instead of watching the process from outside, the program asks the kernel for the count it keeps anyway [S24].

What it does. `GetProcessIoCounters` fills an `IO_COUNTERS` structure for the process [S24]. The member that answers the benchmark's question is `WriteOperationCount` [S23]. The previous section quotes both pages in full. What is new here is the binding. `PyWin32` exposes the call as `win32process.GetProcessIoCounters(hProcess)`. It documents the call as returning "a dictionary representing an `IO_COUNTERS` struct" [S29]. That is why

the sample subscripts the result. The PyWin32 page documents the argument as a "Process handle as returned by OpenProcess" [S29]. The sample passes `win32api.GetCurrentProcess()` instead. A program that reads its own counters needs no `OpenProcess` call [inference].

What it cost, measured. I ran the script on Windows on 15 September 2026. The counter read 0 before the loop and 1 after it [S35]. Python collects the three writes in one buffer [S6]. Note where the second reading happens: after the `with` block closes the file. So that single operation covers the three writes and the close together [S35]. Microsoft does not document which call increments the counter. The `IO_COUNTERS` page defines the members and stops there [S23], so I infer the rest [inference]. The run has one obvious limit. It ran once, on one machine, and it did not record the Windows version or the Python build [S35]. Three `f.write` calls in the source, one write operation in the kernel — that gap is what I built section 2 to show.

The lab does not need the PyWin32 package. `demo/iodemo-python/iodemo.py --count` reads the same counter through `ctypes` on Windows, and reads `/proc/self/io` on Linux, taking the field `syscw` [S25]. The count covers the whole process rather than your loop, so the interpreter may write before your first line runs [inference]. That is why the sample reads the counter twice and subtracts. The two platforms charge different start-up costs, and the numbers show it. My Windows run began at 0. The Linux run of the demo program began with `syscw` already at 9 [S35]. Same question, two counters, two different starting points. So compare the difference, never the absolute number [inference].

The same numbers are visible with no code at all, in Task Manager or Process Explorer — check yours rather than taking my word for it [inference]. And the limit from the previous section still stands: these are I/O operations, not crossings. A seek is a crossing and it is not a write [inference].

4 3. It is not one buffer — it is a stack

There are four levels between your call and the medium. A different call empties each one.

LEVEL	WHO OWNS IT	EMPTIED BY
1 Application	the language runtime's own buffer [S8]	<code>f.flush()</code> [S17]
2 C library	the <code>stdio</code> buffer on a <code>FILE*</code> [S3]	<code>fflush()</code> [S3]
3 Kernel	the page cache, in operating-system memory [V32]	<code>fsync()</code> [S13], or <code>Flush-FileBuffers</code> on Windows [V37]
4 Device	the drive's own write cache [V20]	a cache-flush command [V33]

To flush means to hand a buffer's contents to the next level down. It does not mean the data reached a disk. The manual page says so in its own words: "Note that `fflush()` flushes only the user-space buffers provided by the C library. To ensure that the data is physically stored on disk the kernel buffers must be flushed too" [S3]. Do not write *save*, *commit* or *persist* for this operation. All three claim a durability that flushing does not provide [inference].

The **page cache** is level 3. It is the operating system's own buffer for file data and it lives in the machine's memory. It is not the disk's cache — the disk has its own, one level further down [V20]. A busy machine shows very little free memory because the page cache has grown into it [V32]. That is correct behaviour rather than a leak [V32].

The stack is not write-only. The kernel also moves data upward, and the same levels work in both directions [inference]. Level 3 fetches pages ahead of a sequential reader, so "a sequential `read()` just happens to return faster because the data was already fetched" [V34]. Assignment 1 measures writes, so every demonstration in the lecture is a write. The author says the read direction aloud, so that nobody leaves with a one-way picture.

4.1 One rule instead of four facts

Every level takes a large block from the level below and hands out small pieces to the level above. It buys wholesale and sells retail, and that asymmetry is why every level holds a buffer [inference]. The source states the same rule: each level "hands out subdivisions of something it bought wholesale from the level below, and each level's unit of purchase is larger than its unit of sale" [V18]. Apply that to level 2. On one measured ext4 file the C library takes a buffer of 4096 bytes, which is that file's own block size [V7]. It then sells single bytes to your loop [V7]. The gap between those units is the saving measured in section 1 [inference].

The rule travels to memory allocators, processor caches and network stacks [speculative]. That is why I teach the rule rather than the four facts. I deliberately offer no physical-world image for the page cache. A container image reinstalls the very intuition I am trying to remove [speculative].

4.2 One program, one buffer — C and Python take different paths

Each program passes through at most one user-space buffer, and the two programs do not use the same one. The C program, `demo/iodemo-c/iodemo.c`, calls `fwrite()`. That call fills the C library's `stdio` buffer, which is level 2. One `write()` then empties that buffer into the page cache [S3] [V7]. The Python program, `demo/iodemo-python/iodemo.py`, calls `f.write()`. That call fills a `BufferedWriter`, which is level 1. PEP 3116 named that class and specified the layers in 2007 [S8]. One `write()` empties that buffer into the page cache [S6].

The four-level table has a wrong reading, and I want to close it off. A Python write does not fall through level 1 into level 2. "CPython reimplements the same policy above `libc` rather than inheriting it" [V6]. A Python program uses level 1. A C program uses level 2.

The four levels are the union of the two paths, not the path of any one program. Each program uses at most one of levels 1 and 2, and never both [inference]. Both paths then meet in the kernel. A file opened with `buffering=0` uses neither [S6].

This was a design decision rather than an accident of implementation [inference]. PEP 3116 specified "a defined raw byte-based I/O interface" with separate raw, buffered and text layers in 2007, independently of C `stdio` [S8].

Windows does not add a level between 1 and 2. CPython's `FileIO.write` "Only makes one system call" and calls `_Py_write` [S26]. `_Py_write` calls `write()`, and the `MS_WINDOWS` branch calls the same `write` as the other branch [S27]. On Windows that `write` is the C runtime's low-level entry point, and that is not the buffered `stdio` layer. Microsoft describes the low-level routines as functions that "invoke the operating system directly for lower-level operation than that provided by stream I/O" [S28]. The same page states that "Low-level input and output calls don't buffer or format data" [S28]. PEP 3116 places Python's own buffer in the layer above. It describes the raw layer below as methods that "are wrappers around the appropriate operating system calls" [S8]. So levels 1 and 2 are the same two levels on both platforms [inference]. This says nothing about levels 3 and 4, which the rest of this section treats separately.

The sizes are the evidence a reader can check. Python documents a typical buffer of 128 kilobytes [S6]. On one measured machine, "On an ext4-backed file with `st_blksize == 4096` the allocated `stdio` buffer is therefore 4096 bytes, not the 8 KiB commonly

quoted" [V7]. That is a measurement of one machine, and the 128 kilobytes above is documentation rather than a measurement. Two adjacent levels, two sizes a factor of thirty-two apart, and neither number appears in the source code [inference].

4.3 The bottom level, and the limit on the thesis

Level 4 acknowledges early too. "most HDDs and many SSDs acknowledge a write once it lands in an onboard DRAM cache, before it reaches NAND or platter; a power cut in that window loses it" [V20]. The specification is permissive about the repair: on `fsync`, POSIX states that "It is explicitly intended that a null implementation is permitted" [S13].

This is where my thesis stops. No trace separates a device that flushed from one that only reported it — the note's own wording is "No syscall trace can distinguish an honest flush from a fast lie" [V5]. Everything above level 4 is observable with a tool a student can run. Level 4 is not observable from software at all [inference].

4.4 The demonstration, and what it does not prove

```
import os
print("hello")
os._exit(0)
```

Run to a terminal, this prints. Redirected to a file, the file is empty. The rule sits in the C language standard rather than in the operating system: standard output is "fully buffered if and only if the stream can be determined not to refer to an interactive device" [S5]. A terminal is interactive, so the line goes out. A file is not, so the line waits [inference]. The third statement then ends the process without cleanup — the standard permits termination paths that "need not close all files properly" [S5]. A trace of the program holds no `write` call at all [inference].

The demonstration proves level 1 exists and can lose data. It proves nothing about levels 3 and 4. "the honest classroom demo is a power-cut, not a `kill -9`. Killing the process loses only the *runtime's* buffer; the kernel still writes back." [V14] A student who tests with a `kill` watches the file survive. The experiment does not support the conclusion that flushing was enough [inference].

4.5 `ltrace`: the same idea, one layer up

I introduce the second tracer here, because I use it in the next subsection.

What it does. Its manual page calls it "A library call tracer" and states that it "intercepts and records the dynamic library calls which are called by the executed process and the signals which are received by that process. It can also intercept and print the system calls executed by the program" [S31]. The same page states that "Its use is very similar to `strace(1)`" [S31]. So this is the idea section 2 built, moved one layer up. `strace` attaches at the system call. `ltrace` attaches at the call into the shared library [inference].

Where it comes from. The project page states that "ltrace was originally authored by Juan Cespedes" and gives 0.7.3 as the latest stable version at its 2019 revision [S32]. The manual page's AUTHOR section names both Juan Cespedes and Petr Machata [S31], so Machata is a later maintainer rather than a co-author of the original [inference]. On installation the same page says "Your Linux distribution will probably have it available", and gives the command for Debian and Ubuntu as `apt-get install ltrace` [S32]. Neither page gives a licence or a first-release year, so I leave both out.

How to use it.

```
ltrace ./iodemo      # the C library calls: fopen, fwrite, fclose
ltrace -S ./iodemo   # library calls and the system calls between them
ltrace -c ./iodemo   # a count for each library call
```

The manual page documents `-S` as "Display system calls as well as library calls" and `-c` as "Count time and calls for each library call and report a summary on program exit" [S31]. You install it in the lab. You then run both tracers on one program, which is the exercise I set up in the next subsection.

4.6 Which boundaries a tool can actually watch

- **Application to library** — the tool above [V10]. "`printf` is visible to `ltrace` and `write` to `strace`, and a deadlock entirely inside a library is invisible to `strace` because the kernel never hears about it". For `iodemo` the library call is `fwrite` rather than `printf`, and the `write` underneath it is the same [inference]. That pair makes takeaway 2 falsifiable rather than asserted [inference].
- **Library to kernel** — where the system call tracer attaches [S1].
- **Kernel to device** — block-layer tracing [V30], which I do not teach in this course.
- **Device to medium** — not observable from a system-call trace [V5].

The justification for the whole approach sits one level up from any of these tools. No level reports which one holds the data, so counting the crossings is the only way to find out: "You cannot deduce which layer is lying; you have to make the boundary crossings visible and count them" [V16].

5 4. The same write, at three levels

LEVEL	PYTHON	C	BUFFERED?
Application	<code>f.write(b"x")</code> [S6]	—	yes [S6]
C library	—	<code>fwrite(...)</code> [S3]	yes [S3]
System call	<code>os.write(...)</code> [S20]	<code>write(...)</code> [S2]	no [inference]

Each level holds a buffer, or it passes the bytes straight down. That is the mechanism I measured in section 1, written out as an interface [inference].

A **file descriptor** is the small integer a Linux program uses to name an open file in system calls. It is the `= 3` in the trace line above, and it reappears as the first argument of every later call on that file [S1].

5.1 The demo program

`demo/iodemo-c/iodemo.c` is short. It opens a file that does not exist. It then opens the file we mean to write. It makes three `fwrite` calls totalling fourteen bytes. It closes the file and checks the return value of the close.

The trace shows **one** `write` call. Three source lines, fourteen bytes, one system call — takeaway 2 as an artefact rather than a claim [inference]. The bytes are adjacent, so the buffer held all of them and handed them down together [inference]. The write does not happen at the third `fwrite`. It happens when `fclose` runs, because `fclose` hands the buffer down to the kernel [inference].

Two deliberate design choices in that program are worth stating.

It prints nothing. Every `write` line in its trace then comes from the three `fwrite` calls, and not from a print statement [inference]. A silent program therefore makes a trace that is easier to read [speculative]. A Windows console may take a different path from file I/O. No source in this registry establishes that, so the author does not assert it.

The first open is meant to fail. Without a failure both traces are success all the way down, and the most informative platform difference never appears. Both design choices are the author's own stipulations about a program he wrote, so neither carries a citation. Whether silence makes the trace easier to read is his judgement rather than a stipulation, so that sentence carries a marker.

5.2 `fwrite` takes a size and a count

`fwrite("three\n", 1, 6, f)` passes six items of one byte each. The return value counts items, not bytes, and equals a byte count only when the item size is 1 [S21]. Fewer than requested is legal at the system call below: "Note that a successful `write()` may transfer fewer than count bytes" [S2]. The library call has its own version of that contract, reporting fewer items than requested, and the two are not the same guarantee [inference]. Ignoring either return value is the difference between having written the bytes and believing you did [inference].

6 5. Two operating systems, side by side

6.1 Process Monitor: what the kernel did

What it does. Microsoft describes it as "an advanced monitoring tool for Windows that shows real-time file system, Registry and process/thread activity" [S18]. Read that list closely. That is a filesystem-and-registry view of the machine, and it is not a list of the system calls a program made [inference].

Where it comes from. The page sits in Microsoft's Sysinternals documentation and is bylined "By Mark Russinovich" [S18]. The tool "combines the features of two legacy Sysinternals utilities, Filemon and Regmon" [S18]. You download it from that page. You can also run it from Sysinternals Live without installing anything [S18]. The page gives the supported platforms as "Runs on: Client: Windows 11 and higher. Server: Windows Server 2019 and higher" [S18]. It also links a Linux build, "Download Procmon for Linux (GitHub)" [S18]. The page gives no licence and no first-release year, so I leave both out.

How to use it. It is a graphical tool, so the steps replace a command line.

```
Procmon.exe                # it starts capturing at once
Filter → Process Name is iodemo.exe # keep one process
Operation column           # CreateFile opens, WriteFile writes
```

Those operation names are what I expect the tool to display in that column [speculative]. The page does not list them. Check them on the lab machine.

6.2 NtTrace: a tracer for the Windows Native API

What it does. Its project page states that "NtTrace provides a simple trace facility for the Windows Native API. It is roughly equivalent to strace on Linux" [S10]. Roger Orr wrote that comparison about his own tool, so the reader should know whose sentence it is. The same page states that "NtTrace uses the debugging interface on Windows to intercept the returns from the native API and display the input arguments and return code" [S10]. The consequence, in the author's reading note: it traces "via the debug interface so the tool needs no driver and no administrator rights" [V21].

And what it cannot see. The same note names two blind spots, and both follow from that attachment point. First, "driver-originated kernel calls and any direct `int 2e / syscall / sysenter` stub are invisible" [V47]. Anything that skips `ntdll` skips the tool [V47]. Second, "the debug interface is event-based, so simultaneous calls from multiple threads are serialised in the trace" [V48]. So "the observed interleaving is an artefact of the tracer, not of the program" [V49]. Neither blind spot affects `iodemo`, which is single-threaded and calls through `ntdll` [inference].

Where it comes from. Roger Orr wrote it and still maintains it. The project page lists releases from "Version 967 - 17-Nov-2011" to February 2026 [S10]. The page also states the licence first-party: "The NtTrace source code is available under the MIT license" [S10]. GitHub describes the repository as "An strace-like program for the Windows 'native' API" [S33]. Each release ships "pre-built NtTrace64.exe and NtTrace86.exe binaries" [S10], so you compile nothing and install nothing [inference].

How to use it.

```
NtTrace iodemo.exe           # every native call, one line each
NtTrace -filter File iodemo.exe # calls whose names contain "File"
NtTrace -only iodemo.exe      # this process, not its children
```

The page's own worked example is `NtTrace -filter File cmd` [S10]. Its syntax list documents `-filter` as a "Comma delimited list of substrings to filter on" [S10]. It documents `-only` as "Only debug the first process, don't debug child processes" [S10].

6.3 Why NtTrace and not Process Monitor

The Windows tool is NtTrace rather than Process Monitor. Those Process Monitor operations are filesystem events rather than system calls [inference]. Comparing that tool with a system call tracer would attribute a tool difference to the platform [inference]. NtTrace watches the native API, which its project page calls "the interface between the application space and the OS kernel", provided by `ntdll.dll` [S10]. **That is not the**

same attachment point as `strace`, which sits at a documented kernel boundary. That gap between the two attachment points is what I want you to take from this section [inference].

The layer above the native API is Win32, the documented interface a Windows program calls [S19]. The 32 is history rather than a width [inference]. Microsoft renamed the interface the Windows API. The new name "more accurately reflects its roots in 16-bit Windows and its support on 64-bit Windows" [S22].

Needing no driver and no administrator rights is what makes NtTrace usable here at all [V21]. The alternative on Windows is a kernel-mode tracer, which needs a signed driver and a boot-time change to signature enforcement. A teaching lab does not grant either, so nobody can run that tool [inference].

The deck reads four lines from the author's own capture of the course's own program. The capture traces `iodemo.exe` `iodemo.out` under `NtTrace64.exe` on 15 September 2026, with the filter the makefile uses [S36]. It runs to 45 lines and ships with this deck as `demo/iodemo-c/NtTrace64-IODEMO-Output.txt`, so a reader can check the excerpt against the whole file [S36]. The four lines are the last four traced calls. Three non-call lines follow them, and every call above them is the loader's [S36]. The loader opened and closed its own files before `main` ran [S36] [inference]. The slide keeps every name, every value and every return code, and the three dots mark the arguments it cuts [S36].

The author also keeps a second capture, of the Windows command shell rather than `iodemo` [S34]. It supplies one line the program's own two opens cannot: an object-manager path [S34] [S36].

One reading habit transfers badly from Linux here. A return of `=> 0` on these calls means success, and a return value that starts `0xc` is a failure code [S36] [inference].

A handle is not a file descriptor. Our program's open returns `FileHandle=...` [`0x124`] on Windows [S36]. The same open in the Linux container returns `= 3` [S37]. A handle is an opaque value naming an open object, and the object need not be a file. A file descriptor is a small integer that names an open file, and `strace` prints it as the return value of the open [S1]. They occupy the same position in a trace and are different mechanisms [inference]. The two are not returned the same way, either. Linux carries the descriptor in the return value [S37]. Windows returns a status and hands the handle back through an argument [S36] [inference].

Two error namespaces arrive in one return value. The program's deliberate first `fopen` fails on both systems. On Windows the trace carries `=> 0xc0000034` [`2 'The system cannot find the file specified.'`] [S36] — an `NTSTATUS` together with the Win32 error it translates into [inference]. The capture prints both values and names neither

namespace. The identification is mine. On Linux the same open returns `-1 ENOENT` [S37]. Linux has one `errno` and nothing to translate, because only one layer names errors there [inference].

The object path shows the layer. "The `\\??\\` prefix is the object-manager namespace, which a Win32 program never types. This is the layer difference, visible in one string." [V22] A Win32 program never types that prefix, and the trace shows it because the tracer sits below Win32, at the layer that does [inference]. The deck takes that string from the shell capture `(NtQueryAttributesFile(ObjectAttributes=\\??\\C:\\WINDOWS\\system32\\c_852.nls", ...) => 0)` and says so on the slide [S34]. Our own program's two opens carry no such prefix, because both name the file relative to an open directory [S36] [inference]. The loader's calls above the breakpoint do carry the prefix, on the executable and on a system database [S36]. The deck reads the shell's line instead, because that one names an ordinary file [S34].

6.4 Why the boundaries sit where they do

Microsoft states its own policy: the NT entry points are "internal to the operating system and subject to change from one release of Windows to another" [S9]. Linux treats the system call interface as a public contract. The accurate summary is neither a ranking nor a complaint: "Both are stable at the layer they nominate. The disagreement is about *which layer is nominated*" [V8].

Why the Windows boundary sits where it does is contested, and the author does not settle it [inference]. One reading is design. NT had three operating environments: POSIX, OS/2 and Win32 [S19]. Each served its own clients through its own interface. Each reached NT through the native API [S19]. An unstable lower layer is then a consequence rather than an accident [inference]. The other reading is vestige: Windows had a POSIX subsystem built "to meet 1980s era DoD Orange Book and FIPS specifications to sell to the defense market", and "The team focus shifted to Win32 so much that the Client-Server Runtime Subsystem (CSRSS) that hosts the Win32 API became mandatory and OS/2 and POSIX subsystems were never really completed" [S15]. That second account is one practitioner's recollection on a personal blog rather than a first-party history [S15]. Teach the mechanism as settled and the intent as contested [speculative]. Linux is the clearer case: it "historically could not control userspace... so it had to make the kernel boundary itself the stable interface" [V38].

And the tempting conclusion is backwards. "Linux is the only mainstream system that treats the raw syscall interface as a public contract." [V9] FreeBSD, OpenBSD, NetBSD, Solaris/illumos and macOS "all behave like Windows here" [V43], so they publish no equivalent promise [V9]. **I found more detail on FreeBSD in my notes:** it "offers strong

ABI compatibility *in practice* via compiled-in compatibility layers, while declining to state a formal cross-major-release guarantee" [V36]. The list does not claim five equally unstable systems [V36]. What the other four do in practice is not established here [inference]. Say *different*, never *worse* [inference].

6.5 One program, two kernels

One program, one file, fourteen bytes. The author recorded both traces on 15 September 2026 [S36] [S37]. Read across the rows rather than down the columns. Each row is the same step of the same source file.

STEP	LINUX [S37]	WINDOWS [S36]
open that fails	<code>openat(AT_FDCWD, "no-such-file.txt", O_RDONLY) = -1 ENOENT</code>	<code>NtCreateFile(... ObjectAttributes=0x58:"no-such-file.txt", ...) => 0xc0000034 [2 '...']</code>
open	<code>openat(AT_FDCWD, "iodemo.out", O_WRONLY O_CREAT O_TRUNC, 0666) = 3</code>	<code>NtCreateFile(FileHandle=... [0x124], ... ObjectAttributes=0x58:"iodemo.out", ...) => 0</code>
write	<code>write(3, "one\ntwo\nthree\n", 14) = 14</code>	<code>NtWriteFile(FileHandle=0x124, ..., Length=0xe, ...) => 0</code>
close	<code>close(3) = 0</code>	<code>NtClose(Handle=0x124) => 0</code>

Row one is the deliberate failure. Linux returns `-1` and one error name [S37]. Windows returns a native code together with the Win32 error it translates into [S36] [inference]. Row two is the open. Linux hands back the file descriptor `3` [S37]. Windows hands back the handle `0x124` [S36]. Row three is the write. Fourteen bytes on Linux, and `Length=0xe` on Windows, which is the same fourteen [S36] [S37]. Three `fwrite` calls stand in the source, and one call appears in each trace. Both C libraries buffered them [inference]. Row four is the close, the same step under two names [S36] [S37]. Same program, same fourteen bytes, one write on each side. The program is the controlled variable here. What differs is the two operating systems, and the two tracers that print them [inference]. One number carries over from section 2 and needs reconciling. The

`--count` run reported seven write operations on Linux and one on Windows [S37]. Six of the seven were the loader's, so the program's own single write is the same on both sides [S37] [inference].

Thin ground, and it is about the tool rather than about the pair. NtTrace is one author's work [S10]. Microsoft documents those native entry points as "internal to the operating system and subject to change from one release of Windows to another" [S9]. So read the Windows column as the shape of the boundary. Do not read it as an interface a program may depend on [inference].

What the pair replaced. The author's working note on the sample he used to teach from records the defect plainly: "It is not a matched pair." [V23] This one is. Both captures ship in this deck's `demo/iodemo-c/` folder, and the first lab reruns them [S36] [S37]. The shell capture stays for two reasons. Its 275 unfiltered lines show how much a real Windows process does before it reaches your code [S34]. And it names an ordinary file by an object-manager path, which the filtered trace of `iodemo` does not [S34] [S36].

7 6. What we have not looked at

`write()` returns success and the bytes are not on a disk. "A successful return from `write()` does not make any guarantee that data has been committed to disk" [S2].

That is legitimate rather than broken. Saltzer, Reed and Clark gave the general form in 1984: "The function in question can completely and correctly be implemented only with the knowledge and help of the application standing at the end points of the communication system... (Sometimes an incomplete version of the function provided by the communication system may be useful as a performance enhancement.)" [S11] Every level in the stack is an instance of that parenthesis [inference].

When it is actually safe is the durability lecture's question. I deliberately leave it open today.

7.1 Buffering moves your errors

`fwrite` returned success because the bytes reached a buffer. No error from the device has reached the library yet [inference]. The failure surfaces where the library flushes, which means it arrives at `fclose`, attached to no particular write [inference]. You cannot handle the error where you caused it, because only the call that flushes can report it [inference].

This is why `demo/iodemo-c/iodemo.c` checks the return value of `fclose`. Almost no real code does [speculative]. The idiom that hides it is universal: "the most common durability idiom in every mainstream language — open a file, write, let a `with` block or `using` block or destructor close it — contains no durability step anywhere" [V4]. A Python `with` block does at least raise if the close fails [inference]. The two languages differ in what they force a programmer to notice [inference].

Do not retry. "Retrying the `close()` after a failure return is the wrong thing to do, since this may cause a reused file descriptor from another thread to be closed" [S4]. The same conclusion was reached one level down in 2018, at `fsync`, for a different reason. "The response was uniform across engines: PostgreSQL, InnoDB, and WiredTiger all replaced retry with **crash/abort** during 2018" [V40]. MongoDB's WiredTiger engine wrote the reason into its source comment: "we don't retry `fsync`-style calls and panic if a flush operation fails" [V24]. PostgreSQL, MySQL's InnoDB and MongoDB's WiredTiger are three database engines with three different architectures, and all three replaced the retry with a deliberate crash [V41]. That is how little a failed `fsync` tells you about the state you are in [inference].

Be exact about what that convergence is, because it is easy to overstate. It is three codebases with three architectures, not three separate discoveries. The WiredTiger comment quoted above cites the LWN report on the PostgreSQL case as its justification [V24]. A common trigger is therefore on the record [inference].

This is *error reporting*. *Error recovery* belongs to the durability lecture.

7.2 Networking and IPC

A socket on Linux is a file descriptor, and `read` and `write` work on it. "Linux publishes that handle as a **file descriptor** and makes the file-descriptor contract binding. Windows publishes it as an **opaque SOCKET** and makes the *Winsock API* the contract, even though the NT kernel underneath is also holding a file object" [V28]. That is section 5's pattern arriving a second time, in a place a student does not expect it [inference]. Pipes are file descriptors too, so one program can wait on a socket and a pipe at the same time: "Because sockets, pipes, terminals, `epoll` instances, `timerfd s` and `eventfd s` are all file descriptors, a single event loop can wait on all of them at once" [V29]. Depth belongs to the next lecture — the point here is only that networking is not a separate subject.

7.3 Honest limits

- **The device level is not observable from a system-call trace** [V5]. "No syscall trace can distinguish an honest flush from a fast lie".
- **Asynchronous I/O breaks the model** [V11]. "with `IORING_SETUP_SQPOLL` a kernel-side polling thread means steady-state I/O submission needs **no syscalls at all**". The tracer sees the setup and nothing after it [inference].
- **strace is not a production instrument** [V25]. It has a second and cheaper mode, selected with the `--seccomp-bpf` option. On the workload of rebuilding `strace` itself, plain `strace` costs 3.06× the untraced run. That mode costs 1.44× [V39]. A contributor disqualifies it anyway — `strace --seccomp-bpf` "still stops at each syscall of interest and should therefore not be used on production systems!" [V25]. Production Linux uses eBPF and production Windows uses event tracing [speculative]. I have no per-call figure for either, so I am not giving you one. `strace` and Nt-Trace watch the layer this lecture is about, which is why I use them.
- **Memory mapping and crash dumps** are named so that nobody leaves believing I/O is finished [speculative]. Each has its own lecture.

8 What follows

1. **Ask "accepted or done?" of every fast call** [V12]. A fast return reports that the next level accepted the bytes. It does not report that the work is complete. The same structure governs a message broker acknowledging a message [speculative].
2. **Count, do not reason** [V16]. "You cannot deduce which layer is lying; you have to make the boundary crossings visible and count them".
3. **Name the level** [inference]. "Where is my data" has four answers — your own buffer, the C library's buffer, the page cache, and the device cache.
4. **Measure your own machine** [S7]. The default buffer size held at 8192 bytes for sixteen years, then moved by a factor of sixteen [S7].

8.1 Assignment 1

Assignment 1 measures takeaway 1 — buffered against unbuffered, sequential against random, over a 1 MiB file, with the explanation written in Markdown. The two programs from section 1 are the starting point [inference].

Report `io.DEFAULT_BUFFER_SIZE` and the Python version beside every number. Count under the tracer, in one run [V17]. Time the program without the tracer, in a second run [V17]. Say in the write-up that you did both.

The counting tool is Linux-only, so the counts come from the course dev container on either platform. When the count ratio and the time ratio disagree by orders of magnitude, explain the gap rather than assuming a mistake [inference].

9 References

- [S1] *strace(1)*. Linux man-pages project. <https://man7.org/linux/man-pages/man1/strace.1.html>
- [S2] *write(2)*. Linux man-pages project. <https://man7.org/linux/man-pages/man2/write.2.html>
- [S3] *fflush(3)*. Linux man-pages project. <https://man7.org/linux/man-pages/man3/fflush.3.html>
- [S4] *close(2)*. Linux man-pages project. <https://man7.org/linux/man-pages/man2/close.2.html>
- [S5] *C99 draft*, §7.19.3. Publicly hosted working draft. <https://busybox.net/~landley/c99-draft.html>
- [S6] *Built-in Functions — `open()`*. Python documentation. <https://docs.python.org/3/library/functions.html#open>
- [S7] *Increase `io.DEFAULT_BUFFER_SIZE`*. CPython issue 117151. <https://github.com/python/cpython/issues/117151>
- [S8] *PEP 3116 — New I/O*. Python Enhancement Proposals (2007). <https://peps.python.org/pep-3116/>
- [S9] *winternl.h reference*. Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>
- [S10] *NtTrace — Native API tracing for Windows*. Roger Orr, project page. <https://roger-orr.github.io/NtTrace/>
- [S11] Saltzer, J. H., Reed, D. P. and Clark, D. D. *End-to-End Arguments in System Design* (1984), p. 2. <https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf>
- [S12] Lampson, B. W. *Hints for Computer System Design* (1983), p. 19. <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/acrobat-17.pdf>
- [S13] *fsync*. The Open Group Base Specifications. <https://pubs.opengroup.org/onlinepubs/9699919799/functions/fsync.html>

- **[S14]** Taber, K. *Predict-Observe-Explain*. science-education-research.com. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>
- **[S15]** Reiter, B. *The sad history of the Microsoft POSIX subsystem* (2010). <https://brian-reiter.org/2010/08/24/the-sad-history-of-the-microsoft-posix-subsystem/>
- **[S16]** Soares, L. and Stumm, M. *FlexSC: Flexible System Call Scheduling with Exception-Less System Calls*. OSDI 2010. https://www.usenix.org/legacy/event/osdi10/tech/full_papers/Soares.pdf
- **[S17]** `io.IOBase.flush()`. Python documentation, `io` module. <https://docs.python.org/3/library/io.html#io.IOBase.flush>
- **[S18]** *Process Monitor*. Sysinternals, Microsoft Learn. <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>
- **[S19]** Russinovich, M. *Inside Native Applications*. Microsoft Learn, Sysinternals (2006). <https://learn.microsoft.com/en-us/sysinternals/resources/inside-native-applications>
- **[S20]** `os` — *Miscellaneous operating system interfaces*, `os.write()`. Python documentation. <https://docs.python.org/3/library/os.html#os.write>
- **[S21]** `fread(3)`, the page that documents `fwrite(3)`. Linux man-pages project. <https://man7.org/linux/man-pages/man3/fwrite.3.html>
- **[S22]** *Windows API index*. Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list>
- **[S23]** `IO_COUNTERS` (`winnt.h`). Microsoft Learn, Win32 API reference. https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-io_counters
- **[S24]** `GetProcessIoCounters` (`winbase.h`). Microsoft Learn, Win32 API reference. <https://learn.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-getprocessiocounters>
- **[S25]** `proc_pid_io(5)`. Linux man-pages project, release 6.19. https://man7.org/linux/man-pages/man5/proc_pid_io.5.html
- **[S26]** `Modules/_io/fileio.c`. CPython source, `main` branch. https://github.com/python/cpython/blob/main/Modules/_io/fileio.c
- **[S27]** `Python/fileutils.c`. CPython source, `main` branch. <https://github.com/python/cpython/blob/main/Python/fileutils.c>
- **[S28]** *Low-level I/O*. Microsoft Learn, Microsoft C runtime library. <https://learn.microsoft.com/en-us/cpp/c-runtime-library/low-level-i-o>
- **[S29]** `win32process.GetProcessIoCounters`. PyWin32 documentation, Mark Hammond. https://mhammond.github.io/pywin32/win32process__GetProcessIoCounters_meth.html

- **[S30]** *strace*. Project page. <https://strace.io/>
- **[S31]** *ltrace(1)*. Linux man-pages project. <https://man7.org/linux/man-pages/man1/ltrace.1.html>
- **[S32]** *ltrace*. Project page. <https://ltrace.org/>
- **[S33]** *NtTrace*. Source repository, Roger Orr. <https://github.com/rogerorr/NtTrace>
- **[S34]** *NtTrace64 capture of cmd.exe /c echo Hello World!*. PB158 course material, captured 2026-09-15; ships with this deck as `demo/NtTrace64-CMD-ECHO-Output.txt`. <https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/NtTrace64-CMD-ECHO-Output.txt>
- **[S35]** *iocount_win32.py — the recorded Windows run*. PB158 course material, run 2026-09-15; recorded verbatim in `demo/iocount-win32/README.md`. <https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/iocount-win32/README.md>
- **[S36]** *NtTrace64 capture of iodemo.exe*. PB158 course material, captured 2026-09-15; ships with this deck as `demo/iodemo-c/NtTrace64-IODEMO-Output.txt`. <https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/iodemo-c/NtTrace64-IODEMO-Output.txt>
- **[S37]** *iodemo on Linux and on Windows — the recorded runs*. PB158 course material, run 2026-09-15; recorded verbatim in `demo/iodemo-c/README.md`. <https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/iodemo-c/README.md>
- **[V1]** *A system call is not a function call* — the two-term cost model.
- **[V2]** *A system call is not a function call* — the measured footprint of one crossing.
- **[V3]** *A system call is not a function call* — `pread` and `pwrite` merge the seek.
- **[V4]** *A Successful write() Is a Handoff, Not a Save* — the idiom with no durability step.
- **[V5]** *A Successful write() Is a Handoff, Not a Save* — the device level is unobservable.
- **[V6]** *Standard output is buffered differently depending on where it points* — CPython above libc.
- **[V7]** *Standard output is buffered differently depending on where it points* — the measured 4096-byte buffer.
- **[V8]** *Windows and Linux place the stable syscall boundary at different layers* — the nomination argument.
- **[V9]** *Windows and Linux place the stable syscall boundary at different layers* — Linux as the outlier.
- **[V10]** *The System Call Boundary* — `ltrace` against `strace`.
- **[V11]** *The System Call Boundary* — `io_uring` with a polling thread.

- **[V12]** *Where the Abstraction Leaks* — accepted, not done.
- **[V13]** *Where the Abstraction Leaks* — trust the counts, not the times.
- **[V14]** *Where the Abstraction Leaks* — a kill is not a power cut.
- **[V15]** *Where the Abstraction Leaks* — a privilege transition, not a jump.
- **[V16]** *Where the Abstraction Leaks* — count, do not deduce.
- **[V17]** *Observing a Running System* — the 442-times worst case.
- **[V18]** *The Memory Hierarchy from Python to Physical Pages* — wholesale down, retail up.
- **[V19]** *Virtual Memory and the Page Cache* — name the cache level in every number.
- **[V20]** *File IO and Durability* — the device acknowledges early.
- **[V21]** *NtTrace — Native API tracing for Windows (Roger Orr)* — a debugger, not a driver.
- **[V22]** *NtTrace — Native API tracing for Windows (Roger Orr)* — the object-manager path prefix.
- **[V23]** *NtTrace — Native API tracing for Windows (Roger Orr)* — a trace of a different program is not a matched pair.
- **[V24]** *PostgreSQL's fsync() surprise* — never retry after fsync fails.
- **[V25]** *Every Tracing Mechanism Trades Observation Cost for a Blind Spot* — not for production.
- **[V26]** *The System Call Boundary* — the calls a tracer cannot see.
- **[V27]** *A system call is not a function call* — what merging the seek did to overall duration.
- **[V28]** *Sockets and Network I-O* — a Linux socket is a file descriptor, a Windows socket is not.
- **[V29]** *Sockets and Network I-O* — sockets, pipes and terminals are all file descriptors.
- **[V30]** *File IO and Durability* — block-layer tracing watches the kernel-to-device boundary.
- **[V31]** *Observing a Running System* — the published range for the same benchmark, on other machines.
- **[V32]** *Virtual Memory and the Page Cache* — the page cache is the machine's own memory.
- **[V33]** *File IO and Durability* — the kernel issues the device cache-flush command.
- **[V34]** *Virtual Memory and the Page Cache* — readahead moves data up the same four levels.
- **[V36]** *Windows and Linux place the stable syscall boundary at different layers* — the five systems do not decline the promise equally.

- **[V37]** *File IO and Durability* — `FlushFileBuffers` is the Windows counterpart of `fsync`.
- **[V38]** *Windows and Linux place the stable syscall boundary at different layers* — why Linux could not move its own boundary.
- **[V39]** *Every Tracing Mechanism Trades Observation Cost for a Blind Spot* — what the cheaper `seccomp-bpf` mode costs.
- **[V40]** *PostgreSQL's `fsync()` surprise* — three engines dropped the retry in 2018.
- **[V41]** *PostgreSQL's `fsync()` surprise* — the three engines are independent of each other.
- **[V43]** *Windows and Linux place the stable syscall boundary at different layers* — the five systems that behave like Windows.
- **[V44]** *A system call is not a function call* — the 2010 counts should not be quoted as current.
- **[V45]** *A system call is not a function call* — the cost model is graded confident, not verified.
- **[V46]** *A system call is not a function call* — the measured spread in per-crossing cost across hosts.
- **[V47]** *NtTrace — Native API tracing for Windows (Roger Orr)* — the calls that bypass `ntdll` are invisible.
- **[V48]** *NtTrace — Native API tracing for Windows (Roger Orr)* — the debug interface serialises threads.
- **[V49]** *NtTrace — Native API tracing for Windows (Roger Orr)* — the serialised order is the tracer's, not the program's.

10 Appendix — sources from the author's working notes

The talk cites material from a private research vault. Each entry below summarises one such note and lists every external source it draws on.

10.1 V1 — The cost structure of a system call

The note argues that a system call is a mechanically distinct event from a function call, not merely a slower one, and that its dominant cost is frequently paid after the call returns, in user-mode code whose processor state the crossing destroyed. From that reframing follows the note's central claim: total I/O cost is better modelled as `N_crossings × fixed_cost + bytes × marginal_cost` than as a function of bytes alone. For small payloads the first term dominates. The evidence is Soares and Stumm's FlexSC

measurements (USENIX OSDI 2010), which found a single `stat` call evicting 32 L1-instruction, 186 L1-data, 660 L2, and 2,559 L3 cache lines plus 21 TLB entries and dropping IPC to 0.37 — a footprint paid per crossing and largely independent of payload size, which is exactly what the two-term model captures. A direct demonstration holding bytes constant makes the same point empirically: writing 100,000 characters via unbuffered `write()` versus buffered `fputc()` moves identical data, but kernel time falls from roughly 1.510s to 0.002s because buffering collapses roughly 100,000 crossings into a few dozen — about a 300x difference driven purely by crossing count, not payload. The note also cites `pread / pwrite` replacing `lseek + read` as a second, syscall-choice-level instance of the same effect, and notes that both Linux (`readline()` over `io.DEFAULT_BUFFER_SIZE`) and Windows (`FileStreamOptions.BufferSize`) hide the same crossing-count knob behind an ordinary buffer-size default.

The note is explicit that this model has boundaries and is not a universal constant. It identifies several regimes where the "count crossings" heuristic breaks down: when there is no crossing at all, because Linux's vDSO and Windows' `KUSER_SHARED_DATA` serve some calls entirely in user space; when batching amortizes the fixed cost toward the marginal cost, as with `io_uring` and Windows I/O Rings, though batching itself is not free at batch size 1, where framework overhead can exceed the syscall it replaces; when the workload is device-bound rather than crossing-bound, so I/O latency rather than crossing count or bytes is the binding constraint; and at extreme call frequency, where the direct cost of the exception and pipeline flush takes over from the indirect cache/TLB-eviction cost the model is built on. The note states outright that the crossover point between these regimes is unquantified and has to be measured per workload rather than derived from the sources it cites. It also flags the Windows side of the cost story as comparatively unproven: the crossing mechanism is documented to the same depth as Linux's, but no comparable cache/TLB-eviction measurement exists for Windows, so the model's constants there are inferred rather than measured — and the argument as a whole is scoped to x86-64, with ARM64 and Apple Silicon left entirely uncovered.

Referenced sources

- [SYSCALL — Fast System Call \(Intel 64 and IA-32 Architectures Software Developer's Manual\)](#) — Intel Corporation **HIGH**
- [The Definitive Guide to Linux System Calls](#) — packagecloud **MEDIUM**
- [FlexSC: Flexible System Call Scheduling with Exception-Less System Calls](#) — Livio Soares and Michael Stumm, University of Toronto — USENIX OSDI (2010) **HIGH**
- [System calls on Windows x64](#) — n4r1b (2019) **MEDIUM**
- [Page Table Isolation \(PTI\) — The Linux Kernel documentation](#) — Linux kernel maintainers **HIGH**

- [KPTI/KAISER Meltdown Initial Performance Regressions](#) — Brendan Gregg, Netflix (2018) **MEDIUM**
- [Kernel page-table isolation](#) — Wikipedia **LOW**
- [What Makes System Calls Expensive: A Linux Internals Deep Dive](#) — Abhinav Upadhyay (2025) **MEDIUM**
- [Two frequently used system calls are ~77% slower on AWS EC2](#) — packagecloud **MEDIUM**
- [Randomizing the KUSER_SHARED_DATA Structure on Windows](#) — Microsoft Security Response Center (2022) **HIGH**
- [io_uring for High-Performance DBMSs: When and How to Use It](#) — Jasny et al. (2025) **MEDIUM**
- [SubmitIoRing function \(io_uring.h\)](#) — Microsoft Learn **HIGH**
- [One I/O Ring to Rule Them All: A Full Read/Write Exploit Primitive on Windows 11](#) — Yarden Shafir, Winsider Seminars & Solutions **MEDIUM**
- [io](#) — Core tools for working with streams (Python documentation) — Python Software Foundation **HIGH**
- [FileStreamOptions.BufferSize Property \(System.IO\)](#) — Microsoft Learn **HIGH**
- [Buffered I/O \(stdio library\) vs. Unbuffered I/O \(Sys. Calls\)](#) — circuitlabs (2025) **LOW**
- [15-213 Lecture 16 Activity Solutions: Unix I/O and Standard I/O](#) — Carnegie Mellon University, 15-213 Introduction to Computer Systems **HIGH**
- [Meltdown/Spectre Benchmarks](#) — Open Telekom Cloud Image Factory — T-Systems / Open Telekom Cloud **MEDIUM**
- [On the Costs of Syscalls \(2021\)](#) — Hacker News discussion — Hacker News commenters **LOW**
- [On the Costs of Syscalls](#) — Georg Sauthoff (2021) **MEDIUM**
- [Inspection of I/O Operations from System Call Traces using Directly-Follows-Graph](#) — Aravind Sankaran, Ilya Zhukov, Wolfgang Frings, Paolo Bientinesi (2024) **MEDIUM**
- [A Deep Dive into Zero-Copy Networking and io_uring](#) — Jatin Umamtora, Medium **LOW**
- [Comment thread on syscall cost and out-of-order execution](#) — Hacker News — commenter "anarazel" (2021) **LOW**

10.2 V2 — the true cost of a system call

This note argues that a system call is not a slower function call but a different mechanism, and it grounds that claim in a specific measurement: a single `stat` retires 4,972 instructions in 13,585 cycles — an IPC of 0.37 — while evicting 32 L1 instruction-cache lines, 186 L1 data-cache lines, 660 L2 lines, 2,559 L3 lines, and 21 data-TLB entries. That figure comes from Soares and Stumm's FlexSC paper, and the note treats it as the load-bearing evidence for its central reframing: the instruction count is not the story, because every one of those evicted cache lines is a miss that user-mode code pays for later, after the call has already returned, with work that has nothing to do with the kernel. Mechanically, the note traces this back to the `SYSCALL` instruction itself on x86-64 — used identically by Linux and Windows — which loads `RIP` from a model-specific register, does not preserve the stack pointer, and acts as a speculation barrier, none of which resembles an ordinary call. The practical consequence the note draws is that the number of kernel crossings, not the number of bytes transferred, becomes the first-order performance variable, which is why buffering and syscall-merging (`pread / pwrite` in place of `lseek + read`) change cost so much more than shrinking payloads does.

The note is explicit about where this reframing stops holding. It does not claim every kernel entry pays this cost: the vDSO on Linux and `KUSER_SHARED_DATA` on Windows serve some calls entirely in user space with no crossing at all, and batched interfaces like `io_uring` and Windows I/O Rings amortize the fixed cost across many operations, in which regimes the "just a slower call" model becomes approximately correct again. It also flags the FlexSC numbers as measured on 2010-era hardware, so the qualitative claim is treated as robust while the specific eviction counts are not presented as current. The note marks the magnitude of related costs, such as the KPTI page-table-isolation overhead, as genuinely contested across its sources rather than resolved to one number, and it states outright that no comparable cache/TLB-eviction measurement exists for Windows — the Windows mechanism is documented, but its cost is inferred from shared hardware behavior, not separately measured. ARM64 and Apple Silicon are described as entirely uncovered by the sources consulted.

Referenced sources

- [SYSCALL — Fast System Call \(Intel 64 and IA-32 Architectures Software Developer's Manual\)](#) — Intel Corporation **HIGH**
- [The Definitive Guide to Linux System Calls](#) — packagecloud **MEDIUM**
- [FlexSC: Flexible System Call Scheduling with Exception-Less System Calls](#) — Livio Soares and Michael Stumm, University of Toronto — USENIX OSDI (2010) **HIGH**
- [System calls on Windows x64](#) — n4r1b (2019) **MEDIUM**

- [Page Table Isolation \(PTI\) — The Linux Kernel documentation](#) — Linux kernel maintainers **HIGH**
- [KPTI/KAISER Meltdown Initial Performance Regressions](#) — Brendan Gregg, Netflix (2018) **MEDIUM**
- [Kernel page-table isolation](#) — Wikipedia **LOW**
- [What Makes System Calls Expensive: A Linux Internals Deep Dive](#) — Abhinav Upadhyay (2025) **MEDIUM**
- [Two frequently used system calls are ~77% slower on AWS EC2](#) — packagecloud **MEDIUM**
- [Randomizing the KUSER_SHARED_DATA Structure on Windows](#) — Microsoft Security Response Center (2022) **HIGH**
- [io_uring for High-Performance DBMSs: When and How to Use It](#) — Jasny et al. (2025) **MEDIUM**
- [SubmitIoRing function \(ioringapi.h\)](#) — Microsoft Learn **HIGH**
- [One I/O Ring to Rule Them All: A Full Read/Write Exploit Primitive on Windows 11](#) — Yarden Shafir, Winsider Seminars & Solutions **MEDIUM**
- [io](#) — Core tools for working with streams (Python documentation) — Python Software Foundation **HIGH**
- [FileStreamOptions.BufferSize Property \(System.IO\)](#) — Microsoft Learn **HIGH**
- [Buffered I/O \(stdio library\) vs. Unbuffered I/O \(Sys. Calls\)](#) — circuitlabs (2025) **LOW**
- [15-213 Lecture 16 Activity Solutions: Unix I/O and Standard I/O](#) — Carnegie Mellon University, 15-213 Introduction to Computer Systems **HIGH**
- [Meltdown/Spectre Benchmarks](#) — Open Telekom Cloud Image Factory — T-Systems / Open Telekom Cloud **MEDIUM**
- [On the Costs of Syscalls \(2021\)](#) — Hacker News discussion — Hacker News commenters **LOW**
- [On the Costs of Syscalls](#) — Georg Sauthoff (2021) **MEDIUM**
- [Inspection of I/O Operations from System Call Traces using Directly-Follows-Graph](#) — Aravind Sankaran, Ilya Zhukov, Wolfgang Frings, Paolo Bientinesi (2024) **MEDIUM**
- [A Deep Dive into Zero-Copy Networking and io_uring](#) — Jatin Umamtora, Medium **LOW**
- [Comment thread on syscall cost and out-of-order execution](#) — Hacker News — commenter "anarazel" (2021) **LOW**

10.3 V3 — System-call count as the first-order I/O cost driver

This note argues that a kernel crossing's dominant cost is not the work the kernel performs but the cache and TLB state the crossing evicts, which user-mode code then pays for after the call returns — a distinction it grounds in a peer-reviewed measurement showing a single `stat` call evicting thousands of cache lines and dozens of TLB entries, with the indirect cost dominating at moderate call frequency. Because that eviction cost is largely independent of payload size, the note's operational conclusion is that the number of crossings, not the number of bytes moved, is the variable to optimize first. The quoted span backs the sharpest applied instance of that conclusion: replacing separate seek-then-read/write calls with `pread64 / pwrite64` "combine[s] file access and seek operations into a single command... thereby reducing the number of system calls issued from the user environment," and a measured comparison on identical data volumes found this call-merging produced a real reduction in overall duration. The note treats this alongside two other concrete techniques documented for the same underlying reason — buffering, which absorbed 100,000 unbuffered write calls into a few dozen and cut system time roughly 300-fold for identical bytes, and batched submission via `io_uring` or Windows I/O Rings, which amortizes the fixed per-crossing cost toward its marginal cost.

The note scopes this claim narrowly and says so directly. It does not claim syscalls are always expensive relative to a function call: it documents cases where the crossing-count heuristic stops applying entirely, because a call is served from user space with no crossing to count at all — Linux's vDSO and Windows' `KUSER_SHARED_DATA` are given as structurally identical escape hatches on the two platforms. It also does not claim call-merging or batching is free: batching amortizes the fixed cost but adds its own overhead that can make it slower than the alternative at very small batch sizes, and at very high call frequencies the note states the direct cost of the exception and pipeline flush takes over from the indirect cache-eviction cost it otherwise emphasizes. The call-merging result itself rests on a single arXiv preprint with documented methodology but unconfirmed peer-review status. The note also flags that no comparable cache/TLB footprint measurement exists for Windows, that its cache/TLB mechanism claims are grounded only in x86-64 sources, and that the exact crossover point between these regimes is not derivable from its sources — it says this has to be measured per workload rather than assumed.

Referenced sources

- [SYSCALL — Fast System Call \(Intel 64 and IA-32 Architectures Software Developer's Manual\)](#) — Intel Corporation **HIGH**
- [The Definitive Guide to Linux System Calls](#) — packagecloud **MEDIUM**

- [FlexSC: Flexible System Call Scheduling with Exception-Less System Calls](#) — Livio Soares and Michael Stumm, University of Toronto — USENIX OSDI (2010) **HIGH**
- [System calls on Windows x64](#) — n4r1b (2019) **MEDIUM**
- [Page Table Isolation \(PTI\)](#) — [The Linux Kernel documentation](#) — Linux kernel maintainers **HIGH**
- [KPTI/KAISER Meltdown Initial Performance Regressions](#) — Brendan Gregg, Netflix (2018) **MEDIUM**
- [Kernel page-table isolation](#) — Wikipedia **LOW**
- [What Makes System Calls Expensive: A Linux Internals Deep Dive](#) — Abhinav Upadhyay (2025) **MEDIUM**
- [Two frequently used system calls are ~77% slower on AWS EC2](#) — packagecloud **MEDIUM**
- [Randomizing the KUSER_SHARED_DATA Structure on Windows](#) — Microsoft Security Response Center (2022) **HIGH**
- [io_uring for High-Performance DBMSs: When and How to Use It](#) — Jasny et al. (2025) **MEDIUM**
- [SubmitIoRing function \(ioringapi.h\)](#) — Microsoft Learn **HIGH**
- [One I/O Ring to Rule Them All: A Full Read/Write Exploit Primitive on Windows 11](#) — Yarden Shafir, Winsider Seminars & Solutions **MEDIUM**
- [io](#) — [Core tools for working with streams \(Python documentation\)](#) — Python Software Foundation **HIGH**
- [FileStreamOptions.BufferSize Property \(System.IO\)](#) — Microsoft Learn **HIGH**
- [Buffered I/O \(stdio library\) vs. Unbuffered I/O \(Sys. Calls\)](#) — circuitlabs (2025) **LOW**
- [15-213 Lecture 16 Activity Solutions: Unix I/O and Standard I/O](#) — Carnegie Mellon University, 15-213 Introduction to Computer Systems **HIGH**
- [Meltdown/Spectre Benchmarks](#) — [Open Telekom Cloud Image Factory](#) — T-Systems / Open Telekom Cloud **MEDIUM**
- [On the Costs of Syscalls \(2021\)](#) — [Hacker News discussion](#) — Hacker News commenters **LOW**
- [On the Costs of Syscalls](#) — Georg Sauthoff (2021) **MEDIUM**
- [Inspection of I/O Operations from System Call Traces using Directly-Follows-Graph](#) — Aravind Sankaran, Ilya Zhukov, Wolfgang Frings, Paolo Bientinesi (2024) **MEDIUM**
- [A Deep Dive into Zero-Copy Networking and io_uring](#) — Jatin Umamtora, Medium **LOW**

- [Comment thread on syscall cost and out-of-order execution](#) — Hacker News — commenter "anarazel" (2021) **LOW**

10.4 V4 — The Non-Durability of a Successful `write()` or `WriteFile()` Call

This note's central claim is that a successful `write()` on Linux or `WriteFile()` on Windows tells a program only that the kernel accepted its bytes, not that any storage device holds them. Both platforms state this in their own primary documentation: the Linux `write(2)` man page says outright that a successful return "does not make any guarantee that data has been committed to disk," and Microsoft describes `WriteFile` as writing to "an internal buffer that the operating system writes to a disk or communication pipe on a regular basis," with a cache-manager lazy-writer process deciding when that happens. From this the note draws its sharpest consequence: the most common durability idiom in every mainstream language — open a file, write, let a `with` block or `using` block or destructor close it — contains no durability step anywhere. `close()` is documented on Linux as not guaranteeing a disk save; `fflush()` only moves bytes from a language runtime's buffer into the kernel; Python's `close()` is documented as "flush and close," which reaches the same library boundary and no further. The note extends this pattern through Java, .NET, and traces it to a real consequence: PostgreSQL held an incorrect model of what a successful `fsync()` guarantees for roughly two decades, an error surfaced publicly in 2018 and fixed by making the database crash rather than trust a retried `fsync()`.

The note is explicit about where this claim stops holding. It is a default, not a law: opening a file with `O_SYNC / O_DSYNC`, or with Windows' write-through and no-buffering flags together, restores a per-write durability guarantee; a device cache backed by battery or capacitor is already durable once data reaches it; deliberately disabled write barriers on protected hardware trade the guarantee away on purpose; and engines such as PostgreSQL, SQLite, LMDB, and LevelDB have already built their own sync protocols on top of the gap, so an application built on one of them has delegated the problem rather than ignored it. The note also flags unsettled ground rather than presenting a tidy resolution: POSIX explicitly permits an `fsync()` implementation that does nothing at all, which puts it at odds with the stronger wording in the Linux man page; whether the kernel's practice of discarding dirty pages after a failed writeback is a bug or a necessity is recorded as an open, unresolved disagreement among named kernel and PostgreSQL developers; and the note lists several explicit knowledge gaps, including the lack of any citable measurement of how much data is actually non-durable at a random instant under default settings, and whether the note's own power-cut classroom experiment reproduces reliably on modern hypervisors.

Referenced sources

- [write\(2\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project **HIGH**
- [fsync\(2\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project **HIGH**
- [close\(2\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project **HIGH**
- [fsync — The Open Group Base Specifications Issue 7](#) — The Open Group / IEEE **HIGH**
- [ext4 General Information — The Linux Kernel documentation](#) — Linux kernel documentation project **HIGH**
- [File Caching — Win32 apps](#) — Microsoft Learn **HIGH**
- [FlushFileBuffers function \(fileapi.h\) — Win32 apps](#) — Microsoft Learn **HIGH**
- [fflush\(3\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project **HIGH**
- [CreateFileW function \(fileapi.h\) — Win32 apps](#) — Microsoft Learn **HIGH**
- [Querying for the Write Cache Property — Windows drivers](#) — Microsoft Learn **HIGH**
- [DTrace on Windows — Windows drivers](#) — Microsoft Learn **HIGH**
- [microsoft/DTrace-on-Windows](#) — Microsoft **HIGH**
- [_commit — C runtime library reference](#) — Microsoft Learn **HIGH**
- [os — Miscellaneous operating system interfaces \(Python 3 documentation\)](#) — Python Software Foundation **HIGH**
- [io — Core tools for working with streams \(Python 3 documentation\)](#) — Python Software Foundation **HIGH**
- [FileOutputStream \(Java SE 25 & JDK 25\)](#) — Oracle **HIGH**
- [FileStream.Flush Method — .NET API reference](#) — Microsoft Learn **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau, USENIX ATC (2020) **HIGH**
- [PANIC on fsync\(\) failure — PostgreSQL commit 6534d54](#) — Thomas Munro / PostgreSQL Global Development Group **HIGH**
- [PostgreSQL's fsync\(\) surprise](#) — Jonathan Corbet / LWN.net **MEDIUM**
- [Page Cache Writeback — Linux Kernel Internals](#) — kernel-internals.org **MEDIUM**
- [Fsync Errors — PostgreSQL wiki](#) — PostgreSQL Global Development Group **HIGH**
- [Linux Page Cache Basics](#) — Thomas-Krenn.AG wiki **MEDIUM**
- [Windows Write Caching — Part 2: An Overview for Application Developers](#) — winntfs.com **MEDIUM**
- [PostgreSQL Documentation: 28.1. Reliability](#) — PostgreSQL Global Development Group **HIGH**

- Disk write cache buffer and separate power supply — Super User (Stack Exchange)

LOW

- Fsyncgate: errors on fsync are unrecoverable — Dan Luu (archive of the pgsql-hackers thread, 2018)

MEDIUM

10.5 V5 — The Non-Guarantee Behind a Successful `write()` or `WriteFile()` Call

The note's claim is that a successful `write()` on Linux or `WriteFile()` on Windows tells a program only that the kernel accepted its bytes, never that any storage device holds them. It grounds this in primary documentation from both platforms: the Linux `write(2)` man page states directly that a successful return "does not make any guarantee that data has been committed to disk," and Microsoft describes `WriteFile` as writing to "an internal buffer that the operating system writes to a disk ... on a regular basis." The bytes sit in a page cache or cache-manager buffer and are pushed toward the device later, on a schedule set by timers, memory pressure, or an explicit sync request, not by the write call returning. The note follows this gap to the point where the claim is hardest to escape: even after a program calls the correct flush primitive — `fsync()` or `FlushFileBuffers()` — and the kernel reports success, whether the underlying device actually committed the data to persistent media, rather than merely acknowledging the flush request, is firmware behavior invisible to any operating-system API. PostgreSQL's own reliability documentation warns that many consumer drives do not honor cache-flush commands by default. From this the note draws its conclusion: no instrument available to software, including a syscall trace, can tell an honest flush apart from a device that reported completion without having done the work. That is the boundary the quoted sentence names.

The note is explicit that this is a default, not an absolute law. It lists the configurations where the claim stops holding: files opened with synchronous-write flags, a device cache that is battery- or capacitor-backed, write barriers deliberately disabled on hardware known to be protected, and application code sitting on an engine — a database, an embedded store — that already manages its own sync protocol underneath it. It also names unresolved territory rather than presenting a closed case: whether the kernel's practice of discarding dirty pages after a failed write-back is a bug or a necessity is an open, documented disagreement among kernel and PostgreSQL developers, and the note lists several explicit gaps of its own — no current measurement of how often writes are actually non-durable at a randomly chosen instant, no confirmed Windows equivalent to Linux's dirty-page counter, and no verification that the power-cut classroom experiment reproduces reliably on modern hypervisors.

Referenced sources

- [write\(2\) — Linux manual page](#) **HIGH**
- [fsync\(2\) — Linux manual page](#) **HIGH**
- [close\(2\) — Linux manual page](#) **HIGH**
- [fsync — The Open Group Base Specifications Issue 7](#) **HIGH**
- [ext4 General Information — The Linux Kernel documentation](#) **HIGH**
- [File Caching — Win32 apps](#) **HIGH**
- [FlushFileBuffers function \(fileapi.h\) — Win32 apps](#) **HIGH**
- [fflush\(3\) — Linux manual page](#) **HIGH**
- [CreateFileW function \(fileapi.h\) — Win32 apps](#) **HIGH**
- [Querying for the Write Cache Property — Windows drivers](#) **HIGH**
- [DTrace on Windows — Windows drivers](#) **HIGH**
- [microsoft/DTrace-on-Windows](#) **HIGH**
- [_commit — C runtime library reference](#) **HIGH**
- [os — Miscellaneous operating system interfaces \(Python 3 documentation\)](#) **HIGH**
- [io — Core tools for working with streams \(Python 3 documentation\)](#) **HIGH**
- [FileOutputStream \(Java SE 25 & JDK 25\)](#) **HIGH**
- [FileStream.Flush Method — .NET API reference](#) **HIGH**
- [Can Applications Recover from fsync Failures?](#) **HIGH**
- [PANIC on fsync\(\) failure — PostgreSQL commit 6534d54](#) **HIGH**
- [PostgreSQL's fsync\(\) surprise](#) **MEDIUM**
- [Page Cache Writeback — Linux Kernel Internals](#) **MEDIUM**
- [Fsync Errors — PostgreSQL wiki](#) **HIGH**
- [Linux Page Cache Basics](#) **MEDIUM**
- [Windows Write Caching — Part 2: An Overview for Application Developers](#) **MEDIUM**
- [PostgreSQL Documentation: 28.1. Reliability](#) **HIGH**
- [Disk write cache buffer and separate power supply](#) **LOW**
- [Fsyncgate: errors on fsync are unrecoverable](#) **MEDIUM**

10.6 V6 — Destination-dependent buffering of standard output

The note's central claim, stated in the quoted span, is that CPython does not simply inherit its stdout/stderr buffering behavior from the C library it sits on — it reimplements the same destination-dependent policy at its own layer. At the C level, glibc's

`_IO_file_doallocate` decides a stream's buffering mode once, at first use: it `fstat`s the file descriptor, and only if that shows a character device confirmed by `isatty` does it choose line buffering; a pipe or regular file gets full (block) buffering instead. CPython applies the identical interactive/non-interactive split at its own layer rather than delegating to libc: interactive `stdout` is line-buffered, non-interactive `stdout` is block-buffered, and `stderr` is line-buffered in both cases, per CPython's `sys` module reference. The note backs this with an executed check — on a pipe, `sys.stdout.line_buffering` reports `False` while `sys.stderr.line_buffering` reports `True` — offered as the reason Python's `logging` module, which writes to `stderr`, keeps producing visible output in a container while a bare `print()` to `stdout` can go silent. It also names the overrides available at this layer: the process-wide `-u` flag / `PYTHONUNBUFFERED` variable, and the per-call `print(..., flush=True)`.

The note is explicit about what this does not establish. Python's `stderr` became line-buffered rather than fully unbuffered only as of version 3.9; the note flags as contested whether `stderr` should be called "unbuffered" at all, since C99 requires only that it not be fully buffered, and Python deliberately implements the weaker guarantee. It states plainly that this Python-specific reimplementation does not transfer by analogy to other managed runtimes — Java, .NET, Node, and Rust are named as needing separate verification, since a runtime that issues its own `write()` calls owns its own flush policy independent of the C standard or C-level overrides like `stdbuf`. The note also records that `stdbuf`'s `LD_PRELOAD`-based injection is documented not to reliably affect CPython's own buffering layer, which is why `-u` is recommended instead. It makes no claim about the Windows UCRT's default `stdout` behavior for a console versus a redirected handle — that question is left an explicitly open gap, not answered by analogy from glibc.

Referenced sources

- [setbuf\(3\) — Linux manual page](#) — Michael Kerrisk / man7.org **HIGH**
- [The C99 Draft \(N869, 18 January 1999\), §7.19.3 Files](#) — ISO/IEC JTC1/SC22/WG14 (1999) **HIGH**
- [glibc source: libio/filedoalloc.c \(`\_IO\_file\_doallocate`\)](#) — GNU C Library, revision glibc-2.39-288 (2024) **HIGH**
- [Output is buffered and lost if the program aborts · Issue #1235 · actions/runner](#) — GitHub / actions-runner issue tracker (2021) **HIGH**
- [sys — System-specific parameters and functions: `sys.stdout` / `sys.stderr`](#) — Python Software Foundation **HIGH**
- [1. Command line and environment — the `-u` option](#) — Python Software Foundation **HIGH**
- [Why pipes sometimes get "stuck": buffering](#) — Julia Evans (2024) **HIGH**

- [How stdbuf works](#) — Harry Marr (2022) **MEDIUM**
- [stdio buffering](#) — Pádraig Brady **HIGH**
- [setvbuf — C runtime library reference](#) — Microsoft Learn (2020, msvc-170) **HIGH**
- [Issue 13601: sys.stderr should be line-buffered when stderr is not a TTY](#) — Python issue tracker (2011–2019) **HIGH**
- [libio: Always use 8192 bytes as the default buffer size \[BZ #4099\]](#) — Florian Weimer / glibc patchwork (2016) **HIGH**
- [Implementation-Defined ISO/IEC C99 Behavior](#) — Oracle Developer Studio C User's Guide — Oracle Corporation **HIGH**
- [Stdout Buffering](#) — Evan Klitzke **MEDIUM**
- [Buffering in pipes](#) — Thomas Habets (2008) **MEDIUM**
- [Why won't journalctl show logs when the program output is piped?](#) — Super User (community Q&A) **MEDIUM**
- [Kubernetes "logs" does not show any output for a simple python program running in a container in a pod](#) — Stack Overflow (community Q&A, 2021) **MEDIUM**
- [Rationale for International Standard — Programming Languages — C, revision 5.10](#) — ISO/IEC JTC1/SC22/WG14 (2003) **HIGH**
- [coreutils source: src/stdbuf.c](#) — GNU coreutils **HIGH**
- [SSH output isn't line buffered?](#) — Unix & Linux Stack Exchange (community Q&A) **MEDIUM**

10.7 V7 — Stdio buffer sizing on regular files

The note establishes that stdout's buffering mode and buffer size are chosen by the C library, not the operating system or the program, based on what file descriptor 1 refers to: line-buffered against a terminal, fully block-buffered against a pipe or regular file. It traces the size decision specifically to glibc's `_IO_file_doallocate`, which `fstat`s the descriptor and, for a regular file, uses `st_blksize` in place of the library constant `BUFSIZ` (8192) whenever `st_blksize` is the smaller of the two. The quoted claim is the note's correction of a widely repeated figure: on an ext4-backed file, where `st_blksize` is 4096, the allocated stdio buffer is 4096 bytes, not 8 KiB. The evidence is twofold — reading the pinned glibc 2.39 source for that allocator function, and a local reproduction on Ubuntu 24.04/glibc 2.39 that writes 6000 bytes to a redirected file and observes exactly 4096 bytes on disk mid-run, before the process sleeps and exits. The note reads that observation as the buffer filling and flushing once at 4096 bytes rather than at the as-

sumed 8192, and frames the correction as narrowing the folk figure rather than discarding it: 8192 remains real, as glibc's `BUFSIZ` constant and as the size actually used whenever `st_blksize` is not the smaller value.

The note is explicit about what this does not establish. It does not claim 4096 is a universal buffer size: `st_blksize` varies by filesystem and device, and the note leaves the case of `stdout` attached to a socket as an open question, noting only that the allocator's character-device test implies full buffering there without any source or measurement confirming a buffer size. It also leaves unresolved why the current glibc source tree still carries the `st_blksize < BUFSIZ` branch despite a 2016 patch proposing an unconditional 8192-byte default — the note states plainly that the commit reconciling the two was not located. The scope is glibc-specific: while the note treats destination-dependent buffering *mode* as a convention shared across C libraries, citing Oracle's runtime documentation as a second implementation, it does not extend the 4096-versus-8192 buffer-size mechanism to other libcs, and it states outright that none of this transfers by analogy to runtimes that bypass libc stdio, such as Go, Java, .NET, Node, or Rust.

Referenced sources

- [setbuf\(3\) — Linux manual page](#) — Michael Kerrisk / man7.org **HIGH**
- [The C99 Draft \(N869, 18 January 1999\), §7.19.3 Files](#) — ISO/IEC JTC1/SC22/WG14 **HIGH**
- [glibc source: libio/filedoalloc.c \(_IO_file_doallocate \)](#) — GNU C Library **HIGH**
- [Output is buffered and lost if the program aborts · Issue #1235 · actions/runner](#) — GitHub / actions-runner issue tracker **HIGH**
- [sys — System-specific parameters and functions: sys.stdout / sys.stderr](#) — Python Software Foundation **HIGH**
- [1. Command line and environment — the -u option](#) — Python Software Foundation **HIGH**
- [Why pipes sometimes get "stuck": buffering](#) — Julia Evans **HIGH**
- [How stdbuf works](#) — Harry Marr **MEDIUM**
- [stdio buffering](#) — Pádraig Brady **HIGH**
- [setvbuf — C runtime library reference](#) — Microsoft Learn **HIGH**
- [Issue 13601: sys.stderr should be line-buffered when stderr is not a TTY](#) — Python issue tracker **HIGH**
- [libio: Always use 8192 bytes as the default buffer size \[BZ #4099\]](#) — Florian Weimer / glibc patchwork **HIGH**
- [Implementation-Defined ISO/IEC C99 Behavior](#) — Oracle Developer Studio C User's Guide — Oracle Corporation **HIGH**

- [Stdout Buffering](#) — Evan Klitzke **MEDIUM**
- [Buffering in pipes](#) — Thomas Habets **MEDIUM**
- [Why won't journalctl show logs when the program output is piped?](#) — Super User (community Q&A) **MEDIUM**
- [Kubernetes "logs" does not show any output for a simple python program running in a container in a pod](#) — Stack Overflow (community Q&A) **MEDIUM**
- [Rationale for International Standard — Programming Languages — C, revision 5.10 — ISO/IEC JTC1/SC22/WG14](#) **HIGH**
- [coreutils source: `src/stdbuf.c`](#) — GNU coreutils **HIGH**
- [SSH output isn't line buffered?](#) — Unix & Linux Stack Exchange (community Q&A) **MEDIUM**

10.8 V8 — The layer at which Windows and Linux stabilize the syscall boundary

The note argues that Windows and Linux each promise application developers a stable interface to the kernel, but they nominate that stability at a different layer of the stack, and that this placement — not carefulness — is the whole disagreement: "Both are stable at the layer they nominate. The disagreement is about which layer is nominated." Linux fixes its promise at the syscall trap itself: syscall numbers are permanent, per-architecture entries in an in-tree table, and the kernel's own ABI documentation states the interface is "expected to never change and always be available." Windows fixes its promise one layer higher, at Win32; the Native API in `ntdll.dll` beneath it is explicitly disclaimed by Microsoft as "internal to the operating system and subject to change from one release of Windows to another," and the System Service Number beneath that is undocumented and is demonstrably renumbered between builds, and even between service packs within a single release. The note backs this with data recomputed directly from the canonical Windows syscall table: 52% of x64 functions decrease in number at least once across versions, and 63% of functions differ between two concurrently-shipping SKUs. So Windows guarantees a name where Linux guarantees a number, and everything downstream — where `open()` bottoms out, where a tracer attaches, what a stale hardcoded number does — follows from that one placement decision.

The note explicitly does not conclude that Windows is the unstable outlier; it argues the opposite — that Linux is the exception among Unix-family systems, since FreeBSD, OpenBSD, NetBSD, Solaris/illumos and macOS all decline a syscall-number guarantee much as Windows does. Several sub-questions are left unsettled rather than resolved: whether the Win32/`ntdll` split is deliberate design or a vestige of NT's abandoned multi-subsystem architecture; whether `ntdll` counts as a stable interface at all or only

a traced convention; whether Microsoft's disclosure of `winternl.h` prototypes was compelled by the 2002 antitrust settlement or was an independent decision; and whether Windows's syscall-number churn is purely mechanical, arising from alphabetical re-ordering of a service table, or partly deliberate. The note also flags that FreeBSD's classification as a no-guarantee platform is itself contested by a former core-team member, and that "Linux never breaks userspace" is a stronger claim than Linux's own stable-ABI document makes, since that document hedges its guarantee to two years and allows exceptions for security or grave-error fixes.

Referenced sources

- [ABI stable symbols — The Linux Kernel documentation](#) HIGH
- [j00ru/windows-syscalls — Windows System Call Tables](#) HIGH
- [Windows X86-64 System Call Table \(XP/2003/Vista/7/8/10/11 and Server\)](#) HIGH
- [NtQuerySystemInformation function \(winternl.h\) — Win32 apps](#) HIGH
- [Calling Internal APIs — Win32 apps](#) HIGH
- [arch/x86/entry/syscalls/syscall_64.tbl](#) HIGH
- [include/uapi/asm-generic/unistd.h](#) HIGH
- [Do other Unix-like kernels have stable syscall ABIs?](#) MEDIUM
- [FreeBSD Vendor Information](#) HIGH
- [openbsd-tech: system call origin verification](#) HIGH
- [all: stop using non-libc system calls on OpenBSD \(golang/go#36435\)](#) HIGH
- [syscall\(2\) — NetBSD Manual Pages](#) HIGH
- [port-solaris mailing list, November 2010: syscall number reservation](#) HIGH
- [NtTrace — Native API tracing for Windows](#) HIGH
- [strace — linux syscall tracer](#) HIGH
- [DTrace on Windows — Windows drivers](#) HIGH
- [DTrace — Windows Commands](#) HIGH
- [Inside the Native API](#) HIGH
- [NT OS/2 Subsystem Design Rationale \(NT Design Workbook\)](#) HIGH
- [The Sad History of the Microsoft POSIX Subsystem](#) MEDIUM
- [Syscall ABI compatibility: Linux vs Windows/macOS](#) MEDIUM
- [Never break userspace](#) HIGH
- [Updating Syscall Tables — SysWhispers4 documentation](#) LOW
- [CRT-free in 2023: tips and tricks — discussion](#) MEDIUM
- [Architecture of Windows NT](#) MEDIUM
- [U.S. v. Microsoft Corporation — Information on the Settlement](#) HIGH

• Syscalls on Windows 11 MEDIUM

10.9 V9 — Where Windows and Linux place the stable syscall boundary

This note argues that Windows and Linux each promise application developers a stable interface to the kernel, but nominate that stability at different layers, and that everything downstream — tracer attach points, whether a syscall number is safe to hardcode, what `open()` bottoms out in — follows from that one placement decision. Linux fixes the contract at the syscall trap itself: the kernel's own ABI documentation says syscall interfaces are "expected to never change and always be available," and the x86-64 syscall table backs this with fixed numbers (`open` = 2, `openat` = 257) that have held for the life of the architecture port. Windows nominates the contract one layer higher, at Win32; below it, `ntdll`'s Native API is explicitly disclaimed by Microsoft's own SDK documentation as "internal to the operating system and subject to change," and the raw System Service Numbers are undocumented and measurably renumbered — 52% of 506 functions decrease at least once across Windows versions, and two concurrently shipped SKUs disagree on 63% of shared function numbers. This supports the note's central, quoted claim: "Linux is the only mainstream system that treats the raw syscall interface as a public contract." That framing is deliberately narrower than "Windows is unstable" — the note surveys FreeBSD, OpenBSD, NetBSD, Solaris/illumos, and macOS, each of which explicitly declines a syscall-number guarantee in its own official statements, making Windows the normal case and Linux the exception needing explanation.

The note is explicit about what it does not claim. It does not say Windows is unstable in an absolute sense — both systems are stable at their chosen layer, and Windows guarantees Win32 names and semantics as firmly as Linux guarantees syscall numbers. Nor does it treat Linux's guarantee as absolute: it cites LWN's pushback on "we don't break userspace" as a retold slogan, and notes the kernel's own ABI document hedges its promise to two years and allows changes for security fixes. Several questions are left explicitly unsettled: whether the Win32/`ntdll` split is deliberate design or a vestige of NT's abandoned multi-subsystem architecture; whether `ntdll` counts as a stable interface merely because tools traffic in it, against Microsoft's own disclaimer; whether Microsoft's disclosure of `winternl.h` prototypes was compelled by the 2002 antitrust settlement, graded only "partially verified" since the correlation is temporal and no causal link was found; and whether FreeBSD truly belongs among the no-guarantee systems, given a core-team contributor's dissenting claim of de facto ABI stability via compiled-in compatibility flags. The note also flags that "Windows has no syscall tracer" is now only partly true, since DTrace ships in Windows Server 2025, off by default.

Referenced sources

- [ABI stable symbols — The Linux Kernel documentation](#) — *The Linux Kernel Organization* **HIGH**
- [j00ru/windows-syscalls — Windows System Call Tables](#) — *Mateusz "j00ru" Jurczyk* **HIGH**
- [Windows X86-64 System Call Table \(XP/2003/Vista/7/8/10/11 and Server\)](#) — *Mateusz "j00ru" Jurczyk* **HIGH**
- [NtQuerySystemInformation function \(winternl.h\) — Win32 apps](#) — *Microsoft* **HIGH**
- [Calling Internal APIs — Win32 apps](#) — *Microsoft* **HIGH**
- [arch/x86/entry/syscalls/syscall_64.tbl](#) — *Linux kernel source tree* **HIGH**
- [include/uapi/asm-generic/unistd.h](#) — *Linux kernel source tree* **HIGH**
- [Do other Unix-like kernels have stable syscall ABIs?](#) — *Unix & Linux Stack Exchange* **MEDIUM**
- [FreeBSD Vendor Information](#) — *The FreeBSD Project* **HIGH**
- [openbsd-tech: system call origin verification](#) — *OpenBSD project mailing list* **HIGH**
- [all: stop using non-libc system calls on OpenBSD \(golang/go#36435\)](#) — *The Go Project* **HIGH**
- [syscall\(2\) — NetBSD Manual Pages](#) — *The NetBSD Project* **HIGH**
- [port-solaris mailing list, November 2010: syscall number reservation](#) — *port-solaris list archive (Carnegie Mellon mirror)* **HIGH**
- [NtTrace — Native API tracing for Windows](#) — *Roger Orr* **HIGH**
- [strace — linux syscall tracer](#) — *The strace project* **HIGH**
- [DTrace on Windows — Windows drivers](#) — *Microsoft* **HIGH**
- [DTrace — Windows Commands](#) — *Microsoft* **HIGH**
- [Inside the Native API](#) — *Mark Russinovich, Sysinternals* **HIGH**
- [NT OS/2 Subsystem Design Rationale \(NT Design Workbook\)](#) — *Mark H. Lucovsky, Microsoft* **HIGH**
- [The Sad History of the Microsoft POSIX Subsystem](#) — *Brian Reiter (2010)* **MEDIUM**
- [Syscall ABI compatibility: Linux vs Windows/macOS](#) — *Mark Mossberg (2023)* **MEDIUM**
- [Never break userspace](#) — *LWN.net* **HIGH**
- [Updating Syscall Tables — SysWhispers4 documentation](#) — *SysWhispers4 project documentation* **LOW**
- [CRT-free in 2023: tips and tricks — discussion](#) — *Lobsters discussion thread* **MEDIUM**
- [Architecture of Windows NT](#) — *Wikipedia* **MEDIUM**

- [U.S. v. Microsoft Corporation — Information on the Settlement — U.S. Department of Justice, Antitrust Division](#) **HIGH**
- [Syscalls on Windows 11 — hammertux](#) **MEDIUM**

10.10 V10 — The System Call Boundary

The note argues that every operating system draws a "system call boundary" — the point where a request leaves a running program and enters the kernel — and that whether that boundary is also a permanent *contract* determines what debugging and tracing tools can and cannot see. It grounds this in Linux, where the numbered syscall interface is itself the stable, published contract, and where different tracing tools are shown to attach at different layers of that interface. `strace` intercepts at the kernel boundary itself, via `ptrace`, seeing every crossing into the kernel with decoded arguments. `ltrace` attaches one layer higher, at the dynamic linker, intercepting calls into shared libraries before they ever reach the kernel. The note uses this layering to ground the quoted claim precisely: `printf`, a C library function, is visible to `ltrace` because it is a library call, while the `write` syscall it may eventually trigger is what `strace` sees at the kernel boundary; a deadlock that occurs entirely inside a library's own logic never produces a kernel crossing, so `strace` has nothing to intercept and cannot reveal it. The claim is offered as evidence for a broader thesis running through the note: what a tool can observe is fixed by which layer it attaches to, not by how significant the underlying event is.

The note scopes this claim carefully rather than generalizing it. It does not claim `strace` is a complete instrument even within its own layer: elsewhere it notes that `strace`, via `ptrace`, stops the traced process twice per syscall, making it a poor timing instrument even where it is a reliable correctness one. It also does not extend the `strace`/`ltrace` picture to Windows, where it says there is no single stable numbered syscall list to trace against in the first place — the closest analogue, NtTrace, attaches to `ntdll` stubs rather than a kernel entry point, and Process Monitor sits a layer higher still, reporting semantic file and registry operations rather than a syscall list. Where the kernel boundary itself is bypassed — the Linux vDSO, which serves some calls from a shared page with no privilege crossing at all — the note treats absence from an `strace` trace as informative rather than definitive: if such a call unexpectedly does appear in a trace, that is read as a sign the fast path fell back to a real syscall, not as a contradiction. The specific `ltrace` / `strace` distinction is attributed to a single practitioner blog rather than primary kernel or vendor documentation, and the note does not claim this layering is enforced by anything beyond how each tool happens to be built.

Referenced sources

- [Anatomy of a system call, part 1](#) — David Drysdale / LWN.net **HIGH**
- [Anatomy of a system call, part 2](#) — David Drysdale / LWN.net **HIGH**
- [SYSCALL — Fast System Call \(x86 instruction reference\)](#) — Felix Cloutier, transcribed from the Intel SDM **HIGH**
- [x86-64 Syscall Entry](#) — Linux Kernel Internals — kernel-internals.org **MEDIUM**
- [The Linux Kernel Driver Interface \(stable-api-nonsense\)](#) — Greg Kroah-Hartman / Linux kernel documentation **HIGH**
- [Add kernel ↔ userspace ABI stability documentation](#) — Greg Kroah-Hartman / LWN.net **HIGH**
- [x86 Assembly / Interfacing with Linux](#) — Wikibooks contributors **MEDIUM**
- [Do other Unix-like kernels have stable syscall ABIs?](#) — Unix & Linux Stack Exchange **MEDIUM**
- [Libraries and Headers — Windows drivers](#) — Microsoft Learn **HIGH**
- [winternl.h header — Win32 apps](#) — Microsoft Learn **HIGH**
- [NTDLL Functions](#) — Geoff Chappell **HIGH**
- [Windows X86-64 System Call Table \(XP through Windows 11 and Server\)](#) — Mateusz "j00ru" Jurczyk **HIGH**
- [Microsoft Windows library files — NTDLL.DLL](#) — Wikipedia contributors **MEDIUM**
- [Windows X86-64 System Call Table — discussion thread](#) — Hacker News contributors **LOW**
- [strace — linux syscall tracer](#) — The strace project **HIGH**
- [ptrace and Syscall Interception](#) — Linux Kernel Internals — kernel-internals.org **MEDIUM**
- [Process Monitor — Sysinternals](#) — Microsoft Learn / Sysinternals **HIGH**
- [NtTrace — Native API tracing for Windows](#) — Roger Orr **MEDIUM**
- [What Makes System Calls Expensive](#) — Abhinav Upadhyay **MEDIUM**
- [vDSO and Virtual System Calls](#) — Linux Kernel Internals — kernel-internals.org **MEDIUM**
- [Two frequently used system calls are ~77% slower on AWS EC2](#) — packagecloud engineering **MEDIUM**
- [Why you should use io_uring for network I/O](#) — Red Hat Developers **HIGH**
- [Speed up open\(\).read\(\) pattern by reducing the number of system calls](#) — cpython#120754 — CPython issue tracker **HIGH**
- [win32file.CreateFile](#) — PyWin32 documentation — Mark Hammond / Tim Golden (PyWin32 docs) **MEDIUM**

- [ltrace: When strace Is Not Enough](#) — Satesh / unixy.io **MEDIUM**
- [System Calls and the User-to-Kernel Transition](#) — Yunolay **MEDIUM**
- [Windows vs. Linux Syscall Interfaces: Win32 API, NT Native API, and WSL](#) — kinda-technical.com **LOW**
- [In Linux x86_64, are syscalls and int 0x80 related?](#) — Stack Overflow **MEDIUM**
- [A Deep Dive Into Malicious Direct Syscall Detection](#) — Palo Alto Networks / Unit 42 **HIGH**
- [Linux kernel system calls table](#) — Marcin Juskiewicz **HIGH**
- [io_uring Interface Overview](#) — EmergentMind, summarizing Jasny et al. **LOW**

10.11 V11 — The System Call Boundary

The note's closing section, "Where the mechanism claim weakens," names `io_uring` as one of three real boundaries that qualify the lecture's central thesis — that a system call is a distinct hardware mechanism, not merely an expensive function call. It describes `io_uring` as moving submissions and completions into shared ring buffers so that a single `io_uring_enter` call can carry a large batch, and states that with `IORING_SETUP_SQPOLL` a kernel-side polling thread means steady-state I/O submission needs **no syscalls at all**, citing a Red Hat Developers article on using `io_uring` for network I/O as the grounding source. It pairs that with a published microbenchmark figure — per-submission cost falling from roughly 300 cycles unbatched to roughly 40 cycles at a batch of 16 — drawn from a secondary summary of an academic study, and marks that figure [reported] rather than verified. The note folds the whole case into its broader claim that where batching applies, per-crossing cost stops being the right unit of analysis, and that syscall count, not byte count, is what predicts I/O-bound performance, because per-crossing cost is largely independent of payload size.

The note does not claim `io_uring` eliminates the syscall mechanism itself — only that `SQPOLL` can remove syscalls from the steady-state submission path by trading them for a dedicated kernel-side polling thread, a substitution rather than a free elimination, and it does not spell out that thread's own cost. It flags the 300-to-40-cycle figure as coming from "an aggregator summarizing a primary benchmark study rather than the study itself," whose figures "have not been read at source here," so the number is offered as illustrative, not settled — consistent with the note's separate instruction to measure overhead on one's own hardware rather than quote published cycle counts. It presents this `io_uring` item as one of three caveats that scope the "syscall as distinct mechanism" thesis rather than overturn it, alongside the `vDSO` and the microarchitectural-cost discussion, and does not extend the claim into a general statement about `io_uring` or asynchronous I/O beyond the batching case described.

Referenced sources

- [Anatomy of a system call, part 1](#) — David Drysdale / LWN.net (2014) **HIGH**
- [Anatomy of a system call, part 2](#) — David Drysdale / LWN.net (2014) **HIGH**
- [SYSCALL — Fast System Call \(x86 instruction reference\)](#) — Felix Cloutier, transcribed from the Intel SDM (n.d.) **HIGH**
- [x86-64 Syscall Entry](#) — Linux Kernel Internals — kernel-internals.org (n.d.) **MEDIUM**
- [The Linux Kernel Driver Interface \(stable-api-nonsense\)](#) — Greg Kroah-Hartman / Linux kernel documentation (2004–present) **HIGH**
- [Add kernel ↔ userspace ABI stability documentation](#) — Greg Kroah-Hartman / LWN.net (2006) **HIGH**
- [x86 Assembly / Interfacing with Linux](#) — Wikibooks contributors (n.d.) **MEDIUM**
- [Do other Unix-like kernels have stable syscall ABIs?](#) — Unix & Linux Stack Exchange (2018–2023) **MEDIUM**
- [Libraries and Headers — Windows drivers](#) — Microsoft Learn (last updated 2022-07-12) **HIGH**
- [winternl.h header](#) — Win32 apps — Microsoft Learn (n.d.) **HIGH**
- [NTDLL Functions](#) — Geoff Chappell (ongoing) **HIGH**
- [Windows X86-64 System Call Table \(XP through Windows 11 and Server\)](#) — Mateusz "j00ru" Jurczyk (ongoing) **HIGH**
- [Microsoft Windows library files — NTDLL.DLL](#) — Wikipedia contributors (n.d.) **MEDIUM**
- [Windows X86-64 System Call Table — discussion thread](#) — Hacker News contributors (2019) **LOW**
- [strace — linux syscall tracer](#) — The strace project (ongoing) **HIGH**
- [ptrace and Syscall Interception](#) — Linux Kernel Internals — kernel-internals.org (n.d.) **MEDIUM**
- [Process Monitor — Sysinternals](#) — Microsoft Learn / Sysinternals (ongoing) **HIGH**
- [NtTrace — Native API tracing for Windows](#) — Roger Orr (ongoing) **MEDIUM**
- [What Makes System Calls Expensive](#) — Abhinav Upadhyay (2025) **MEDIUM**
- [vDSO and Virtual System Calls](#) — Linux Kernel Internals — kernel-internals.org (n.d.) **MEDIUM**
- [Two frequently used system calls are ~77% slower on AWS EC2](#) — packagecloud engineering (n.d.) **MEDIUM**
- [Why you should use io_uring for network I/O](#) — Red Hat Developers (2023) **HIGH**

- [Speed up open\(\).read\(\) pattern by reducing the number of system calls](#) — cpython#120754 — CPython issue tracker (2024) **HIGH**
- [win32file.CreateFile](#) — PyWin32 documentation — Mark Hammond / Tim Golden (PyWin32 docs, ongoing) **MEDIUM**
- [ltrace: When strace Is Not Enough](#) — Satesh / unixy.io (2026) **MEDIUM**
- [System Calls and the User-to-Kernel Transition](#) — Yunolay (2026) **MEDIUM**
- [Windows vs. Linux Syscall Interfaces: Win32 API, NT Native API, and WSL](#) — kinda-technical.com (2026) **LOW**
- [In Linux x86_64, are syscalls and int 0x80 related?](#) — Stack Overflow (2015–present) **MEDIUM**
- [A Deep Dive Into Malicious Direct Syscall Detection](#) — Palo Alto Networks / Unit 42 (2024) **HIGH**
- [Linux kernel system calls table](#) — Marcin Juskiewicz (regenerated from kernel source; table dated 2026-08-20 from 7.2.0 sources) **HIGH**
- [io_uring Interface Overview](#) — EmergentMind, summarizing Jasny et al. (updated 2026-01-17) **LOW**

10.12 V12 — The Syscall Boundary's Silent Scope Clause

The note argues that the central mismatch between what operating-system abstractions promise and what they actually do is that "success means 'accepted,' not 'done'": most fast OS interfaces return control once the kernel has taken responsibility for a request, not once the underlying work has happened. It grounds this in the file-write case: on Linux, `write()` only copies data into the kernel's page cache and marks it dirty, with writeback to the device happening asynchronously later, per the `fsync` man page's own description of what a subsequent `fsync` call actually flushes. Windows is presented as structurally identical — the OS buffers writes, and an application must call `FlushFileBuffers`, or use unbuffered I/O flags, to force data to disk, per Microsoft's own platform documentation. The note extends the same claim to closing a file handle: whether through `close()`, a destructor, a `using` block, or a Python `with` statement, releasing the handle does not push data to the device and can silently discard errors from earlier writes that only surface at close time. It treats this "accepted-versus-done" pattern as the single most consequential mismatch it documents, and as one instance of a broader rule that language runtimes add a second, usually invisible, layer of the same buffering behaviour on top of what the kernel already does.

The note is explicit that this describes current behaviour rather than a permanent specification, warning that cross-platform "Linux does X, Windows does Y" claims hold only for a given kernel version and API surface, and pointing elsewhere to mechanisms

that have genuinely moved, such as the removal of Windows User-Mode Scheduling. It does not claim `fsync` alone is sufficient for durability: it separately notes that `fsync` on a file does not guarantee the containing directory entry survives a crash, and that even a completed `fsync` depends on the underlying device being honest, which older kernels, some filesystems, and some disk drivers cannot be trusted to do. It also leaves one adjacent question explicitly unresolved — whether `mmap` outperforms `read` for sequential access — presenting it as a case where credible sources disagree and recommending that a reader measure their own access pattern rather than trust either published answer. A numeric estimate used elsewhere in the note to argue that per-crossing costs compound at scale is flagged by the note itself as not traceable to a primary measurement.

Referenced sources

- [close\(2\) — Linux manual page](#) — Michael Kerrisk / man7.org (2024) **HIGH**
- [fsync\(2\) — Linux manual page](#) — Michael Kerrisk / man7.org (2024) **HIGH**
- [Flushing System-Buffered I/O Data to Disk](#) — Microsoft Learn (2021) **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn (2023) **HIGH**
- [Durability: Linux File APIs](#) — Evan Jones (2020) **MEDIUM**
- [The Secret Life of fsync](#) — Andrey Pechkurov / puzpuzpuz.dev (2022) **MEDIUM**
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft DevBlogs (2017) **HIGH**
- [fsync, fdatsync, and O_SYNC](#) — kernel-internals.org (2025) **MEDIUM**
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation (2026) **HIGH**
- [Bypassing the GIL for Parallel Processing in Python](#) — Real Python (2024) **MEDIUM**
- [About Processes and Threads](#) — Microsoft Learn (2023) **HIGH**
- [Fibers](#) — Microsoft Learn (2023) **HIGH**
- [User-Mode Scheduling](#) — Microsoft Learn (2022) **HIGH**
- [The Libuv Thread Pool Trap: Why Node.js Async APIs Stall Under Load](#) — The Practical Developer (2026) **MEDIUM**
- [MapViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn (2022) **HIGH**
- [Which is fastest: read, fread, ifstream or mmap?](#) — Daniel Lemire (2012) **HIGH**
- [Latency implications of virtual memory](#) — Erik Rigtorp (2020) **MEDIUM**
- [Why is mmap\(\) Faster Than Sequential IO? Debunking Common Myths](#) — linux-vox.com (2025) **LOW**
- [File Access: Memory-Mapped vs. I/O System Call Performance](#) — Baeldung on Linux (2024) **MEDIUM**

- [strace\(1\) — Linux manual page — strace project / man7.org \(2024\)](#) **HIGH**
- [DTrace on Windows — Microsoft Learn \(2023\)](#) **HIGH**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations — Yarden Shafir / Winsider Seminars & Solutions \(2021\)](#) **HIGH**
- [Linux I/O Models — Blocking, epoll, io_uring, AIO, Signal-Driven — systeminternals.dev \(2026\)](#) **MEDIUM**
- [The Law of Leaky Abstractions — Joel Spolsky \(2002\)](#) **HIGH**
- [There Is a Mountain Lion in the Datacenter — HFT University \(2026\)](#) **LOW**
- [Sysinternals Suite 2026.19.08 — Neowin \(2026\)](#) **MEDIUM**
- [How Python's GIL Works — Codefinity \(2025\)](#) **LOW**

10.13 V13 — The trustworthiness gap between strace's call counts and its timings

This note's Case 5 concerns a profiler blind spot: a sampling CPU profiler only attributes time to the userspace stack it samples, so time genuinely spent blocked in the kernel gets pinned on whichever call happened to cross into it, not on the true cost, which is the number of boundary crossings. The proposed corrective is `strace -c`, which tabulates syscalls, errors, and time per call — but the note turns the same scrutiny on this fix. `-c` reports system time, CPU time spent inside the kernel, which is independent of wall-clock time; a syscall that blocks for 200ms shows almost no time under that flag. The flag that answers a latency question is `-w`, which sums wall-clock duration per call, and the note recommends running both on the same program to watch the two tables disagree. It then supplies the reason the disagreement exists at all: `strace` does not observe passively. It stops the traced process at every syscall to inspect it, and that stop is itself an intervention in the timing it is trying to report. This is the point the quoted span closes on — because the tracer pauses execution at each boundary crossing, the number and ordering of those crossings remain reliable, but the absolute durations it reports do not, since the act of measuring has already altered them.

The note does not claim `strace` is useless or that its timing output should be discarded outright — only that its two categories of output carry different evidentiary weight, and that treating a reported duration as ground truth repeats the diagnostic error the tool was invoked to correct in the first place. It does not attempt to quantify how much the tracing overhead distorts a given measurement, and it names no threshold past which a reported time becomes unusable; both are left as open, unresolved caveats rather than measured claims. The note frames this as one instance of a broader limitation it states explicitly elsewhere: observation is never free or neutral, since ETW sessions carry their own buffer and CPU cost and a debugger changes scheduling the same way

`strace` does, so no tool's absolute timings should be trusted as much as its counts and orderings. It treats this as a structural property of instrumentation itself, not a defect a better tool would remove.

Referenced sources

- [close\(2\) — Linux manual page](#) — Michael Kerrisk / man7.org (2024) **HIGH**
- [fsync\(2\) — Linux manual page](#) — Michael Kerrisk / man7.org (2024) **HIGH**
- [Flushing System-Buffered I/O Data to Disk](#) — Microsoft Learn (2021) **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn (2023) **HIGH**
- [Durability: Linux File APIs](#) — Evan Jones (2020) **MEDIUM**
- [The Secret Life of fsync](#) — Andrey Pechkurov / puzpuzpuz.dev (2022) **MEDIUM**
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft DevBlogs (2017) **HIGH**
- [fsync, fdatsync, and O_SYNC](#) — kernel-internals.org (2025) **MEDIUM**
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation (2026) **HIGH**
- [Bypassing the GIL for Parallel Processing in Python](#) — Real Python (2024) **MEDIUM**
- [About Processes and Threads](#) — Microsoft Learn (2023) **HIGH**
- [Fibers](#) — Microsoft Learn (2023) **HIGH**
- [User-Mode Scheduling](#) — Microsoft Learn (2022) **HIGH**
- [The Libuv Thread Pool Trap: Why Node.js Async APIs Stall Under Load](#) — The Practical Developer (2026) **MEDIUM**
- [MapViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn (2022) **HIGH**
- [Which is fastest: read, fread, ifstream or mmap?](#) — Daniel Lemire (2012) **HIGH**
- [Latency implications of virtual memory](#) — Erik Rigtorp (2020) **MEDIUM**
- [Why is mmap\(\) Faster Than Sequential IO? Debunking Common Myths](#) — linux-vox.com (2025) **LOW**
- [File Access: Memory-Mapped vs. I/O System Call Performance](#) — Baeldung on Linux (2024) **MEDIUM**
- [strace\(1\) — Linux manual page](#) — strace project / man7.org (2024) **HIGH**
- [DTrace on Windows](#) — Microsoft Learn (2023) **HIGH**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#) — Yarden Shafir / Winsider Seminars & Solutions (2021) **HIGH**
- [Linux I/O Models — Blocking, epoll, io_uring, AIO, Signal-Driven](#) — systeminternals.dev (2026) **MEDIUM**
- [The Law of Leaky Abstractions](#) — Joel Spolsky (2002) **HIGH**

- [There Is a Mountain Lion in the Datacenter](#) — HFT University (2026) **LOW**
- [Sysinternals Suite 2026.19.08](#) — Neowin (2026) **MEDIUM**
- [How Python's GIL Works](#) — Codefinity (2025) **LOW**

10.14 V14 — Leaky OS Abstractions at the Syscall Boundary

The note argues that a successful `write()` or file-write call means only that the kernel has accepted responsibility for the data, not that the data is durable — "accepted" versus "done." On Linux, `write(2)` copies bytes into the page cache and marks them dirty; write-back to the device happens later, asynchronously, so only `fsync(2)` (plus, for a newly created file, a separate `fsync` on the containing directory) forces the transfer. Windows is structurally the same: the OS buffers writes, and only `FlushFileBuffers`, or opening the file with `FILE_FLAG_NO_BUFFERING`, forces the disk I/O. The note draws the classroom implication directly from this: the honest way to demonstrate the gap is a power-cut, not a `kill -9`, because killing the process only discards the language runtime's own buffer — the kernel's page cache survives the process and still gets written back, so the demo would look like it succeeded when it hasn't tested the real risk. Only losing the machine exposes the layer that matters. This is backed by the Linux and Windows manual and API documentation for `close()`, `fsync()`, and `FlushFileBuffers`, together with a practitioner's `blktrace` measurements showing ext4 and xfs issuing a cache flush and a FUA write to the journal on `fsync`, and Microsoft's own note that some EIDE/SATA drivers historically ignored the FUA flag.

The note does not treat this gap as a bug to fix; its stance is that the leak is the abstraction "confessing what was always there," not a design flaw. Nor does it claim `fsync()` alone is sufficient: it notes that `fsync()` on a file does not guarantee the directory entry itself is durable, and that some older kernels or filesystems cannot flush the drive's own cache at all, in which case the drive's write cache has to be disabled. Several adjacent points are left explicitly unsettled or version-scoped rather than asserted as fact: a separate claim that a mode switch costs "hundreds of cycles" is flagged in the note as an aggregator-derived figure with no traceable primary measurement, and the note cautions more generally that its Windows/Linux comparisons and mechanism-level claims are correct for a given kernel and API surface, not timeless, and should be re-verified rather than trusted indefinitely.

Referenced sources

- [close\(2\)](#) — Linux manual page — Michael Kerrisk / man7.org (2024) **HIGH**
- [fsync\(2\)](#) — Linux manual page — Michael Kerrisk / man7.org (2024) **HIGH**
- [Flushing System-Buffered I/O Data to Disk](#) — Microsoft Learn (2021) **HIGH**

- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn (2023) **HIGH**
- [Durability: Linux File APIs](#) — Evan Jones (2020) **MEDIUM**
- [The Secret Life of fsync](#) — Andrey Pechkurov / puzpuzpuz.dev (2022) **MEDIUM**
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft DevBlogs (2017) **HIGH**
- [fsync, fdatsync, and O_SYNC](#) — kernel-internals.org (2025) **MEDIUM**
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation (2026) **HIGH**
- [Bypassing the GIL for Parallel Processing in Python](#) — Real Python (2024) **MEDIUM**
- [About Processes and Threads](#) — Microsoft Learn (2023) **HIGH**
- [Fibers](#) — Microsoft Learn (2023) **HIGH**
- [User-Mode Scheduling](#) — Microsoft Learn (2022) **HIGH**
- [The Libuv Thread Pool Trap: Why Node.js Async APIs Stall Under Load](#) — The Practical Developer (2026) **MEDIUM**
- [MapViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn (2022) **HIGH**
- [Which is fastest: read, fread, ifstream or mmap?](#) — Daniel Lemire (2012) **HIGH**
- [Latency implications of virtual memory](#) — Erik Rigtorp (2020) **MEDIUM**
- [Why is mmap\(\) Faster Than Sequential IO? Debunking Common Myths](#) — linux-vox.com (2025) **LOW**
- [File Access: Memory-Mapped vs. I/O System Call Performance](#) — Baeldung on Linux (2024) **MEDIUM**
- [strace\(1\)](#) — Linux manual page — strace project / man7.org (2024) **HIGH**
- [DTrace on Windows](#) — Microsoft Learn (2023) **HIGH**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#) — Yarden Shafir / Winsider Seminars & Solutions (2021) **HIGH**
- [Linux I/O Models — Blocking, epoll, io_uring, AIO, Signal-Driven](#) — systeminternals.dev (2026) **MEDIUM**
- [The Law of Leaky Abstractions](#) — Joel Spolsky (2002) **HIGH**
- [There Is a Mountain Lion in the Datacenter](#) — HFT University (2026) **LOW**
- [Sysinternals Suite 2026.19.08](#) — Neowin (2026) **MEDIUM**
- [How Python's GIL Works](#) — Codefinity (2025) **LOW**

10.15 V15 — The Leaky Boundary Between Application Abstractions and the Operating System

This note argues that abstractions over the operating system are contracts with a silent scope clause, and it identifies three properties that make OS abstractions leak in ways application-level abstractions do not. Central to the argument is the claim that the boundary a program crosses into the kernel is a privilege transition, not a jump: it is not a function call with a bigger prologue, and the return can be interrupted, delayed, or reordered relative to anything the caller thought it had sequenced. The note grounds this in `EINTR` on Linux, where the kernel surfaces a signal to userspace rather than hiding it. It develops this alongside two companion properties — that a fast OS call typically reports the work as "accepted" rather than "done" (illustrated by `write()` returning once bytes reach the page cache, not the device), and that language runtimes add a second layer of buffering on top of what the kernel already buffers. From these three properties the note derives a diagnostic method — observation rather than reasoning — and applies it across five cases: unflushed writes, unflushed closes, threads that are not OS threads, `mmap` performance, and syscall counts hidden from profilers. Each case follows a fixed shape (belief, reality, observation, cost) and is backed by primary-source documentation (Linux man pages, Microsoft Learn) plus practitioner measurements, treating Windows and Linux as structurally equivalent throughout.

The note does not claim these mechanisms are stable. It flags User-Mode Scheduling as a Windows primitive that existed and was later withdrawn, notes that `io_uring` changed what "syscall count" means on Linux, and states that CPython's GIL constraint is being unwound version by version — so any specific claim carries a version scope rather than being timeless. It treats the question of whether `mmap` is faster than sequential `read()` as genuinely unsettled among its own cited sources, reconciling the disagreement by attributing it to differing access patterns rather than declaring one side wrong. It also concedes that observation is not free or neutral — tracing tools perturb what they measure — and that some leaks, such as device write caches and NUMA topology, sit below what the operating system can abstract away at all, so "learn one layer down" has no fixed floor. One claim in the note, a mode-switch cost estimate used in the syscall-count case, is explicitly marked as ungrounded — sourced only from aggregator content with no traceable primary measurement — and the note recommends back-filling it or having students measure it directly rather than treating it as settled.

Referenced sources

- [close\(2\) — Linux manual page](#), Michael Kerrisk / man7.org (2024) **HIGH**
- [fsync\(2\) — Linux manual page](#), Michael Kerrisk / man7.org (2024) **HIGH**
- [Flushing System-Buffered I/O Data to Disk](#), Microsoft Learn (2021) **HIGH**

- [FlushFileBuffers function \(fileapi.h\)](#), Microsoft Learn (2023) **HIGH**
- [Durability: Linux File APIs](#), Evan Jones (2020) **MEDIUM**
- [The Secret Life of fsync](#), Andrey Pechkurov / puzpuzpuz.dev (2022) **MEDIUM**
- [Revised notes on the reliability of FlushFileBuffers](#), Raymond Chen / Microsoft DevBlogs (2017) **HIGH**
- [fsync, fdatsync, and O_SYNC](#), kernel-internals.org (2025) **MEDIUM**
- [os — Miscellaneous operating system interfaces](#), Python Software Foundation (2026) **HIGH**
- [Bypassing the GIL for Parallel Processing in Python](#), Real Python (2024) **MEDIUM**
- [About Processes and Threads](#), Microsoft Learn (2023) **HIGH**
- [Fibers](#), Microsoft Learn (2023) **HIGH**
- [User-Mode Scheduling](#), Microsoft Learn (2022) **HIGH**
- [The Libuv Thread Pool Trap: Why Node.js Async APIs Stall Under Load](#), The Practical Developer (2026) **MEDIUM**
- [MapViewOfFile function \(memoryapi.h\)](#), Microsoft Learn (2022) **HIGH**
- [Which is fastest: read, fread, ifstream or mmap?](#), Daniel Lemire (2012) **HIGH**
- [Latency implications of virtual memory](#), Erik Rigtorp (2020) **MEDIUM**
- [Why is mmap\(\) Faster Than Sequential IO? Debunking Common Myths](#), linux-vox.com (2025) **LOW**
- [File Access: Memory-Mapped vs. I/O System Call Performance](#), Baeldung on Linux (2024) **MEDIUM**
- [strace\(1\) — Linux manual page](#), strace project / man7.org (2024) **HIGH**
- [DTrace on Windows](#), Microsoft Learn (2023) **HIGH**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#), Yarden Shafir / Winsider Seminars & Solutions (2021) **HIGH**
- [Linux I/O Models — Blocking, epoll, io_uring, AIO, Signal-Driven](#), systeminternals.dev (2026) **MEDIUM**
- [The Law of Leaky Abstractions](#), Joel Spolsky (2002) **HIGH**
- [There Is a Mountain Lion in the Datacenter](#), HFT University (2026) **LOW**
- [Sysinternals Suite 2026.19.08](#), Neowin (2026) **MEDIUM**
- [How Python's GIL Works](#), Codefinity (2025) **LOW**

10.16 V16 — Leaky Operating-System Abstractions and How to Observe Them

"Where the Abstraction Leaks" argues every high-level OS abstraction — a Python file handle, a Java thread, an `mmap`'d buffer — is a contract with an unwritten scope clause: it states what a call does, not when that holds. The quoted claim is the note's central diagnostic stance: a call crossing into the kernel is a privilege transition, not a plain function call, and a fast OS interface returning success usually means the kernel accepted responsibility for the work, not that the work happened. Because the runtime, the kernel, and the device can each independently be lying about state, no amount of reasoning from the API surface tells you which layer is wrong — only instrumenting the boundary and counting crossings does. The note backs this with five paired case studies run on both Linux and Windows — a durable-looking write that isn't, a `close()` that silently discards an earlier write error, a "thread" that names three different underlying objects, an `mmap` slower than `read()` for sequential access, and a syscall count that explains latency a profiler misattributes — each anchored to a named tool and artifact (`strace -c / -w`, `blktrace`, Process Monitor, ETW/WPA), checked against primary man pages, Microsoft Learn documentation, and named practitioner benchmarks such as Evan Jones's `blktrace` traces of ext4/xfs and Daniel Lemire's read-versus-mmap throughput measurements, with 24 of 27 claims independently verified.

The note bounds its own authority. It does not claim the five cases are exhaustive or the underlying mechanisms stable: User-Mode Scheduling was withdrawn from Windows entirely, `io_uring` changed what a "syscall count" even measures on Linux, and CPython's GIL is being unwound across versions 3.12–3.13, so every cross-platform statement carries an implicit version scope rather than standing as timeless fact. It takes no position among trusting the abstraction, pushing durability into a dependency, or bypassing the OS layer entirely — these are laid out as trade-offs, not a recommendation. Observation itself is flagged as imperfect and non-neutral: `strace` stops the traced process at every call and ETW sessions carry their own overhead, so the note trusts measured counts and orderings over absolute timings. One number — that a mode switch costs on the order of hundreds of cycles — is marked explicit inference, sourced only from aggregator content rather than a primary benchmark, with a proper measurement left as future work or a student exercise. Some leaks, it notes, sit below what the OS can abstract at all, with no stated floor.

Referenced sources

- [close\(2\) — Linux manual page](#) — Michael Kerrisk / man7.org (2024) HIGH
- [fsync\(2\) — Linux manual page](#) — Michael Kerrisk / man7.org (2024) HIGH
- [Flushing System-Buffered I/O Data to Disk](#) — Microsoft Learn (2021) HIGH

- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn (2023) **HIGH**
- [Durability: Linux File APIs](#) — Evan Jones (2020) **MEDIUM**
- [The Secret Life of fsync](#) — Andrey Pechkurov / puzpuzpuz.dev (2022) **MEDIUM**
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft DevBlogs (2017) **HIGH**
- [fsync, fdatsync, and O_SYNC](#) — kernel-internals.org (2025) **MEDIUM**
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation (2026) **HIGH**
- [Bypassing the GIL for Parallel Processing in Python](#) — Real Python (2024) **MEDIUM**
- [About Processes and Threads](#) — Microsoft Learn (2023) **HIGH**
- [Fibers](#) — Microsoft Learn (2023) **HIGH**
- [User-Mode Scheduling](#) — Microsoft Learn (2022) **HIGH**
- [The Libuv Thread Pool Trap: Why Node.js Async APIs Stall Under Load](#) — The Practical Developer (2026) **MEDIUM**
- [MapViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn (2022) **HIGH**
- [Which is fastest: read, fread, ifstream or mmap?](#) — Daniel Lemire (2012) **HIGH**
- [Latency implications of virtual memory](#) — Erik Rigtorp (2020) **MEDIUM**
- [Why is mmap\(\) Faster Than Sequential IO? Debunking Common Myths](#) — linux-vox.com (2025) **LOW**
- [File Access: Memory-Mapped vs. I/O System Call Performance](#) — Baeldung on Linux (2024) **MEDIUM**
- [strace\(1\)](#) — Linux manual page — strace project / man7.org (2024) **HIGH**
- [DTrace on Windows](#) — Microsoft Learn (2023) **HIGH**
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#) — Yarden Shafir / Winsider Seminars & Solutions (2021) **HIGH**
- [Linux I/O Models — Blocking, epoll, io_uring, AIO, Signal-Driven](#) — systeminternals.dev (2026) **MEDIUM**
- [The Law of Leaky Abstractions](#) — Joel Spolsky (2002) **HIGH**
- [There Is a Mountain Lion in the Datacenter](#) — HFT University (2026) **LOW**
- [Sysinternals Suite 2026.19.08](#) — Neowin (2026) **MEDIUM**
- [How Python's GIL Works](#) — Codefinity (2025) **LOW**

10.17 V17 — Mechanisms for Observing a Running System

"Observing a Running System" argues that every mechanism for watching a live program attaches at a specific point in the path between application code and the kernel, and that where a mechanism attaches mechanically fixes both what it cannot see and what it costs — with cost scaling by event *rate*, not by program size. The `ptrace` / `strace` mechanism on Linux is the note's sharpest illustration of the cost side of that claim. Brendan Gregg's worst case — `dd if=/dev/zero of=/dev/null bs=1 count=500k`, a program doing nothing but syscalls — runs 442× slower under `strace`, and does so even when tracing a syscall the program never makes, because the stop is paid regardless. The note reads this as proof that `strace`'s overhead tracks syscall volume rather than program size or complexity: the kernel stops the traced process twice per syscall, once on entry and once on exit, and pays that cost whether or not the tracer's filter would have kept the event. It backs the 442× figure to Gregg's own reproducible, published benchmark, and cross-checks it against independent measurements of the identical `dd` case on different hardware and speculative-execution mitigation states, which report roughly 37×–136×, alongside other sources quoting "2–10×" or "15×" for differently-shaped workloads. The note's conclusion from that spread is that none of the figures is wrong — they describe different workloads — and that quoting an overhead number without naming the workload that produced it is itself the methodological error.

The note is explicit that 442× is not a general claim about `strace`'s cost: it is one workload's result, offered to demonstrate rate-proportionality, not a figure to carry into a different program. It separately argues, citing peer-reviewed work, that tracing overhead is non-uniform and can distort concurrent *behaviour* rather than merely elapsed time — but keeps that claim distinct from the syscall-rate argument the 442× figure supports. Passages the note treats as its own extrapolation rather than sourced fact are marked `[inference]`; the 442× figure and its workload are not among them. What the note leaves open is how that fixed per-stop cost decomposes on any given machine — context switch, scheduler latency, or the `ptrace` stop itself — and it does not attempt to resolve that, treating it as outside a single note's scope.

Referenced sources

- [ptrace and Syscall Interception](#) — Linux Kernel Internals MEDIUM
- [strace Wow Much Syscall](#) — Brendan Gregg (2014) HIGH
- [Windows vs. Linux Syscall Interfaces: Win32 API, NT Native API, and WSL](#) — kinda-technical.com (2026) MEDIUM
- [Syscall ABI compatibility: Linux vs Windows/macOS](#) — Mark Mossberg (2023) MEDIUM
- [NtTrace — Native API tracing for Windows](#) — Roger Orr HIGH

- [rogerorr/NtTrace — "How it works"](#) — Roger Orr **HIGH**
- [vdso\(7\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [StraceNT — System Call Tracer for Windows NT](#) — Pankaj Garg **MEDIUM**
- [Debugging eBPF verifier: Best Practices for High-Performance Kernel Programming](#) — Raven **MEDIUM**
- [Teaching Operating Systems with FreeBSD through Tracing, Analysis, and Experimentation](#) — George V. Neville-Neil et al., USENIX ;login: (2017) **HIGH**
- [Tracing and its observer effect on concurrency](#) — Indigo Orton, Alan Mycroft — ACM (2021) **HIGH**
- [We Have It Easy, But Do We Have It Right? / observer effect and measurement bias](#) — Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, Peter Sweeney (2008) **HIGH**
- [System Calls and the User-to-Kernel Transition](#) — Yunolay (2026) **MEDIUM**
- [Windows System Call Tables](#) — Mateusz "j00ru" Jurczyk **HIGH**
- [strace: fight for performance \(FOSDEM 2020 slides\)](#) — strace maintainers, FOSDEM (2020) **MEDIUM**
- [System Call Tracing With eBPF: Move Beyond Ptrace](#) — SysTutorials **LOW**
- [Ptrace & You](#) — Omar S. Navarro Leija **MEDIUM**
- [vDSO and Virtual System Calls](#) — Linux Kernel Internals **MEDIUM**
- [System-wide syscalls tracing and monitoring in Linux using bpftrace](#) — Narasimha Prasanna HN **MEDIUM**
- [Notes and Thoughts on Strace for Windows](#) — Jarek Przygódzki **MEDIUM**
- [ipankajg/stracent — StraceNT](#) — Pankaj Garg **MEDIUM**
- [Unveiling Dynamic Binary Instrumentation Techniques](#) — arXiv preprint (2025) **MEDIUM**
- [How to do a million watchpoints: Efficient Debugging using Dynamic Instrumentation](#) — Qin Zhao et al., CC (2008) **HIGH**
- [What is eBPF? An Introduction and Deep Dive into the eBPF Technology](#) — eBPF Foundation **HIGH**
- [Inside the eBPF Verifier: Ensuring Program Safety and Complexity Bounds](#) — How Tech — Systems Programming **MEDIUM**
- [Loops and Iterators in eBPF](#) — Teodor J. Podobnik **MEDIUM**
- [CVE-2021-33624](#) — Red Hat Customer Portal — Red Hat **HIGH**
- [Eliminating eBPF Tracing Overhead on Untraced Processes](#) — Milo Craun et al., eBPF '24 **HIGH**
- [DTrace Capabilities](#) — Oracle — Solaris Dynamic Tracing Guide **HIGH**

- [Dynamic Instrumentation of Production Systems](#) — Bryan M. Cantrill, Michael W. Shapiro, Adam H. Leventhal — USENIX ATC (2004) **HIGH**
- [DTrace on Windows](#) — Microsoft Learn **HIGH**
- [Announcing DTrace for Windows Insider](#) — Packt (2019) **MEDIUM**
- [ETW — Event Tracing for Windows](#) — Mjlnir Security **MEDIUM**
- [NT Kernel Logger Trace Session](#) — Microsoft Learn **HIGH**
- [Windows Performance Recorder](#) — Microsoft Learn **HIGH**
- [Doubling Down: Detecting In-Memory Threats with Kernel ETW Call Stacks](#) — Elastic Security Labs **MEDIUM**
- [prash-wghats/DTrace-win32](#) — prash-wghats **MEDIUM**
- [CSE 422S Studio 5: Tracing the Linux Kernel](#) — Christopher Gill, Washington University in St. Louis **MEDIUM**
- [microsoft/DTrace-on-Windows](#) — Microsoft **HIGH**
- [bpf-helpers\(7\)](#) — Linux manual page — Linux man-pages project **HIGH**
- [bpf: Change value of MAX_TAIL_CALL_CNT from 32 to 33](#) — Linux kernel mailing list / stable backport series (2022) **HIGH**
- [Windows 11, version 26H1](#) — Microsoft Support (2026) **HIGH**
- [Updating Syscall Tables](#) — SysWhispers4 — SysWhispers4 project documentation **MEDIUM**
- [strace 7.1 release notes](#) — strace project (2026) **HIGH**

10.18 V18 — The Memory Hierarchy from a Python Object to a Physical Page

This note traces one allocation through four layers — a Python object in a pymalloc pool and arena, a C-library or Windows heap chunk, a kernel-tracked memory region (a Linux VMA or a Windows reserved/committed region), and a physical page frame — around a single claim: **each level hands out subdivisions of something it bought wholesale from the level below, and each level's unit of purchase is larger than its unit of sale.** That asymmetry is offered as the reason RSS, PSS, and allocator-reported figures disagree without contradicting each other. The evidence combines first-party sources — CPython's C-API reference and internal headers, glibc's manual, Microsoft's Win32 API documentation — with first-person measurements taken on one CI host (CPython 3.13.15, glibc 2.39, Linux 6.8, aarch64): interpreter arena accounting via `sys._debugmallocstats()`, a C program's allocations crossing glibc's 128 KiB mmap threshold as seen in `/proc/<pid>/maps` and `pmap -x`, and a `fork()` experiment showing RSS unchanged while PSS halves then partially recovers as copy-on-write faults

land. Windows is held at equal depth throughout, using `VirtualAlloc`'s reserve/commit split and `VMMMap`'s categories as the structural counterpart to each Linux mechanism.

The note covers only the allocation path, deferring deallocation/retention behavior and the fault taxonomy to two companion notes rather than repeating them. Several findings are flagged as unsettled: `glibc`'s dynamic `mmap` threshold was removed and then reverted within one 2026 release cycle after benchmark regressions, so the note calls it "a contested default with a live bug number," not settled behavior. It states its numeric constants — arena sizes, thresholds, pool geometry — are specific to one host and build and will go stale, noting that most secondary explainers already quote outdated figures. Physical page-frame identity is named as something an unprivileged user cannot observe directly; the note substitutes proportional-set-size and shared/dirty accounting and says so, rather than claiming frame-level verification it did not have. Where a demonstration could not be run locally — an `strace` trace, since `strace` was not installed on the CI host — the note marks it as quoted from a published source rather than measured, preserving the distinction between what it executed itself and what it cites.

Referenced sources

- [mallopt\(3\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project **HIGH**
- [Malloc Tunable Parameters — The GNU C Library manual \(2.44\)](#) — Free Software Foundation **HIGH**
- [Memory Management — Python/C API Reference Manual](#) — Python Software Foundation **HIGH**
- [CPython — Include/internal/pycore_obmalloc.h](#) — Python core developers **HIGH**
- [\[patch\] Make the mmap/brk threshold in malloc dynamic to improve performance](#) — Arjan van de Ven and Val Henson, `libc-alpha` (2006) **HIGH**
- [\[glibc\] Revert "malloc: Remove dynamic mmap/trim threshold \[BZ #30769\]"](#) — Peter Bergner, `glibc-cvs` (2026-07-17) **HIGH**
- [VMMMap v3.4 — Sysinternals](#) — Mark Russinovich / Microsoft Learn (2023) **HIGH**
- [Windows Process Memory Usage Demystified](#) — DZone **MEDIUM**
- [proc_pid_maps\(5\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [brk vs mmap — Linux Kernel Internals](#) — kernel-internals.org **MEDIUM**
- [The /proc Filesystem — The Linux Kernel documentation](#) — The Linux kernel documentation project **HIGH**
- [VMMMap Basics: How to Read a Windows Process's Memory Layout](#) — Pavel Yosifovich / TrainSec **MEDIUM**

- [pagemap, from the userspace perspective — The Linux Kernel documentation — The Linux kernel documentation project](#) **HIGH**
- [VirtualAlloc function \(memoryapi.h\) — Microsoft Learn](#) **HIGH**
- [Heap Functions — Win32 apps — Microsoft Learn](#) **HIGH**
- [How to analyze a Linux process' memory map with pmap — Red Hat \(2022\)](#) **MEDIUM**
- [SYSTEM_INFO structure \(sysinfoapi.h\) — Microsoft Learn](#) **HIGH**
- [Malloc Internals and You — Red Hat Developer \(2017\)](#) **HIGH**
- [Python allocators — memray documentation — Bloomberg](#) **MEDIUM**
- [Increased memory usage with mimalloc \(python/cpython#135153\) — CPython core developers and contributors](#) **HIGH**
- [Legacy NT Heap — windows-heap-exploitation — pwnfuzz \(2025\)](#) **MEDIUM**
- [mallinfo\(3\) / mallinfo2\(3\) — Linux manual page — Michael Kerrisk / The Linux man-pages project](#) **HIGH**
- [Linux Memory Management — COMS W4118, Columbia University — Columbia University course notes \(2023\)](#) **MEDIUM**
- [Python Memory Optimization: How CPython's Memory Pool Works — DEV Community](#) **LOW**
- [Copy-on-Write — Linux Kernel Internals — kernel-internals.org](#) **MEDIUM**
- [Leaking memory with VirtualAlloc — Software Verify \(2022\)](#) **MEDIUM**
- [Your memory leak might be standard Python behavior — Ramon Perez, DEV Community \(2026\)](#) **MEDIUM**

10.19 V19 — The cache layers behind a benchmark number

The note's discussion of the page cache (§3) argues that a warm cache is a benchmarking hazard: a second read of the same file returns fast because the data is already resident in RAM, so a "cold" run is easy to measure by accident as a warm one. Citing the DBTest 2018 database-benchmarking paper, it prescribes stopping the server, dropping OS caches, restarting, and running exactly one query before measuring. But it then complicates that prescription with the strongest caveat in the section, which backs the quoted claim: `drop_caches` does not touch every layer that can cache data. It flushes the OS page cache but not the storage device's own DRAM cache or the controller/HBA cache, so most published "cold-cache benchmarks" are wrong. Dropping the cache also perturbs the very access pattern being measured, since page-cache prefetch and write-back change what I/O the disk actually receives — which is why, for disk benchmarking specifically, `O_DIRECT` rather than `drop_caches` is the correct instrument, since it is the

only way to guarantee the device actually receives the I/O. In a cloud VM, the note adds, eliminating every layer may be impossible outright, because the hypervisor host runs its own cache beneath the guest. The conclusion the note draws from this chain of evidence is the quoted one: rather than claiming a benchmark is truly "cold," state which cache layer the reported number includes, because no single technique removes all of them at once.

The note does not claim to enumerate every cache layer that could be in play — it names the OS page cache, device DRAM, controller/HBA cache, and virtualization-host cache, but does not claim this list is exhaustive, and it does not address CPU-level caches in this context. It does not offer a technique for eliminating every layer; the fix it proposes is disclosure, not elimination. It also does not quantify how common the resulting measurement error is in practice — the assertion that most published cold-cache benchmarks are wrong is stated as a citation-backed claim, not something this note independently measures. Finally, the note treats this specific claim as settled while flagging, elsewhere, that other parts of the same subsystem — Linux's reclaim policy under MGLRU — remain genuinely unsettled as of the note's writing; it draws no connection between the two and does not suggest the benchmarking guidance itself is provisional.

Referenced sources

- [Concepts overview — The Linux Kernel documentation](#) — The Linux Kernel Organization (2026) **HIGH**
- [Windows Kernel Internals: Cache Manager](#) — Microsoft Corporation, University of Tokyo lecture series (n.d.) **HIGH**
- [Page fault](#) — Grokipedia — Grokipedia (2026) **MEDIUM**
- [Working Set - Win32 apps](#) — Microsoft Learn (2026) **HIGH**
- [Copy-on-Write and the Page-Fault Lifecycle, Step by Step](#) — PandaStack (n.d.) **MEDIUM**
- [How to Check Memory Usage per Process in Linux](#) — GoLinuxCloud (n.d.) **MEDIUM**
- [perf-trace\(1\) — Linux manual page](#) — man7.org / Linux man-pages project (2026) **HIGH**
- [The Basics of Page Faults](#) — Microsoft Community Hub, AskPerf team blog (n.d.) **MEDIUM**
- [Windows Performance Monitor basics](#) — SQLShack (n.d.) **MEDIUM**
- [Memory Management Tracing Events - Win32 apps](#) — Microsoft Learn (2021) **HIGH**
- [Why does Linux, by default, overcommit memory?](#) — Unix & Linux Stack Exchange (n.d.) **MEDIUM**
- [Virtual vs Physical vs Resident Memory](#) — Linux Kernel Internals (n.d.) **MEDIUM**

- [The /proc Filesystem — The Linux Kernel documentation](#) — The Linux Kernel Organization (2026) **HIGH**
- [RAMMap - Sysinternals](#) — Microsoft Learn / Sysinternals (2026) **HIGH**
- [Accurately measuring memory usage](#) — Phusion (n.d.) **MEDIUM**
- [Readahead - Linux Kernel Internals](#) — Linux Kernel Internals (n.d.) **MEDIUM**
- [Page Cache Writeback - Linux Kernel Internals](#) — Linux Kernel Internals (n.d.) **MEDIUM**
- [Windows Memory Management in Windows 10 and Windows 11](#) — CyberRaiden (2025) **LOW**
- [Fair Benchmarking Considered Difficult: Common Pitfalls In Database Performance Testing](#) — Raasveldt, Holanda, Gubner & Mühleisen, DBTest (2018) **HIGH**
- [Understanding and tuning the page cache on Debian](#) — Big Iron (2026) **MEDIUM**
- [Excessive Page Faults Generated By Windows Applications](#) — Broadcom Knowledge Base (n.d.) **MEDIUM**
- [Common disk benchmarking mistakes](#) — Stefan Hajnoczi (2017) **HIGH**
- [Memory-Mapped I/O - Linux Kernel Internals](#) — Linux Kernel Internals (n.d.) **MEDIUM**
- [File Mapping - Win32 apps](#) — Microsoft Learn (2026) **HIGH**
- [Creating Named Shared Memory](#) — MicrosoftDocs/win32 repository (2026) **HIGH**
- [Memory-Mapped Files \(mmap\): A Practical Guide to Faster I/O and Shared Memory](#) — martinuke0 (2025) **MEDIUM**
- [mmap SIGBUS error and initializing the file](#) — Stack Overflow (2017) **MEDIUM**
- [MapViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn (2026) **HIGH**
- [mmap — Memory-mapped file support](#) — Python Software Foundation (2026) **HIGH**
- [Are You Sure You Want to Use MMAP in Your Database Management System?](#) — Crotty, Leis & Pavlo, CIDR (2022) **HIGH**
- [Teaching Operating Systems with FreeBSD through Tracing, Analysis, and Experimentation](#) — Neville-Neil et al., USENIX ;login: (2017) **HIGH**
- [Teaching Operating Systems Concepts with Execution Visualization](#) — Giraldeau, Dagenais & Boucheneb, ASEE (n.d.) **HIGH**
- [Mastering strace, perf, and eBPF](#) — 2026 Cheat Sheet — Tech Bytes (2026) **MEDIUM**
- [CS 110: Principles of Computer Systems — assign3](#) — Stanford University (2021) **HIGH**
- [Page Cache - Linux Kernel Internals](#) — Linux Kernel Internals (n.d.) **MEDIUM**

- [On pages and folios](#) — Jonathan Corbet, LWN.net (2026) **HIGH**
- [Reconsidering the multi-generational LRU](#) — Jonathan Corbet, LWN.net (2026-03-05) **HIGH**

10.20 V20 — File I/O and durability

This note argues that durability is not a property of a single write, but of a sequence of up to four independent flushes, each crossing one boundary between an application and the physical storage medium, and each able to fail on its own. The quoted claim covers the last of these four boundaries: the storage device's own cache. The note states that most HDDs and many SSDs acknowledge a write as soon as it lands in an onboard DRAM cache, before the bytes reach NAND or platter, and that a power cut during that window loses the data even though the operating system already reported the write as complete. This is attributed to a source that traces the kernel's flush path down to the device-level FLUSH command. The note explains that the kernel closes this specific gap by issuing an explicit cache-flush command — SYNCHRONIZE CACHE on SCSI/SATA, FLUSH on NVMe — or by marking an individual write FUA (Force Unit Access) to bypass the device cache for that write. It adds two further points about how reliable that closure actually is, both drawn from SQLite's own design documentation: a drive controller can misreport a flush as complete while the data is still sitting in volatile cache, and most consumer SSDs as of 2026 still lack the capacitors that would protect a device cache against power loss.

The note does not claim this failure mode is universal: it specifically says "most" HDDs and "many" SSDs, not all, and treats enterprise hardware with power-loss protection as a distinct case elsewhere. It does not claim that issuing the correct flush command is sufficient by itself, since the same section states that a device's honesty about having completed a flush cannot be verified from software — durability at this layer ultimately rests on trusting hardware the note cannot audit. Independently of this specific claim, the note also flags that its entire four-boundary model is a teaching simplification: network filesystems, memory-mapped I/O, and copy-on-write filesystems each reshape or add to the layers it describes, and applying the table to those cases without adjustment would mislead. Finally, the note leaves one adjacent question explicitly open rather than answered: which Windows storage-tracing tool, if any, can observe this device-level flush behavior as cleanly as Linux's block-layer tracing can — it names this a gap rather than asserting an answer either way.

Referenced sources

- [fsync\(\), Barriers, and the Hardware Durability Contract](#) — Sagar Madala / Medium (2026) **MEDIUM**

- [fsync\(2\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [Durability: Linux File APIs](#) — Evan Jones (2020) **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn **HIGH**
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft Developer Blogs (2017) **HIGH**
- [io — Core tools for working with streams](#) — Python Software Foundation **HIGH**
- [FileStreamOptions.BufferSize Property](#) — Microsoft Learn **HIGH**
- [CSE333 L07: Buffering, POSIX I/O, Syscalls](#) — University of Washington, CSE333 (2022) **MEDIUM**
- [Bug 1400144 — glibc: Performance penalty due to block size-based buffering in stdio](#) — Red Hat Bugzilla **HIGH**
- [What Buffer Flush Means in C \(and C++\)](#) — TheLinuxCode **MEDIUM**
- [Page Cache Writeback](#) — Linux Kernel Internals **MEDIUM**
- [Performance Tuning for Cache and Memory Manager Subsystems](#) — Microsoft Learn **HIGH**
- [File Caching](#) — Microsoft Learn **HIGH**
- [Atomic Commit In SQLite](#) — SQLite / D. Richard Hipp et al. **HIGH**
- [Power Loss Protection shouldn't be an enterprise exclusive](#) — How-To Geek (2026) **LOW**
- [Life of a Write](#) — Linux Kernel Internals **MEDIUM**
- [The errseq_t datatype](#) — The Linux Kernel documentation — Linux kernel documentation **HIGH**
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curator); pgsql-hackers thread participants (2018) **HIGH**
- [Fsync Errors](#) — PostgreSQL wiki **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau / USENIX ATC (2020) **HIGH**
- [CreateFileW function — FILE_FLAG_WRITE_THROUGH and FILE_FLAG_NO_BUFFERING semantics](#) — Microsoft Learn **HIGH**
- [File Buffering](#) — Microsoft Learn **HIGH**
- [Windows Write Caching — Part 2: An Overview for Application Developers](#) — Dilip Naik (2012) **MEDIUM**
- [Discussion: does FlushFileBuffers\(\) on Windows also forget failed writes? / Files Are Hard thread](#) — Hacker News (2019) **LOW**
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — Stack Overflow (2019) **MEDIUM**

- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation **HIGH**
- [Monitoring certain system calls done by a process in Windows](#) — Stack Overflow **MEDIUM**
- [DTrace on Windows](#) — Microsoft Learn **HIGH**
- [FlushViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn **HIGH**
- [Write-Ahead Logging \(SQLite\)](#) — SQLite **HIGH**

10.21 V21 — NtTrace, a debug-interface tracer for the Windows Native API

NtTrace is a user-mode tool for tracing the calls a Windows process makes into `ntdll.dll`, the native API boundary between application code and the NT kernel. The note frames the tool's central contribution as a design decision rather than a feature: it attaches through the Windows debug interface and writes breakpoints around the native call stubs, so it can "trace via the debug interface so the tool needs no driver and no administrator rights." That claim rests on the tool author's own first-party documentation, which states the rationale directly — staying in user mode means the tool can be used without requiring administrative rights or permission to install device drivers — and the note treats it as structurally self-evident, since a debugger-based design is inherently an ordinary-user-rights design, while also noting that two independent third-party reviews of Windows tracing tools corroborate the underlying mechanism. The contrast the note draws is with a kernel-mode alternative, a DTrace-on-Windows reimplementation that must be installed as a driver and requires driver signature enforcement to be disabled at every boot, with no flag to disable it permanently. Against that, NtTrace's no-privilege requirement is presented as what makes it usable on a locked-down or lab machine.

The note is explicit about what this design does not buy. Because NtTrace never crosses the ring boundary, it cannot see kernel calls made by device drivers on behalf of an application, nor any direct use of the `int 2e`, `syscall`, or `sysenter` mechanism — the note treats the second of these as the standard technique for evading user-mode monitoring, not a hypothetical gap. It also flags that the boundary NtTrace observes, `ntdll`, is undocumented and changes between Windows versions, so the tool's knowledge of it lives in a reverse-engineered configuration file rather than a published specification. Separately, the note treats the underlying trade as unsettled rather than resolved: it records a live disagreement over whether `ntdll` should be understood as the interface Windows actually maintains, or merely a convention that looks like an interface until an adversarial program bypasses it with a direct `syscall`, and it notes that two inde-

pendent developers who attempted the same debug-interface design abandoned it as too slow to trace even basic GUI programs — without a published benchmark from the tool's own author to weigh against that report.

Referenced sources

- [NtTrace — Native API tracing for Windows](#), Roger Orr **HIGH**
- [rogerorr/NtTrace — repository README](#), Roger Orr **HIGH**
- [Home · rogerorr/NtTrace Wiki](#), Roger Orr **HIGH**
- [Notes and Thoughts on Strace for Windows](#), Jarek Przygodzki **MEDIUM**
- [AWStrace: Another Windows Strace](#), downwithup **MEDIUM**
- [StraceNT – System Call Tracer for Windows NT](#), Pankaj Garg **MEDIUM**
- [System Call Tracer for Windows \(drstrace\)](#), Dr. Memory / DynamoRIO project **HIGH**
- [mxmssh/drltrace — README](#), Maksim Shudrak **MEDIUM**
- [Introducing QTrace, a "zero knowledge" system call tracer](#), Roberto Paleari **MEDIUM**
- [strace Wow Much Syscall](#), Brendan Gregg **HIGH**
- [Introducing strace --seccomp-bpf](#), Paul Chaignon **HIGH**
- [Do other Unix-like kernels have stable syscall ABIs?](#), Unix & Linux Stack Exchange **MEDIUM**
- [A 10-minute guide to the Linux ABI](#), Alison Chaiken, opensource.com **HIGH**
- [A Deep Dive Into Malicious Direct Syscall Detection](#), Or Chechik and Ofir Ozer, Palo Alto Networks **HIGH**
- [Overload Content by Article — ACCU](#), ACCU **HIGH**
- [Windows System Calls & Native API: A Guide to Assembly Calling Conventions Explained](#), linuxvox.com **LOW**
- [mandiant/STrace — A DTrace on Windows Reimplementation](#), Mandiant **MEDIUM**

10.22 V22 — NtTrace, a user-mode Windows native-API tracer

The note documents NtTrace, a tool that traces a Windows process's calls into `ntdll.dll` by planting breakpoints through the Windows debug interface, a design chosen specifically so the tool needs no driver and no administrator rights. Its evidence for the layer this exposes is a single worked capture, published on the tool's own project page and reproduced in the note because the vault held no concrete NtTrace output before, only prose describing the mechanism. In that capture, `NtQueryAttributesFile` is called with `ObjectAttributes="\??\C:\WINDOWS\system32\cmd.exe.Local"`. The note

reads that string as the clearest available demonstration that Windows's native API sits one layer beneath the Win32 API an ordinary program calls: the `\\??\\` prefix names the object-manager namespace, a path form a Win32 program never constructs itself and never sees, because the Win32 layer translates ordinary paths into it before the native call is made. The note treats this as visible, direct evidence of a layer boundary that the rest of its argument otherwise has to establish through documentation quotes and third-party claims about undocumented, version-varying native calls.

The note is explicit about what this single capture does not establish. It is not paired with any equivalent trace of the same program under a different tracer, so it cannot itself support a side-by-side comparison; it only shows what a Windows trace looks like. It is also a 32-bit, Windows XP-era capture — visible in the `ntdll.dll` base address, the stack addresses, and the process base address — so a capture taken on a current system would show 64-bit addresses, and the note cautions against presenting it as representative of what a reader would see today. The note further flags that the capture shows `NtFsControlFile` listing the `UserApcContext` parameter twice, and states this is unresolved: it may be a quirk of NtTrace's configuration file or an error in how the source page presents the line, and the note says this should be re-checked against the live page before further use. Finally, the note records the capture's provenance as a fetch-and-convert retrieval rather than a terminal transcript, and asks that it be spot-checked against the source before being relied on further.

Referenced sources

- [NtTrace — Native API tracing for Windows](#) — Roger Orr **HIGH**
- [rogerorr/NtTrace — repository README](#) — Roger Orr **HIGH**
- [Home · rogerorr/NtTrace Wiki](#) — Roger Orr **HIGH**
- [Notes and Thoughts on Strace for Windows](#) — Jarek Przygodzki **MEDIUM**
- [AWStrace: Another Windows Strace](#) — downwithup **MEDIUM**
- [StraceNT – System Call Tracer for Windows NT](#) — Pankaj Garg **MEDIUM**
- [System Call Tracer for Windows \(drstrace\)](#) — Dr. Memory / DynamoRIO project **HIGH**
- [mxmssh/drltrace — README](#) — Maksim Shudrak **MEDIUM**
- [Introducing QTrace, a "zero knowledge" system call tracer](#) — Roberto Paleari **MEDIUM**
- [strace Wow Much Syscall](#) — Brendan Gregg **HIGH**
- [Introducing strace --seccomp-bpf](#) — Paul Chaignon **HIGH**
- [Do other Unix-like kernels have stable syscall ABIs?](#) — Unix & Linux Stack Exchange **MEDIUM**
- [A 10-minute guide to the Linux ABI](#) — Alison Chaiken, opensource.com **HIGH**

- [A Deep Dive Into Malicious Direct Syscall Detection](#) — Or Chechik and Ofir Ozer, Palo Alto Networks **HIGH**
- [Overload Content by Article](#) — ACCU — ACCU (Association of C and C++ Users) **HIGH**
- [Windows System Calls & Native API: A Guide to Assembly Calling Conventions Explained](#) — linuxvox.com **LOW**
- [mandiant/STrace — A DTrace on Windows Reimplementation](#) — Mandiant **MEDIUM**

10.23 V23 — NtTrace's published sample capture

"NtTrace — Native API tracing for Windows" documents a Windows user-mode syscall tracer, and to give a reader something concrete, the note carries one worked capture of NtTrace tracing the `cmd` shell, taken from the tool's own project page. The note is explicit that "It is not a matched pair": the capture shows only the Windows side, tracing `cmd`, with no equivalent `strace` capture of the same program produced alongside it for comparison. On its own terms it still does real work — it demonstrates what a Windows native-API trace looks like: a handle-based first argument (`FileHandle=4`) rather than a file descriptor, NT object paths carrying the `\??\` object-manager prefix a Win32 program never sees, a return value that mixes two error namespaces (an `NTSTATUS` translated into a Win32 message), and small, uncoalesced console writes that interleave with the traced program's own output. Each of these features is read directly off the transcript rather than asserted independently, and the note cross-checks them against NtTrace's documented mechanism — breakpoints placed in `ntdll.dll` via the Windows debug interface.

What the note explicitly does not claim is any side-by-side equivalence with `strace`: establishing that would require two traces of the same program made together, which this single capture does not attempt to be. The note also names concrete limits of the sample itself, as unresolved rather than fixed: it is a 32-bit, XP-era capture (addresses like `77F40000`) and should not be presented as what a trace looks like today; one line lists the argument `UserApcContext` twice, and the note leaves open whether that is a quirk of NtTrace's configuration file or a transcription error on the source page; and because the capture was retrieved through a fetch-and-convert step rather than copied from a live terminal, the note calls for it to be spot-checked against the original before it is reused anywhere else. None of these three caveats is settled in the note — they are flagged as open questions for whoever relies on the capture next.

Referenced sources

- NtTrace — Native API tracing for Windows, Roger Orr — <https://rogerorr.github.io/NtTrace/> **HIGH**

- rogerorr/NtTrace — repository README, Roger Orr — <https://github.com/rogerorr/NtTrace> **HIGH**
- Home · rogerorr/NtTrace Wiki, Roger Orr — <https://github.com/rogerorr/NtTrace/wiki> **HIGH**
- Notes and Thoughts on Strace for Windows, Jarek Przygodzki — <https://jarekprzygodzki.dev/post/notes-and-thoughts-on-strace-for-windows/> **MEDIUM**
- AWStrace: Another Windows Strace, downwithup — <https://downwithup.github.io/blog/post/2026/03/29/post10.html> **MEDIUM**
- StraceNT – System Call Tracer for Windows NT, Pankaj Garg — https://blog.pankajgarg.com/system_call_tracer_for_windows/ **MEDIUM**
- System Call Tracer for Windows (drstrace), Dr. Memory / DynamoRIO project — https://drmemory.org/page_drstrace.html **HIGH**
- mxmssh/drltrace — README, Maksim Shudrak — <https://github.com/mxmssh/drltrace> **MEDIUM**
- Introducing QTrace, a "zero knowledge" system call tracer, Roberto Paleari — <http://roberto.greyhats.it/2014/03/qtrace-part1.html> **MEDIUM**
- strace Wow Much Syscall, Brendan Gregg — <https://www.brendangregg.com/blog/2014-05-11/strace-wow-much-syscall.html> **HIGH**
- Introducing strace --seccomp-bpf, Paul Chaignon — <https://pchaigno.github.io/strace/2019/10/02/introducing-strace-seccomp-bpf.html> **HIGH**
- Do other Unix-like kernels have stable syscall ABIs?, Unix & Linux Stack Exchange — <https://unix.stackexchange.com/questions/473137/do-other-unix-like-kernels-have-stable-syscall-abis> **MEDIUM**
- A 10-minute guide to the Linux ABI, Alison Chaiken, opensource.com — <https://opensource.com/article/22/12/linux-abi> **HIGH**
- A Deep Dive Into Malicious Direct Syscall Detection, Or Chechik and Ofir Ozer, Palo Alto Networks — <https://www.paloaltonetworks.com/blog/security-operations/a-deep-dive-into-malicious-direct-syscall-detection/> **HIGH**
- Overload Content by Article — ACCU, ACCU (Association of C and C++ Users) — https://accu.org/journals/nonmembers/overload_article_members/ **HIGH**
- Windows System Calls & Native API: A Guide to Assembly Calling Conventions Explained, linuxvox.com — <https://linuxvox.com/blog/system-calls-in-windows-native-api/> **LOW**
- mandiant/STrace — A DTrace on Windows Reimplementation, Mandiant — <https://github.com/mandiant/STrace> **MEDIUM**

10.24 V24 — WiredTiger's `fsync()` retry-to-panic fix

The note documents "fsyncgate": Linux reports a writeback error to `fsync()` only once, because the failing page is marked clean and discarded at that point, so a second `fsync()` call finds no dirty pages left and returns success even though the original write was never made durable. The claim this appendix centres on is the note's account of MongoDB's WiredTiger storage engine, which fixed this in WT-4045 and, per the note, "quoted this very LWN article in the source comment as its justification," reproducing the line "Given the shared history of UNIX filesystems, and the difficulty of knowing what specific error will be returned under specific circumstances, we don't retry `fsync`-style calls and panic if a flush operation fails." The note treats this as more than an isolated fix: it is evidence that the LWN reporting itself became load-bearing engineering justification, cited verbatim inside a shipped commit rather than merely read and acted on independently. The commit replaced `WT_SYSCALL_RETRY / WT_RET_MSG` with `WT_SYSCALL` plus `WT_PANIC_RET` on the `fdatasync`, `fsync`, and macOS `F_FULLFSYNC` paths. The note situates this alongside PostgreSQL's own PANIC-on-failure commit and InnoDB's replacement of a sleep-and-retry loop with a fatal error, reading the three-way convergence — independent teams, independent architectures, same conclusion within months of each other in 2018 — as the strongest evidence in its record that "just retry" cannot be made to work.

The note is explicit about what this specific claim does not establish. It does not claim WiredTiger's citation of the article means the article was the sole or original source of the fix's reasoning — the `pgsql-hackers` thread and PostgreSQL's own commit predate and independently motivate the same conclusion, and WiredTiger's engineers could have reached PANIC without it. It does not claim citing a piece of journalism in a source comment is itself unusual practice worth generalising from; the note calls it "a rare case of a code comment citing its own journalism" without asserting broader significance. And it does not claim the WiredTiger fix, or the pattern of convergence more broadly, closes the underlying problem: elsewhere the note is clear that a 2020 peer-reviewed fault-injection study found no application, PostgreSQL included, handles `fsync` failure perfectly even after these fixes shipped, and that direct I/O — the fix all parties agreed was the real long-term answer — remains, as of PostgreSQL 18, a debug-only parameter documented as unfit for production.

Referenced sources

- [PostgreSQL's `fsync\(\)` surprise](#) — Jonathan Corbet / LWN.net (2018) HIGH
- [Fsyncgate: errors on `fsync` are unrecoverable](#) — Dan Luu (archive of the `pgsql-hackers` thread, 2018) HIGH

- [PostgreSQL commit 9ccdd7f66e3324d2b6d3dec282cfa9ff084083f1 — "PANIC on fsync\(\) failure"](#) — Craig Ringer / Thomas Munro, PostgreSQL Global Development Group (2018) **HIGH**
- [PostgreSQL documentation — data_sync_retry](#) — PostgreSQL Global Development Group **HIGH**
- [MySQL Bug #90296 — "Hard error should be report when fsync\(\) return EIO"](#) — Oracle / MySQL bug tracker (2018) **HIGH**
- [mysql-server commit 8590c8e12a3374eeccb547359750a9d2a128fa6a — Bug#27805553](#) — Oracle / MySQL Server (2018) **HIGH**
- [WiredTiger commit ae8bccce3d8a8248afa0e4e0cf67674a43dede96 — WT-4045 "Don't retry fsync calls after EIO failure"](#) — Keith Bostic / Alex Gorrod, MongoDB WiredTiger (2018) **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau, USENIX ATC '20 (2020) **HIGH**
- [fs: introduce new writeback error reporting and convert existing API as a wrapper around it](#) — Jeff Layton, linux-fsdevel (2017) **HIGH**
- [PATCH v2 3/5 vfs: track per-sb writeback errors and report them to syncfs](#) — Jeff Layton, LKML (2018) **HIGH**
- [Linux Filesystems API summary — filemap_set_wb_err / filemap_check_wb_err](#) — The Linux Kernel documentation **HIGH**
- [PostgreSQL 18 and beyond: From AIO to Direct IO?](#) — Cybertec PostgreSQL International (2025) **MEDIUM**
- [debug_io_direct](#) — pgPedia — pgPedia, a PostgreSQL Encyclopedia **MEDIUM**
- [All Your GUCs in a Row: data_sync_retry](#) — The Build (PostgreSQL Experts) **MEDIUM**
- [fsync](#) — POSIX Programmer's Manual — IEEE Std 1003.1 / The Open Group Base Specifications **HIGH**

10.25 V25 — The production-unsuitability of syscall-boundary tracing (ptrace / strace)

The note treats "strace --seccomp-bpf still stops at each syscall of interest and should therefore not be used on production systems!" as the closing verdict on the `ptrace / strace` mechanism: even its most optimised variant cannot escape the stop-and-resume structure that makes syscall tracing expensive, so the disqualification is structural, not a tuning shortfall. The note backs this with the mechanism itself — two context switches per syscall, four under record-and-replay instrumentation — and with meas-

urement at several points: Brendan Gregg's 442× worst case on a syscall-saturated benchmark, the strace maintainers' own FOSDEM 2020 figures of roughly 37× and 136× on the same benchmark on different hardware, about 3× on a realistic workload, and roughly 1.44× once `--seccomp-bpf` is enabled. The quote itself comes from the author of that seccomp-bpf optimisation, and Sysdig's independent write-up reaches the same production verdict from the vendor side. The claim sits inside the note's larger thesis that a tracer's attachment point fixes cost and blind spot together: for `strace`, the blind spot is anything that never crosses the syscall boundary, most visibly `io_uring`, which bypasses syscalls entirely and stays invisible no matter how the tracer is tuned.

The note treats this as a ratio-dependent claim, not a blanket one: it names event rate as the variable that can make the ratio stop mattering, citing DTrace guidance that below roughly a thousand traced events per second the overhead becomes unnoticeable — a lightly loaded process is not what the warning covers. It does not settle what "production-safe" means in general: it records, without resolving, a disagreement between Brendan Gregg (architectural non-disruption is enough) and Dan Luu (only continuous fleet-scale cheapness counts) over whether even the cheaper in-kernel mechanisms qualify. It also flags that its overhead figures are not commensurable across benchmarks — a whole-workload multiplier on a syscall-saturated case and a per-event cost elsewhere measure different things — and declines to collapse them into one number, treating only the ordering across mechanisms, not arithmetic between them, as safe to draw.

Referenced sources

- [strace Wow Much Syscall](#) — Brendan Gregg (2014) **HIGH**
- [Ptrace, Utrace, Uprobes: Lightweight, Dynamic Tracing of User Apps](#) — Keniston, Panchamukhi, Hiramatsu — Ottawa Linux Symposium (2007) **HIGH**
- [Introducing strace --seccomp-bpf](#) — Paul Chaignon (2019) **HIGH**
- [strace: fight for performance \(FOSDEM 2020 slides\)](#) — strace maintainers — FOSDEM (2020) **HIGH**
- [Sysdig vs DTrace vs Strace: A technical discussion](#) — Sysdig (n.d.) **MEDIUM**
- [NtTrace — Native API tracing for Windows](#) — Roger Orr (n.d.) **HIGH**
- [Notes and Thoughts on Strace for Windows](#) — Jarek Przygodzki (n.d.) **MEDIUM**
- [Transparent Dynamic Instrumentation](#) — Derek Bruening, Qin Zhao, Saman Amarasinghe — VEE (2012) **HIGH**
- [DynamoRIO Release Notes](#) — DynamoRIO project (n.d.) **HIGH**
- [Dynamic binary instrumentation \(DBI\) with DynamoRio](#) — Cisco Talos Intelligence (2025) **MEDIUM**
- [Tracing a Subset of Execution](#) — DynamoRIO project (n.d.) **HIGH**

- [DTrace for Linux 2016](#) — Brendan Gregg (2016) **HIGH**
- [Benchmarking Performance Overhead of DTrace on FreeBSD and eBPF on Linux](#) — Mateusz Piotrowski — AsiaBSDCon (2024) **HIGH**
- [ebpf_exporter eBPF overhead benchmark](#) — Cloudflare (n.d.) **HIGH**
- [Pitfalls of relying on eBPF for security monitoring \(and some solutions\)](#) — Artem Dinaburg, Trail of Bits (2023) **HIGH**
- [eBPF Hookpoints Gotchas \(FOSDEM 2026 slides\)](#) — FOSDEM (2026) **HIGH**
- [Instrumenting Your Code with ETW](#) — Microsoft (n.d.) **HIGH**
- [Event Tracing for Windows \(ETW\)](#) — Ryan Myers, Microsoft (archived blog, n.d.) **MEDIUM**
- [Performance Analysis with ETW: Event Tracing for Windows](#) — Atomic Object (n.d.) **MEDIUM**
- [Heisenbug](#) — Wikipedia (2026) **MEDIUM**
- [Engineering Record And Replay For Deployability](#) — O'Callahan, Jones, Froyd, Huey, Noll, Partush — USENIX ATC (2017) **HIGH**
- [rr: lightweight recording & deterministic debugging](#) — rr project (2024) **HIGH**
- [Sampling v. tracing](#) — Dan Luu (n.d.) **HIGH**
- [eBPF monitoring overhead: measure and reduce it](#) — groundcover (n.d.) **MEDIUM**
- [strace of io_uring events? \(strace-devel mailing list\)](#) — strace-devel (2020) **HIGH**
- [strace\(1\)](#) — Linux manual page — man7.org (n.d.) **HIGH**
- [DTrace pid Provider Overhead](#) — Brendan Gregg (2011) **HIGH**
- [Portable, Low-Overhead Observability for io_uring](#) — arXiv preprint (2026) **MEDIUM**
- [io_uring War Stories](#) — kernel-internals.org (n.d.) **MEDIUM**
- [Transparently Integrating Debugging and Dynamic Binary Instrumentation](#) — Rensselaer Polytechnic Institute (n.d.) **MEDIUM**
- [Event Tracing for Windows \(ETW\) — architecture and providers](#) — Salucci (n.d.) **MEDIUM**
- [Bypassing ETW For Fun and Profit](#) — White Knight Labs (2021) **MEDIUM**
- [Support io_uring](#) — rr issue #2613 — Robert O'Callahan, rr project (2020) **MEDIUM**
- [Sampling vs deterministic profilers: which should I use?](#) — Python Developer Tooling Handbook (n.d.) **MEDIUM**

10.26 V26 — The System Call Boundary

The note argues that what gets called "the system call boundary" is really two boundaries conflated: a CPU-level privilege transition (ring 3 to ring 0, via the `syscall` instruction on x86-64) and a separate, policy-level promise that an operating system makes not to change its interface. Linux places both in the same location — its numbered syscall ABI is documented as permanent and append-only. The claim quoted here sits at the point where the note says its own mechanism argument breaks down: the vDSO. Linux maps a kernel-provided page into every process so that certain calls — `clock_gettime`, `gettimeofday`, `getcpu` — are served entirely in user space, with no privilege transition at all. Because `strace` works by using `ptrace` to stop a process at the entry and exit of an actual kernel crossing, and a vDSO call never crosses into the kernel, there is nothing for `strace` to intercept — the call is not merely fast, it is structurally invisible to that class of tool. The note treats this as the clearest classroom demonstration that a syscall is not always "a fundamentally different mechanism than a function call": for vDSO-served calls, it is a function call, full stop.

The note is explicit that this is one of three places its central thesis weakens, alongside I/O batching and the observation that syscall cost is dominated by microarchitectural effects rather than the crossing instruction itself. It does not claim the vDSO fast path is unconditional: on some virtualized hosts the fallback breaks silently, and a call that should be free is served as a real syscall instead when the guest clocksource lacks TSC support — in which case `strace` will show it after all, which the note offers as a diagnostic rather than an exception to ignore. It also does not commit to specific overhead numbers: the cycle counts it cites comparing the vDSO and syscall paths come from single-machine, single-author measurements that the note itself flags as illustrations of method, not settled figures, and it tells the reader to measure their own hardware rather than quote anyone else's.

Referenced sources

- [Anatomy of a system call, part 1](#) — David Drysdale / LWN.net (2014) **HIGH**
- [Anatomy of a system call, part 2](#) — David Drysdale / LWN.net (2014) **HIGH**
- [SYSCALL — Fast System Call \(x86 instruction reference\)](#) — Felix Cloutier, transcribed from the Intel SDM (n.d.) **HIGH**
- [x86-64 Syscall Entry](#) — Linux Kernel Internals — kernel-internals.org (n.d.) **MEDIUM**
- [The Linux Kernel Driver Interface \(stable-api-nonsense\)](#) — Greg Kroah-Hartman / Linux kernel documentation (2004–present) **HIGH**
- [Add kernel ↔ userspace ABI stability documentation](#) — Greg Kroah-Hartman / LWN.net (2006) **HIGH**

- [x86 Assembly / Interfacing with Linux](#) — Wikibooks contributors (n.d.) **MEDIUM**
- [Do other Unix-like kernels have stable syscall ABIs?](#) — Unix & Linux Stack Exchange (2018–2023) **MEDIUM**
- [Libraries and Headers — Windows drivers](#) — Microsoft Learn (last updated 2022-07-12) **HIGH**
- [winternl.h header — Win32 apps](#) — Microsoft Learn (n.d.) **HIGH**
- [NTDLL Functions](#) — Geoff Chappell (ongoing) **HIGH**
- [Windows X86-64 System Call Table \(XP through Windows 11 and Server\)](#) — Mateusz "j00ru" Jurczyk (ongoing) **HIGH**
- [Microsoft Windows library files — NTDLL.DLL](#) — Wikipedia contributors (n.d.) **MEDIUM**
- [Windows X86-64 System Call Table — discussion thread](#) — Hacker News contributors (2019) **LOW**
- [strace — linux syscall tracer](#) — The strace project (ongoing) **HIGH**
- [ptrace and Syscall Interception — Linux Kernel Internals](#) — kernel-internals.org (n.d.) **MEDIUM**
- [Process Monitor — Sysinternals](#) — Microsoft Learn / Sysinternals (ongoing) **HIGH**
- [NtTrace — Native API tracing for Windows](#) — Roger Orr (ongoing) **MEDIUM**
- [What Makes System Calls Expensive](#) — Abhinav Upadhyay (2025) **MEDIUM**
- [vDSO and Virtual System Calls — Linux Kernel Internals](#) — kernel-internals.org (n.d.) **MEDIUM**
- [Two frequently used system calls are ~77% slower on AWS EC2](#) — packagecloud engineering (n.d.) **MEDIUM**
- [Why you should use io_uring for network I/O](#) — Red Hat Developers (2023) **HIGH**
- [Speed up open\(\).read\(\) pattern by reducing the number of system calls](#) — cpython#120754 — CPython issue tracker (2024) **HIGH**
- [win32file.CreateFile — PyWin32 documentation](#) — Mark Hammond / Tim Golden (PyWin32 docs, ongoing) **MEDIUM**
- [ltrace: When strace Is Not Enough](#) — Satesh / unixy.io (2026) **MEDIUM**
- [System Calls and the User-to-Kernel Transition](#) — Yunolay (2026) **MEDIUM**
- [Windows vs. Linux Syscall Interfaces: Win32 API, NT Native API, and WSL](#) — kinda-technical.com (2026) **LOW**
- [In Linux x86_64, are syscalls and int 0x80 related?](#) — Stack Overflow (2015–present) **MEDIUM**
- [A Deep Dive Into Malicious Direct Syscall Detection](#) — Palo Alto Networks / Unit 42 (2024) **HIGH**

- [Linux kernel system calls table](#) — Marcin Juskiewicz (regenerated from kernel source; table dated 2026-08-20 from 7.2.0 sources) **HIGH**
- [io_uring Interface Overview](#) — EmergentMind, summarizing Jasny et al. (updated 2026-01-17) **LOW**

10.27 V27 — The system call as a distinct hardware mechanism, not an expensive function call

This note argues a system call is not a slower function call but a different mechanism, and that most of its cost is paid after the call returns, in ordinary user-mode code whose cache and TLB state the crossing destroyed. On x86-64, Linux and Windows both reach the kernel through the same `SYSCALL` instruction, not a normal call: it loads the entry point from a Model-Specific Register, does not save the stack pointer, skips the descriptor tables, and acts as a full speculation barrier. Each OS registers its own entry point at boot — `entry_SYSCALL_64` on Linux, `KiSystemCall64` on Windows — so the privilege transition is one shared hardware event even though the software above it (a libc wrapper versus an `ntdll.dll` stub) differs. The evidence for where cost actually lives is a peer-reviewed measurement (FlexSC, USENIX OSDI 2010): a single `stat` call retires under 5,000 instructions but evicts thousands of cache lines and dozens of TLB entries, and at moderate frequencies that indirect cost, not the crossing itself, dominates the throughput loss. The practical conclusion: crossing count, not bytes moved, is the first-order performance variable — demonstrated in buffered-versus-unbuffered I/O and in syscall choices like `pread / pwrite` over `lseek + read`.

The note names its own limits explicitly. When a call is served by Linux's vDSO or Windows' `KUSER_SHARED_DATA` — read-only pages mapped into user space to avoid a kernel transition — there is no crossing, and the call behaves like an ordinary function call. Batched submission (io_uring, Windows I/O Rings) amortizes the fixed cost across operations, though batching can itself be slower than what it replaces at very low batch sizes. At very high call frequencies, the direct cost of the exception and pipeline flush overtakes the indirect cache/TLB cost — the thesis's other boundary. Genuine disagreement is preserved rather than resolved: reported KPTI overhead ranges from under 1% to over 800% by workload, and sources even disagree on whether an unmitigated syscall touches the TLB at all. Two gaps are named outright: every quantitative cost breakdown is Linux-specific and from 2010-era hardware, so no comparable Windows footprint or current eviction counts exist; and the mechanism argument is written for x86-64 only, with ARM64 explicitly out of scope.

Referenced sources

- [SYSCALL — Fast System Call \(Intel 64 and IA-32 Architectures Software Developer's Manual\)](#) — Intel Corporation **HIGH**
- [The Definitive Guide to Linux System Calls](#) — packagecloud **MEDIUM**
- [FlexSC: Flexible System Call Scheduling with Exception-Less System Calls](#) — Livio Soares and Michael Stumm, University of Toronto — USENIX OSDI (2010) **HIGH**
- [System calls on Windows x64](#) — n4r1b (2019) **MEDIUM**
- [Page Table Isolation \(PTI\)](#) — [The Linux Kernel documentation](#) — Linux kernel maintainers **HIGH**
- [KPTI/KAISER Meltdown Initial Performance Regressions](#) — Brendan Gregg, Netflix (2018) **MEDIUM**
- [Kernel page-table isolation](#) — Wikipedia **LOW**
- [What Makes System Calls Expensive: A Linux Internals Deep Dive](#) — Abhinav Upadhyay (2025) **MEDIUM**
- [Two frequently used system calls are ~77% slower on AWS EC2](#) — packagecloud **MEDIUM**
- [Randomizing the KUSER_SHARED_DATA Structure on Windows](#) — Microsoft Security Response Center (2022) **HIGH**
- [io_uring for High-Performance DBMSs: When and How to Use It](#) — Jasny et al. (2025) **MEDIUM**
- [SubmitIoRing function \(io_uringapi.h\)](#) — Microsoft Learn **HIGH**
- [One I/O Ring to Rule Them All: A Full Read/Write Exploit Primitive on Windows 11](#) — Yarden Shafir, Winsider Seminars & Solutions **MEDIUM**
- [io](#) — [Core tools for working with streams \(Python documentation\)](#) — Python Software Foundation **HIGH**
- [FileStreamOptions.BufferSize Property \(System.IO\)](#) — Microsoft Learn **HIGH**
- [Buffered I/O \(stdio library\) vs. Unbuffered I/O \(Sys. Calls\)](#) — circuitlabs (2025) **LOW**
- [15-213 Lecture 16 Activity Solutions: Unix I/O and Standard I/O](#) — Carnegie Mellon University, 15-213 Introduction to Computer Systems **HIGH**
- [Meltdown/Spectre Benchmarks](#) — [Open Telekom Cloud Image Factory](#) — T-Systems / Open Telekom Cloud **MEDIUM**
- [On the Costs of Syscalls \(2021\)](#) — [Hacker News discussion](#) — Hacker News commenters **LOW**
- [On the Costs of Syscalls](#) — Georg Sauthoff (2021) **MEDIUM**

- [Inspection of I/O Operations from System Call Traces using Directly-Follows-Graph](#) — Aravind Sankaran, Ilya Zhukov, Wolfgang Frings, Paolo Bientinesi (2024)

MEDIUM

- [A Deep Dive into Zero-Copy Networking and io_uring](#) — Jatin Umamtora, Medium

LOW

- [Comment thread on syscall cost and out-of-order execution](#) — Hacker News — commenter "anarazel" (2021)

LOW

10.28 V28 — The Kernel-Object Model Behind Sockets, and Where Each Platform Pins Its Contract

This note establishes that a socket is a kernel-owned object reached through a per-process handle, and that Linux and Windows diverge not in whether the kernel treats a socket as a file, but in which layer publishes that fact as a binding contract. On Linux, `socket()` returns a file descriptor identical in kind to any other: the kernel binds it to a `struct file` on the `sockfs` pseudo-filesystem and dispatches `read`, `write`, `close`, `select`, and `fstat` through the same `file_operations` table used for ordinary files, for a structural reason rather than a convenient coincidence. The note backs this both from the kernel's own `net/socket.c` source and from a C program and Python script the author ran on Linux 6.8.0, whose `fstat` output and `/proc/self/fd` resolution matched the kernel's own inode-naming code. On Windows, `socket()` returns an opaque `SOCKET`, and Microsoft's own documentation states plainly that a socket descriptor is not a file descriptor and must be used with Winsock functions — yet the NT kernel underneath still models the socket as a file object on a device called `\Device\Afd`, which is exactly what lets `ReadFile`, `WriteFile`, and I/O completion ports operate on it at all. Three consequences follow from where each platform draws its contract line: what else can touch the handle, how it must be released (`close` versus `closesocket`, never `CloseHandle`), and how failure is reported (`errno` versus `WSAGetLastError`).

The note is explicit about what it does not claim. It rejects the common shorthand that "Windows doesn't model sockets as files," arguing instead that both kernels do so and only the published contract differs — a correction it applies deliberately against its own originating brief. Where two Microsoft Learn pages read, taken naively, as flatly contradicting each other on whether a socket handle can double as a file handle, the note marks its reconciliation of them as its own inference, not a claim either source makes outright. Several points are left unsettled rather than resolved: whether Windows `AF_UNIX` has closed the object-model gap (the note finds convergence only in address family and naming, with descriptor-passing and other file-model-dependent behavior

still omitted), and whether `io_uring` has erased the readiness-versus-completion asymmetry with Winsock's overlapped I/O (the note says this narrows the gap and flags the old shorthand as aging, not that the asymmetry is gone).

Referenced sources

- [socket\(2\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project **HIGH**
- [socket\(7\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project **HIGH**
- [linux/net/socket.c](#) — Linux kernel source tree **HIGH**
- [Socket Data Type — Win32 apps](#) — Microsoft **HIGH**
- [Socket Handles — Win32 apps](#) — Microsoft **HIGH**
- [Error Codes - errno, h_errno and WSAGetLastError](#) — Win32 apps — Microsoft **HIGH**
- [WSAStartup function \(winsock.h\)](#) — Microsoft **HIGH**
- [closesocket function \(winsock.h\)](#) — Microsoft **HIGH**
- [CloseHandle function \(handleapi.h\)](#) — Microsoft **HIGH**
- [Select, fd_set, and FD_XXX Macros](#) — Win32 apps — Microsoft **HIGH**
- [socket](#) — Low-level networking interface (Python documentation) — Python Software Foundation **HIGH**
- [select](#) — Waiting for I/O completion (Python documentation) — Python Software Foundation **HIGH**
- [AF_UNIX comes to Windows](#) — Microsoft, Windows Command Line blog (2017) **HIGH**
- [Improving AFD Socket Visibility for Windows Forensics & Troubleshooting](#) — diversenok (2025) **MEDIUM**
- [12 Days of HaXmas: Does it Blend Like a Duck? \(Failing Portably\)](#) — Rapid7 (2014) **MEDIUM**
- [An Introductory 4.4BSD Interprocess Communication Tutorial](#) — Computer Systems Research Group, UC Berkeley **HIGH**
- [An Advanced 4.3BSD Interprocess Communication Tutorial](#) — Samuel J. Leffler et al., UC Berkeley **HIGH**
- [4.4BSD Networking Implementation Notes](#) — Computer Systems Research Group, UC Berkeley **HIGH**
- [rogerorr/NtTrace](#) — Roger Orr **HIGH**
- [Porting Socket Applications to Winsock](#) — Win32 apps — Microsoft **HIGH**
- [Under the Hood of AFD.sys Part 2: TCP handshake](#) — Mateusz Lewczak (2025) **MEDIUM**
- [cpython/Modules/socketmodule.h](#) — Python Software Foundation **HIGH**

- [Closing non-blocking socket causes leak](#) — Microsoft **HIGH**
- [vfs: mark pipes and sockets as stream-like file descriptors](#) — Linus Torvalds, Linux kernel (2019) **HIGH**
- [io_uring zero copy Rx](#) — The Linux Kernel documentation — The Linux Kernel Organization **HIGH**
- [Winsock Programmer's FAQ: General Winsock Information](#) — Warren Young **MEDIUM**

10.29 V29 — The kernel-object model behind sockets on Linux and Windows

This note establishes that a socket is not a protocol, a port, or a connection, but an object the kernel creates and owns, and that Linux and Windows expose that object through different contracts even though both kernels implement it the same way underneath. On Linux, `socket()` returns a literal file descriptor: the kernel allocates a `struct socket`, binds it to a `struct file` on the `sockfs` pseudo-filesystem, and routes `read`, `write`, `close`, `select`, `fstat`, and `fcntl` through the same dispatch table ordinary files use — evidenced by the kernel's own `socket_file_ops` table and by a `/proc/self/fd` trace reproduced for the note, showing `fstat` reporting `S_ISSOCK` with the inode matching the `socket:[...]` symlink. On Windows, `socket()` returns an opaque `SOCKET`, and Microsoft's own documentation states that "a socket descriptor is not a file descriptor and must be used with the Winsock functions," with its own initialization (`WSAStartup`), teardown (`closesocket`), and error namespace (`WSAGetLastError`). The note argues that the tempting shorthand — Unix treats sockets as files, Windows does not — is wrong: the NT kernel also holds a socket as a file object, under the `\Device\Afd` device, which is exactly what lets `ReadFile`, `WriteFile`, and I/O completion ports operate on it. The real divergence is which layer each platform makes binding, not the mechanism itself: Linux nails the promise to the file-descriptor contract, Windows nails it to the Winsock API.

The note does not claim this contract difference produces crashes or compiler errors; it stresses the opposite — that the failure mode is silent, surfacing later as a resource leak or a `WSAENOTSOCK` error. It treats the reconciliation of two seemingly contradictory Microsoft Learn pages — one stating a socket descriptor is not a file descriptor, the other stating a socket handle can optionally be a file handle usable with `ReadFile/WriteFile` — as its own inference rather than a claim either source makes directly, and leaves that tension visible rather than smoothing it over. Several claims about the Windows AFD IOCTL mechanism and the shape of a Windows system-call trace are marked as reported rather than independently verified, since `strace` was unavailable in the

note's own environment and the equivalent Windows trace was not reproduced firsthand. The note also flags as unsettled whether Linux's `io_uring` networking surface fully closes the readiness-versus-completion gap with Windows I/O completion ports, describing the older "Linux is readiness-based, Windows is completion-based" shorthand as merely "aging" rather than resolved.

Referenced sources

- [socket\(2\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project **HIGH**
- [socket\(7\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project **HIGH**
- [linux/net/socket.c](#) — Linux kernel source tree **HIGH**
- [Socket Data Type — Win32 apps](#) — Microsoft **HIGH**
- [Socket Handles — Win32 apps](#) — Microsoft **HIGH**
- [Error Codes - errno, h_errno and WSAGetLastError](#) — Win32 apps — Microsoft **HIGH**
- [WSAStartup function \(winsock.h\)](#) — Microsoft **HIGH**
- [closesocket function \(winsock.h\)](#) — Microsoft **HIGH**
- [CloseHandle function \(handleapi.h\)](#) — Microsoft **HIGH**
- [Select, fd_set, and FD_XXX Macros](#) — Win32 apps — Microsoft **HIGH**
- [socket — Low-level networking interface \(Python documentation\)](#) — Python Software Foundation **HIGH**
- [select — Waiting for I/O completion \(Python documentation\)](#) — Python Software Foundation **HIGH**
- [AF_UNIX comes to Windows](#) — Microsoft, Windows Command Line blog (2017) **HIGH**
- [Improving AFD Socket Visibility for Windows Forensics & Troubleshooting](#) — diversenok (2025) **MEDIUM**
- [12 Days of HaXmas: Does it Blend Like a Duck? \(Failing Portably\)](#) — Rapid7 (2014) **MEDIUM**
- [An Introductory 4.4BSD Interprocess Communication Tutorial](#) — Computer Systems Research Group, UC Berkeley **HIGH**
- [An Advanced 4.3BSD Interprocess Communication Tutorial](#) — Samuel J. Leffler et al., UC Berkeley **HIGH**
- [4.4BSD Networking Implementation Notes](#) — Computer Systems Research Group, UC Berkeley **HIGH**
- [rogerorr/NtTrace](#) — Roger Orr **HIGH**
- [Porting Socket Applications to Winsock](#) — Win32 apps — Microsoft **HIGH**

- [Under the Hood of AFD.sys Part 2: TCP handshake](#) — Mateusz Lewczak (2025)

MEDIUM

- [cpython/Modules/socketmodule.h](#) — Python Software Foundation
- [Closing non-blocking socket causes leak](#) — Microsoft
- [vfs: mark pipes and sockets as stream-like file descriptors](#) — Linus Torvalds, Linux kernel (2019)
- [io_uring zero copy Rx](#) — The Linux Kernel documentation — The Linux Kernel Organization
- [Winsock Programmer's FAQ: General Winsock Information](#) — Warren Young

MEDIUM

10.30 V30 — The gap between a completed write and durable storage on Linux and Windows

This note establishes that durability in file I/O is not a single property of a write call but the result of successfully crossing four independent buffering boundaries — a language-runtime buffer, the kernel's acceptance of a `write()`, the kernel page cache's writeback to the device, and the device cache's flush to physical media — each with its own trigger, cost, and independent failure mode. It argues that a successful `write()` on Linux, or `WriteFile()` on Windows, reports only that the first kernel-side buffer accepted the bytes, a statement about custody rather than persistence, and that neither `fflush()/f.flush()` nor `close()` crosses the durability boundary. It grounds `fsync()`'s guarantee in the Linux man page and documents the 2018 "fsyncgate" incident, in which PostgreSQL's checkpointer retried a failed `fsync()` and unknowingly risked data loss for roughly two decades, because the Linux kernel discards the dirty pages on a writeback failure and reports the error to only the next caller, once — a documented, contract-level disagreement between the kernel and application developers rather than a bug on either side, which PostgreSQL, InnoDB and WiredTiger all resolved by panicking and recovering from a write-ahead log rather than retrying. For Windows, the note documents the same four-layer structure under different names (Cache Manager, lazy writer, `FlushFileBuffers`) but finds — after checking the PostgreSQL wiki's per-OS behavior matrix, a Hacker News thread, and a Stack Overflow answer — that no equivalent post-failure contract has ever been documented by Microsoft or characterized publicly by anyone.

The note explicitly does not claim that Windows shares Linux's non-retryable failure behavior, nor that Windows is safe from it — it states plainly that the question is unresolved, because Microsoft's documentation specifies only the success/failure return convention and says nothing about the state of cached data after a failure, so the honest

position is "undocumented" rather than "same bug" or "no bug." Several supporting claims are flagged rather than asserted as verified: the mapping from Windows CRT `_commit()` to `FlushFileBuffers` is marked as an inference rather than a documented fact, and the claim that no Windows tool exposes device-level flush commands as cleanly as `blktrace` does is marked speculative, with an open "Grounding gap" section requesting further sourcing on both points as well as on whether Windows exposes any dirty-page-age knob. The note also states outright that its four-boundary model is a teaching simplification that does not account for network filesystems, memory-mapped I/O, or copy-on-write filesystems, and warns that a student who over-applies the table to those cases will get the answer wrong.

Referenced sources

- [fsync\(\), Barriers, and the Hardware Durability Contract](#) — Sagar Madala / Medium (2026) **MEDIUM**
- [fsync\(2\)](#) — Linux manual page — Linux man-pages project **HIGH**
- [Durability: Linux File APIs](#) — Evan Jones (2020) **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn **HIGH**
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft Developer Blogs (2017) **HIGH**
- [io](#) — Core tools for working with streams — Python Software Foundation **HIGH**
- [FileStreamOptions.BufferSize Property](#) — Microsoft Learn **HIGH**
- [CSE333 L07: Buffering, POSIX I/O, Syscalls](#) — University of Washington, CSE333 (2022) **MEDIUM**
- [Bug 1400144 — glibc: Performance penalty due to block size-based buffering in stdio](#) — Red Hat Bugzilla **HIGH**
- [What Buffer Flush Means in C \(and C++\)](#) — TheLinuxCode **MEDIUM**
- [Page Cache Writeback](#) — Linux Kernel Internals **MEDIUM**
- [Performance Tuning for Cache and Memory Manager Subsystems](#) — Microsoft Learn **HIGH**
- [File Caching](#) — Microsoft Learn **HIGH**
- [Atomic Commit In SQLite](#) — SQLite / D. Richard Hipp et al. **HIGH**
- [Power Loss Protection shouldn't be an enterprise exclusive](#) — How-To Geek (2026) **LOW**
- [Life of a Write](#) — Linux Kernel Internals **MEDIUM**
- [The errseq_t datatype](#) — The Linux Kernel documentation — Linux kernel documentation **HIGH**

- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curator); pgsql-hackers thread participants (2018) **HIGH**
- [Fsync Errors](#) — PostgreSQL wiki **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau / USENIX ATC (2020) **HIGH**
- [CreateFileW function — FILE_FLAG_WRITE_THROUGH and FILE_FLAG_NO_BUFFERING semantics](#) — Microsoft Learn **HIGH**
- [File Buffering](#) — Microsoft Learn **HIGH**
- [Windows Write Caching — Part 2: An Overview for Application Developers](#) — Dilip Naik (2012) **MEDIUM**
- [Discussion: does FlushFileBuffers\(\) on Windows also forget failed writes? / Files Are Hard thread](#) — Hacker News (2019) **LOW**
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — Stack Overflow (2019) **MEDIUM**
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation **HIGH**
- [Monitoring certain system calls done by a process in Windows](#) — Stack Overflow **MEDIUM**
- [DTrace on Windows](#) — Microsoft Learn **HIGH**
- [FlushViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn **HIGH**
- [Write-Ahead Logging \(SQLite\)](#) — SQLite **HIGH**

10.31 V31 — Mechanisms for observing a running system

This note argues that the tools used to watch a running program — `strace`, NtTrace, StraceNT, dynamic binary instrumentation, eBPF, DTrace, and ETW — are not interchangeable flavors of the same idea. Each one differs in *where it attaches* to the system: at the kernel's syscall entry path, at a user-mode library boundary, in a rewritten instruction stream, inside an in-kernel programmable probe, or via an OS-published event pipeline. That attachment point mechanically determines three properties for every mechanism: what it is positioned to see, the blind spot that is the complement of that visibility, and the cost it imposes, which scales with event rate rather than program size. The note grounds this in the structural difference between Linux, which publishes a stable syscall ABI so `ptrace` can intercept at the kernel boundary and name what it sees, and Windows, which guarantees stability only at the outer Win32 layer, forcing its canonical tracer (NtTrace) to set breakpoints in `ntdll` instead. It backs the claim with concrete, sourced blind spots — `strace` cannot see vDSO-resolved

calls, NtTrace cannot see a direct `syscall` instruction issued outside `ntdll`, eBPF cannot express a program its verifier cannot prove terminates — and with the observer-effect research showing that tracing overhead is non-uniform, capable of distorting concurrent behavior qualitatively rather than merely slowing it down.

The note is explicit about what it does not claim. It does not rank the mechanisms — the recurring point is that "which tracer is best" is the wrong question, and the right one is which blind spot and which perturbation a given investigation can afford. It does not treat published overhead figures as settled facts: it shows the same `strace` benchmark quoted at 2–10×, 15×, 100×, and 442× across different sources and attributes the spread to differing workloads rather than to any single correct number, warning that a figure without its workload is not information. Where the underlying literature disagrees or a claim rests on inference rather than a stated source, the note marks it explicitly as such. It also declines to settle whether an AI coding agent changes this picture, treating that as a separate, sibling question the note deliberately leaves open.

Referenced sources

- [ptrace and Syscall Interception](#) — Linux Kernel Internals. **MEDIUM**
- [strace Wow Much Syscall](#) — Brendan Gregg (2014). **HIGH**
- [Windows vs. Linux Syscall Interfaces: Win32 API, NT Native API, and WSL](#) — kinda-technical.com (2026). **MEDIUM**
- [Syscall ABI compatibility: Linux vs Windows/macOS](#) — Mark Mossberg (2023). **MEDIUM**
- [NtTrace — Native API tracing for Windows](#) — Roger Orr. **HIGH**
- [rogerorr/NtTrace — "How it works"](#) — Roger Orr. **HIGH**
- [vdso\(7\) — Linux manual page](#) — Linux man-pages project. **HIGH**
- [StraceNT — System Call Tracer for Windows NT](#) — Pankaj Garg. **MEDIUM**
- [Debugging eBPF verifier: Best Practices for High-Performance Kernel Programming](#) — Raven. **MEDIUM**
- [Teaching Operating Systems with FreeBSD through Tracing, Analysis, and Experimentation](#) — George V. Neville-Neil et al., USENIX ;login: (2017). **HIGH**
- [Tracing and its observer effect on concurrency](#) — Indigo Orton, Alan Mycroft — ACM (2021). **HIGH**
- [We Have It Easy, But Do We Have It Right? / observer effect and measurement bias](#) — Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, Peter Sweeney (2008). **HIGH**
- [System Calls and the User-to-Kernel Transition](#) — Yunolay (2026). **MEDIUM**
- [Windows System Call Tables](#) — Mateusz "j00ru" Jurczyk. **HIGH**

- [strace: fight for performance \(FOSDEM 2020 slides\)](#) — strace maintainers, FOSDEM (2020). **MEDIUM**
- [System Call Tracing With eBPF: Move Beyond Ptrace](#) — SysTutorials. **LOW**
- [Ptrace & You](#) — Omar S. Navarro Leija. **MEDIUM**
- [vDSO and Virtual System Calls](#) — Linux Kernel Internals. **MEDIUM**
- [System-wide syscalls tracing and monitoring in Linux using bpftrace](#) — Narasimha Prasanna HN. **MEDIUM**
- [Notes and Thoughts on Strace for Windows](#) — Jarek Przygódzki. **MEDIUM**
- [ipankajg/stracent](#) — StraceNT — Pankaj Garg. **MEDIUM**
- [Unveiling Dynamic Binary Instrumentation Techniques](#) — arXiv preprint (2025). **MEDIUM**
- [How to do a million watchpoints: Efficient Debugging using Dynamic Instrumentation](#) — Qin Zhao et al., CC (2008). **HIGH**
- [What is eBPF? An Introduction and Deep Dive into the eBPF Technology](#) — eBPF Foundation. **HIGH**
- [Inside the eBPF Verifier: Ensuring Program Safety and Complexity Bounds](#) — How Tech — Systems Programming. **MEDIUM**
- [Loops and Iterators in eBPF](#) — Teodor J. Podobnik. **MEDIUM**
- [CVE-2021-33624](#) — Red Hat Customer Portal — Red Hat. **HIGH**
- [Eliminating eBPF Tracing Overhead on Untraced Processes](#) — Milo Craun et al., eBPF '24. **HIGH**
- [DTrace Capabilities](#) — Oracle — Solaris Dynamic Tracing Guide. **HIGH**
- [Dynamic Instrumentation of Production Systems](#) — Bryan M. Cantrill, Michael W. Shapiro, Adam H. Leventhal — USENIX ATC (2004). **HIGH**
- [DTrace on Windows](#) — Microsoft Learn. **HIGH**
- [Announcing DTrace for Windows Insider](#) — Packt (2019). **MEDIUM**
- [ETW — Event Tracing for Windows](#) — Mjolnir Security. **MEDIUM**
- [NT Kernel Logger Trace Session](#) — Microsoft Learn. **HIGH**
- [Windows Performance Recorder](#) — Microsoft Learn. **HIGH**
- [Doubling Down: Detecting In-Memory Threats with Kernel ETW Call Stacks](#) — Elastic Security Labs. **MEDIUM**
- [prash-wghats/DTrace-win32](#) — prash-wghats. **MEDIUM**
- [CSE 422S Studio 5: Tracing the Linux Kernel](#) — Christopher Gill, Washington University in St. Louis. **MEDIUM**
- [microsoft/DTrace-on-Windows](#) — Microsoft. **HIGH**
- [bpf-helpers\(7\)](#) — Linux manual page — Linux man-pages project. **HIGH**

- [bpf: Change value of MAX_TAIL_CALL_CNT from 32 to 33](#) — Linux kernel mailing list / stable backport series (2022). HIGH
- [Windows 11, version 26H1](#) — Microsoft Support (2026). HIGH
- [Updating Syscall Tables](#) — SysWhispers4 — SysWhispers4 project documentation. MEDIUM
- [strace 7.1 release notes](#) — strace project (2026). HIGH

10.32 V32 — Virtual memory and the page cache as a fault-driven mechanism

This note argues that virtual memory is not an optimization layered on top of memory but the mechanism by which memory works at all, and that the page fault is its interface rather than an error path. The claim rests on three properties: address translation is indirect, population is lazy (a mapping can exist before any physical page backs it), and physical memory is a reclaimable cache rather than a fixed container. It supports this with the Linux kernel's own description of virtual memory as abstracting physical memory and enabling demand paging, and with Windows kernel-internals teaching material stating that the memory manager is the actual cache manager. From there the note builds a fault taxonomy — minor/soft, demand-zero, copy-on-write, and major/hard faults — with order-of-magnitude costs on each side (microseconds for RAM-resolved faults, roughly 0.1 ms on SSD and 10 ms on HDD for disk-resolved ones), and works through five implementation areas: fault accounting, why RSS is a misleading number (it double-counts shared pages), the page cache and the hazard it creates for benchmarking, `mmap`'s real cost trade-offs including a CIDR 2022 measurement of `mmap` collapsing under page-cache eviction, and the tools that make each mechanism observable on Linux and Windows.

The note is explicit about what it does not settle. It defers the atomic argument that a page fault is not an error, and the argument that `mmap` relocates rather than removes cost, to separate zettels it has not yet spun out, treating itself only as the hub that holds the topic together. It states plainly that no single memory metric (RSS, PSS, USS, working set) is simply correct — each answers a different question, and PSS in particular can rise for surviving processes when a sibling process exits, which looks like new allocation but isn't. It also flags its own model as dated in one respect: Linux has largely replaced the page as the page-cache's literal data structure with folios since kernel 5.16, so the page-granular teaching model, while still pedagogically sound, no longer matches what a reader of `mm/` source would find. Finally, it treats reclaim policy (MGLRU) as a genuinely open, contested area rather than settled material, citing an active 2026 removal proposal.

Referenced sources

- [Concepts overview — The Linux Kernel documentation](#) — The Linux Kernel Organization **HIGH**
- [Windows Kernel Internals: Cache Manager](#) — Microsoft Corporation, University of Tokyo lecture series **HIGH**
- [Page fault](#) — Grokipedia — Grokipedia **MEDIUM**
- [Working Set - Win32 apps](#) — Microsoft Learn **HIGH**
- [Copy-on-Write and the Page-Fault Lifecycle, Step by Step](#) — PandaStack **MEDIUM**
- [How to Check Memory Usage per Process in Linux](#) — GoLinuxCloud **MEDIUM**
- [perf-trace\(1\)](#) — Linux manual page — man7.org / Linux man-pages project **HIGH**
- [The Basics of Page Faults](#) — Microsoft Community Hub, AskPerf team blog **MEDIUM**
- [Windows Performance Monitor basics](#) — SQLShack **MEDIUM**
- [Memory Management Tracing Events - Win32 apps](#) — Microsoft Learn **HIGH**
- [Why does Linux, by default, overcommit memory?](#) — Unix & Linux Stack Exchange **MEDIUM**
- [Virtual vs Physical vs Resident Memory](#) — Linux Kernel Internals **MEDIUM**
- [The /proc Filesystem](#) — The Linux Kernel documentation — The Linux Kernel Organization **HIGH**
- [RAMMap - Sysinternals](#) — Microsoft Learn / Sysinternals **HIGH**
- [Accurately measuring memory usage](#) — Phusion **MEDIUM**
- [Readahead - Linux Kernel Internals](#) — Linux Kernel Internals **MEDIUM**
- [Page Cache Writeback - Linux Kernel Internals](#) — Linux Kernel Internals **MEDIUM**
- [Windows Memory Management in Windows 10 and Windows 11](#) — CyberRaiden **LOW**
- [Fair Benchmarking Considered Difficult: Common Pitfalls In Database Performance Testing](#) — Raasveldt, Holanda, Gubner & Mühleisen, DBTest **HIGH**
- [Understanding and tuning the page cache on Debian](#) — Big Iron **MEDIUM**
- [Excessive Page Faults Generated By Windows Applications](#) — Broadcom Knowledge Base **MEDIUM**
- [Common disk benchmarking mistakes](#) — Stefan Hajnoczi **HIGH**
- [Memory-Mapped I/O - Linux Kernel Internals](#) — Linux Kernel Internals **MEDIUM**
- [File Mapping - Win32 apps](#) — Microsoft Learn **HIGH**
- [Creating Named Shared Memory](#) — MicrosoftDocs/win32 repository **HIGH**

- [Memory-Mapped Files \(mmap\): A Practical Guide to Faster I/O and Shared Memory](#) — martinuke0 **MEDIUM**
- [mmap SIGBUS error and initializing the file](#) — Stack Overflow **MEDIUM**
- [MapViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn **HIGH**
- [mmap — Memory-mapped file support](#) — Python Software Foundation **HIGH**
- [Are You Sure You Want to Use MMAP in Your Database Management System?](#) — Crotty, Leis & Pavlo, CIDR **HIGH**
- [Teaching Operating Systems with FreeBSD through Tracing, Analysis, and Experimentation](#) — Neville-Neil et al., USENIX ;login: **HIGH**
- [Teaching Operating Systems Concepts with Execution Visualization](#) — Giraldeau, Dagenais & Boucheneb, ASEE **HIGH**
- [Mastering strace, perf, and eBPF — 2026 Cheat Sheet](#) — Tech Bytes **MEDIUM**
- [CS 110: Principles of Computer Systems — assign3](#) — Stanford University **HIGH**
- [Page Cache - Linux Kernel Internals](#) — Linux Kernel Internals **MEDIUM**
- [On pages and folios](#) — Jonathan Corbet, LWN.net **HIGH**
- [Reconsidering the multi-generational LRU](#) — Jonathan Corbet, LWN.net **HIGH**

10.33 V33 — File I/O durability across the OS buffering stack

This note establishes that durability is not a property of a single write call but of a sequence of independent flushes, each crossing one of four boundaries between a program and the physical medium: a language-runtime buffer, a libc/user-space buffer, the kernel's page cache, and the storage device's own cache. A successful `write()` reports only that the first kernel-side buffer accepted the bytes — a statement about custody, not persistence — and neither `close()` nor Python's `with` block closes the remaining gap. The evidence for the sharpest claim is the 2018 PostgreSQL "fsync-gate" incident: Linux's `errseq_t` mechanism reports a writeback error to only the next `fsync()` call and then discards the failed pages, so a database that retried `fsync()` after a checkpoint failure was, unknowingly, retrying against data the kernel had already thrown away. The note frames this as a genuine contract disagreement between two internally coherent positions — POSIX never specified behavior after a failed `fsync` — rather than a bug on either side, and traces how PostgreSQL, InnoDB, and WiredTiger independently converged on panicking and recovering from the write-ahead log instead of retrying. Windows is covered at the same depth: it has the identical four-layer structure but, per the note's research, no documented contract at all for what happens after a flush failure, which it argues is the worse of the two positions because no correct response can be engineered from silence.

The note is explicit about what it does not claim. It does not assert that Windows shares Linux's exact non-retryable failure class — it states plainly that nobody has publicly characterized this and Microsoft has not documented it, offering only an inference about the likely shape of the problem. It does not treat the four-layer model as complete, flagging network filesystems, memory-mapped I/O, and copy-on-write filesystems as cases the simplification does not fit. It also does not present PANIC-on-error as a universal rule, noting that this response was calibrated to 2018-era local disks and may cost availability unnecessarily on cloud block storage, where I/O errors are often transient. Two atomic claims — the exact meaning of a successful `write()`, and the non-retryability of a failed `fsync()` — are deliberately left unexpanded here, tracked instead as separate, not-yet-written notes.

Referenced sources

- [fsync\(\), Barriers, and the Hardware Durability Contract](#) — Sagar Madala / Medium (2026) **MEDIUM**
- [fsync\(2\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [Durability: Linux File APIs](#) — Evan Jones (2020) **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn **HIGH**
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft Developer Blogs (2017) **HIGH**
- [io — Core tools for working with streams](#) — Python Software Foundation **HIGH**
- [FileStreamOptions.BufferSize Property](#) — Microsoft Learn **HIGH**
- [CSE333 L07: Buffering, POSIX I/O, Syscalls](#) — University of Washington, CSE333 (2022) **MEDIUM**
- [Bug 1400144 — glibc: Performance penalty due to block size-based buffering in stdio](#) — Red Hat Bugzilla **HIGH**
- [What Buffer Flush Means in C \(and C++\)](#) — TheLinuxCode **MEDIUM**
- [Page Cache Writeback](#) — Linux Kernel Internals **MEDIUM**
- [Performance Tuning for Cache and Memory Manager Subsystems](#) — Microsoft Learn **HIGH**
- [File Caching](#) — Microsoft Learn **HIGH**
- [Atomic Commit In SQLite](#) — SQLite / D. Richard Hipp et al. **HIGH**
- [Power Loss Protection shouldn't be an enterprise exclusive](#) — How-To Geek (2026) **LOW**
- [Life of a Write](#) — Linux Kernel Internals **MEDIUM**
- [The errseq_t datatype](#) — The Linux Kernel documentation — Linux kernel documentation **HIGH**

- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curator); pgsql-hackers thread participants (2018) **HIGH**
- [Fsync Errors](#) — PostgreSQL wiki **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau / USENIX ATC (2020) **HIGH**
- [CreateFileW function — FILE_FLAG_WRITE_THROUGH and FILE_FLAG_NO_BUFFERING semantics](#) — Microsoft Learn **HIGH**
- [File Buffering](#) — Microsoft Learn **HIGH**
- [Windows Write Caching — Part 2: An Overview for Application Developers](#) — Dilip Naik (2012) **MEDIUM**
- [Discussion: does FlushFileBuffers\(\) on Windows also forget failed writes? / Files Are Hard thread](#) — Hacker News (2019) **LOW**
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — Stack Overflow (2019) **MEDIUM**
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation **HIGH**
- [Monitoring certain system calls done by a process in Windows](#) — Stack Overflow **MEDIUM**
- [DTrace on Windows](#) — Microsoft Learn **HIGH**
- [FlushViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn **HIGH**
- [Write-Ahead Logging \(SQLite\)](#) — SQLite **HIGH**

10.34 V34 — Virtual Memory and the Page Cache

This note is the lecture-theme hub for a course unit on virtual memory, and the claim it backs concerns the page cache: once a file has been read, its contents stay resident in RAM so that a later read of the same data is served from memory rather than the device. The note extends this beyond simple re-reads to readahead — the kernel detects a sequential access pattern on a file, maintains a per-file-description readahead window, and issues I/O for pages before the process actually asks for them, which is why "a sequential `read()` just happens to return faster because the data was already fetched." The evidence for this is a Linux-kernel-internals reference that quotes the relevant kernel source directly (`mm/readahead.c` and `struct file_ra_state`), and the note also states the scope boundary explicitly: readahead applies only to buffered I/O, and `O_DIRECT` bypasses the page cache and therefore bypasses readahead entirely. The

same section extends the mechanism to the Windows side (a Cache Manager and standby list producing the analogous cheap-soft-fault effect) and to writes, which are buffered and flushed in batches for the same performance reason.

The note does not claim that a cache hit is free of hazard — the opposite is one of its central points. It calls the warm page cache a benchmarking trap: cold and hot runs differ enough that they must be measured and reported separately, and a run that is meant to be cold is easily mismeasured as warm because the OS caches disk blocks regardless of whether the database process itself was restarted. It goes further and treats the obvious fix as unreliable rather than settled: dropping OS caches does not produce a truthful cold measurement, because it does not touch the storage device's own cache or a controller/HBA cache, and in a cloud VM a meaningful cache drop may not be achievable at all, since caching also happens at the virtualization host. The note's stated position is explicitly not "drop the cache before benchmarking" but that no single number can be called clean — you must state which cache layer your measurement includes, because none of them can be eliminated with certainty.

Referenced sources

- [Concepts overview — The Linux Kernel documentation](#) — The Linux Kernel Organization **HIGH**
- [Windows Kernel Internals: Cache Manager](#) — Microsoft Corporation, University of Tokyo lecture series **HIGH**
- [Page fault](#) — Grokipedia — Grokipedia **MEDIUM**
- [Working Set - Win32 apps](#) — Microsoft Learn **HIGH**
- [Copy-on-Write and the Page-Fault Lifecycle, Step by Step](#) — PandaStack **MEDIUM**
- [How to Check Memory Usage per Process in Linux](#) — GoLinuxCloud **MEDIUM**
- [perf-trace\(1\)](#) — Linux manual page — man7.org / Linux man-pages project **HIGH**
- [The Basics of Page Faults](#) — Microsoft Community Hub, AskPerf team blog **MEDIUM**
- [Windows Performance Monitor basics](#) — SQLShack **MEDIUM**
- [Memory Management Tracing Events - Win32 apps](#) — Microsoft Learn **HIGH**
- [Why does Linux, by default, overcommit memory?](#) — Unix & Linux Stack Exchange **MEDIUM**
- [Virtual vs Physical vs Resident Memory](#) — Linux Kernel Internals **MEDIUM**
- [The /proc Filesystem](#) — The Linux Kernel documentation — The Linux Kernel Organization **HIGH**
- [RAMMap - Sysinternals](#) — Microsoft Learn / Sysinternals **HIGH**
- [Accurately measuring memory usage](#) — Phusion **MEDIUM**

- [Readahead - Linux Kernel Internals](#) — Linux Kernel Internals **MEDIUM**
- [Page Cache Writeback - Linux Kernel Internals](#) — Linux Kernel Internals **MEDIUM**
- [Windows Memory Management in Windows 10 and Windows 11](#) — CyberRaiden **LOW**
- [Fair Benchmarking Considered Difficult: Common Pitfalls In Database Performance Testing](#) — Raasveldt, Holanda, Gubner & Mühleisen, DBTest **HIGH**
- [Understanding and tuning the page cache on Debian](#) — Big Iron **MEDIUM**
- [Excessive Page Faults Generated By Windows Applications](#) — Broadcom Knowledge Base **MEDIUM**
- [Common disk benchmarking mistakes](#) — Stefan Hajnoczi **HIGH**
- [Memory-Mapped I/O - Linux Kernel Internals](#) — Linux Kernel Internals **MEDIUM**
- [File Mapping - Win32 apps](#) — Microsoft Learn **HIGH**
- [Creating Named Shared Memory](#) — MicrosoftDocs/win32 repository **HIGH**
- [Memory-Mapped Files \(mmap\): A Practical Guide to Faster I/O and Shared Memory](#) — martinuke0 **MEDIUM**
- [mmap SIGBUS error and initializing the file](#) — Stack Overflow **MEDIUM**
- [MapViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn **HIGH**
- [mmap — Memory-mapped file support](#) — Python Software Foundation **HIGH**
- [Are You Sure You Want to Use MMAP in Your Database Management System?](#) — Crotty, Leis & Pavlo, CIDR **HIGH**
- [Teaching Operating Systems with FreeBSD through Tracing, Analysis, and Experimentation](#) — Neville-Neil et al., USENIX ;login: **HIGH**
- [Teaching Operating Systems Concepts with Execution Visualization](#) — Giraldeau, Dagenais & Boucheneb, ASEE **HIGH**
- [Mastering strace, perf, and eBPF — 2026 Cheat Sheet](#) — Tech Bytes **MEDIUM**
- [CS 110: Principles of Computer Systems — assign3](#) — Stanford University **HIGH**
- [Page Cache - Linux Kernel Internals](#) — Linux Kernel Internals **MEDIUM**
- [On pages and folios](#) — Jonathan Corbet, LWN.net **HIGH**
- [Reconsidering the multi-generational LRU](#) — Jonathan Corbet, LWN.net **HIGH**

10.35 V36 — The placement of the stable syscall boundary across operating systems

This note's central argument is that Linux and Windows each promise application developers a stable interface to the kernel, but draw the contractual line at different layers — Linux at the syscall trap itself, a numbered, per-architecture table documented to

never change, and Windows one layer higher at Win32, with the Native API in `ntdll` and the underlying System Service Number both undocumented and known to shift between builds. The note extends this into a broader correction: it argues Linux, not Windows, is the outlier among Unix-family systems, since FreeBSD, OpenBSD, NetBSD, Solaris/illumos and macOS all decline to guarantee syscall-number stability. Within that correction, the note weighs a specific complication about FreeBSD raised by conflicting sources: a Unix StackExchange survey, drawing on FreeBSD's own vendor wiki, places FreeBSD in the no-guarantee column, while a former FreeBSD core-team contributor argues the practical situation is closer to genuine stability, achieved via kernel build options (`COMPAT_FREEBSD*`) that keep older syscall versions available going back to FreeBSD 4. The note treats these two accounts as agreeing on the underlying mechanism — versioned syscalls in `syscalls.master` , compiled in via `COMPAT` flags — and differing only on whether a guarantee contingent on a build option counts as a guarantee at all. Its resolution, quoted above, is that FreeBSD offers strong ABI compatibility in practice through these compiled-in compatibility layers, while declining to state a formal cross-major-release guarantee the way the FreeBSD project documents its own policy.

The note is explicit that this nuance does not overturn its headline correction: FreeBSD still publishes no Linux-style promise, so it remains on the list of systems that decline to guarantee syscall-number stability. What the note does withdraw is any implication that the five non-Linux systems are equally unstable in practice — FreeBSD's `COMPAT` -flag mechanism gives it materially stronger real-world compatibility than, for instance, OpenBSD's policy of refusing non-libc syscalls outright at the kernel level. The note grades the FreeBSD question as "Contested" rather than resolved, and elsewhere flags that its broader structural explanation — that Linux alone treats the raw syscall interface as contractual because it never controlled its own libc, unlike Windows and macOS — is offered as a well-argued account rather than a settled, first-party-confirmed fact. It does not attempt to adjudicate whether a compile-time compatibility option should count as a "guarantee" in the formal sense the FreeBSD project itself withholds; that framing disagreement is left open for the reader rather than resolved in either direction.

Referenced sources

- [ABI stable symbols — The Linux Kernel documentation](#) — *The Linux Kernel Organization* **HIGH**
- [j00ru/windows-syscalls — Windows System Call Tables](#) — *Mateusz "j00ru" Jurczyk* **HIGH**
- [Windows X86-64 System Call Table \(XP/2003/Vista/7/8/10/11 and Server\)](#) — *Mateusz "j00ru" Jurczyk* **HIGH**

- [NtQuerySystemInformation function \(winternl.h\)](#) — Win32 apps — Microsoft **HIGH**
- [Calling Internal APIs](#) — Win32 apps — Microsoft **HIGH**
- [arch/x86/entry/syscalls/syscall_64.tbl](#) — Linux kernel source tree **HIGH**
- [include/uapi/asm-generic/unistd.h](#) — Linux kernel source tree **HIGH**
- [Do other Unix-like kernels have stable syscall ABIs?](#) — Unix & Linux Stack Exchange **MEDIUM**
- [FreeBSD Vendor Information](#) — The FreeBSD Project **HIGH**
- [openbsd-tech: system call origin verification](#) — OpenBSD project mailing list **HIGH**
- [all: stop using non-libc system calls on OpenBSD \(golang/go#36435\)](#) — The Go Project **HIGH**
- [syscall\(2\)](#) — NetBSD Manual Pages — The NetBSD Project **HIGH**
- [port-solaris mailing list, November 2010: syscall number reservation](#) — port-solaris list archive (Carnegie Mellon mirror) **HIGH**
- [NtTrace](#) — Native API tracing for Windows — Roger Orr **HIGH**
- [strace](#) — linux syscall tracer — The strace project **HIGH**
- [DTrace on Windows](#) — Windows drivers — Microsoft **HIGH**
- [DTrace](#) — Windows Commands — Microsoft **HIGH**
- [Inside the Native API](#) — Mark Russinovich, Sysinternals **HIGH**
- [NT OS/2 Subsystem Design Rationale \(NT Design Workbook\)](#) — Mark H. Lucovsky, Microsoft **HIGH**
- [The Sad History of the Microsoft POSIX Subsystem](#) — Brian Reiter (2010) **MEDIUM**
- [Syscall ABI compatibility: Linux vs Windows/macOS](#) — Mark Mossberg (2023) **MEDIUM**
- [Never break userspace](#) — LWN.net **HIGH**
- [Updating Syscall Tables](#) — SysWhispers4 documentation — SysWhispers4 project documentation **LOW**
- [CRT-free in 2023: tips and tricks](#) — discussion — Lobsters discussion thread **MEDIUM**
- [Architecture of Windows NT](#) — Wikipedia **MEDIUM**
- [U.S. v. Microsoft Corporation](#) — Information on the Settlement — U.S. Department of Justice, Antitrust Division **HIGH**
- [Syscalls on Windows 11](#) — hammertux **MEDIUM**

10.36 V37 — The four-layer buffering-and-durability boundary model for file I/O

This note argues that durability is not a property of a single write call but of a sequence of flushes across four independent boundaries between a program and persistent media — a language-runtime buffer, a kernel-side write buffer, the kernel's page cache, and the storage device's own cache — each with its own trigger and its own way to fail silently. Its central claim is that a successful `write()` or `WriteFile()` call only proves the kernel accepted the bytes into the first of these buffers, not that the data reached persistent media. The evidentiary core is the 2018 PostgreSQL "fsyncgate" incident: Linux reports a writeback error only to the next `fsync()` call, then marks the affected pages clean and discards their contents, so a correct-looking retry-on-`EIO` strategy silently loses data — a mechanism the note grounds in the kernel's own `errseq_t` documentation and the `pgsql-hackers` mailing-list thread, and which led PostgreSQL, InnoDB and WiredTiger to independently converge on crash-and-replay rather than retry. The note extends the same analysis to Windows, arguing that `FlushFileBuffers`, `FILE_FLAG_WRITE_THROUGH` and `FILE_FLAG_NO_BUFFERING` are routinely conflated despite crossing different boundaries, and that Windows's post-failure behavior is not simply stricter or laxer than Linux's but genuinely undocumented — arguably the worse position for a system that must not lose data. A cross-platform fault-injection study is cited as evidence that no application-level strategy among five widely used systems is currently sufficient against these failure modes on Linux.

The note is explicit about its limits. It does not claim Windows shares Linux's exact non-retryable failure class — it states that nobody has publicly characterized this and Microsoft has not documented it, rejecting both "Windows is safe" and "Windows has the same bug" as unsupported conclusions. It flags its own four-layer model as a teaching simplification that does not fully cover network filesystems, memory-mapped I/O, or copy-on-write filesystems. Two claims are marked explicitly unresolved: whether any Windows tracing tool exposes device-level flush commands as cleanly as Linux's `blktrace`, and whether Python's `os.fsync-to-_commit()` mapping on Windows actually reaches `FlushFileBuffers`, which the note calls an inference rather than a documented fact. It also notes that PostgreSQL's PANIC-on-failure response was calibrated for 2018-era local disks, and that on cloud block storage the same error is often transient — changing the cost of that response without changing its underlying correctness. These gaps are presented as findings rather than resolved speculatively.

Referenced sources

- [fsync\(\), Barriers, and the Hardware Durability Contract](#) — Sagar Madala / Medium (2026) MEDIUM

- [fsync\(2\) — Linux manual page](#) — Linux man-pages project **HIGH**
- [Durability: Linux File APIs](#) — Evan Jones (2020) **HIGH**
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn **HIGH**
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft Developer Blogs (2017) **HIGH**
- [io — Core tools for working with streams](#) — Python Software Foundation **HIGH**
- [FileStreamOptions.BufferSize Property](#) — Microsoft Learn **HIGH**
- [CSE333 L07: Buffering, POSIX I/O, Syscalls](#) — University of Washington, CSE333 (2022) **MEDIUM**
- [Bug 1400144 — glibc: Performance penalty due to block size-based buffering in stdio](#) — Red Hat Bugzilla **HIGH**
- [What Buffer Flush Means in C \(and C++\)](#) — TheLinuxCode **MEDIUM**
- [Page Cache Writeback](#) — Linux Kernel Internals **MEDIUM**
- [Performance Tuning for Cache and Memory Manager Subsystems](#) — Microsoft Learn **HIGH**
- [File Caching](#) — Microsoft Learn **HIGH**
- [Atomic Commit In SQLite](#) — SQLite / D. Richard Hipp et al. **HIGH**
- [Power Loss Protection shouldn't be an enterprise exclusive](#) — How-To Geek (2026) **LOW**
- [Life of a Write](#) — Linux Kernel Internals **MEDIUM**
- [The errseq_t datatype](#) — The Linux Kernel documentation — Linux kernel documentation **HIGH**
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curator); pgsql-hackers thread participants (2018) **HIGH**
- [Fsync Errors](#) — PostgreSQL wiki **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau / USENIX ATC (2020) **HIGH**
- [CreateFileW function — FILE_FLAG_WRITE_THROUGH and FILE_FLAG_NO_BUFFERING semantics](#) — Microsoft Learn **HIGH**
- [File Buffering](#) — Microsoft Learn **HIGH**
- [Windows Write Caching — Part 2: An Overview for Application Developers](#) — Dilip Naik (2012) **MEDIUM**
- [Discussion: does FlushFileBuffers\(\) on Windows also forget failed writes? / Files Are Hard thread](#) — Hacker News (2019) **LOW**
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — Stack Overflow (2019) **MEDIUM**

- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation

HIGH

- [Monitoring certain system calls done by a process in Windows](#) — Stack Overflow

MEDIUM

- [DTrace on Windows](#) — Microsoft Learn

HIGH

- [FlushViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn

HIGH

- [Write-Ahead Logging \(SQLite\)](#) — SQLite

HIGH

10.37 V38 — the layer at which Windows and Linux fix their syscall contract

Both operating systems promise application developers a stable interface to the kernel, but they draw that contract at different layers, and the note treats this as a structural difference rather than one system being careless and the other not. Linux fixes the contract at the syscall trap itself: syscall numbers are permanent, per-architecture entries in an in-tree table (`arch/x86/entry/syscalls/syscall_64.tbl`), and the kernel's own ABI documentation states such interfaces are "expected to never change and always be available." A program may load a syscall number into a register and trap directly, and that is a supported use. Windows fixes its contract one layer higher, at Win32. Below it, `ntdll`'s Native API is exported and named but explicitly disclaimed by Microsoft as "subject to change from one release of Windows to another," and the raw System Service Number beneath that is undocumented and does move — recomputed directly from the canonical `j00ru` syscall-table dataset, the note reports that 52% of x64 functions' numbers decrease at least once across versions, that concurrently-shipping SKUs (Windows Server 2022 vs. Windows 11 21H2) disagree on 63% of shared functions, and that even a single service pack (Vista SP0 to SP1) moved a number. The consequence: Linux guarantees a *number*, Windows guarantees a *name*, and each tracer attaches at its platform's actual stable handle — `ptrace` at the syscall interface on Linux, the `ntdll` debug interface on Windows.

The note is explicit about what this does not establish. It does not argue Windows is the anomaly — it argues Linux is, since FreeBSD, OpenBSD, NetBSD, Solaris/illumos, and macOS all likewise decline to guarantee syscall-number stability, making Windows' posture the ordinary case in the Unix-adjacent world. Several questions are left open or contested rather than resolved: whether the Win32/`ntdll` split reflects deliberate design intent or is a vestige of NT's abandoned multi-subsystem architecture; whether `ntdll` counts as a real interface or only a tracing convention that tools have grown around; whether Microsoft's disclosure of Native API prototypes in `winternl.h` was compelled by the 2002 antitrust settlement, which the note flags as an unresolved caus-

al question; and whether FreeBSD's ABI stance truly belongs beside the other four "no guarantee" systems, given a dissenting practitioner account of its compiled-in compatibility layers. The note also cautions against overstating Linux's own guarantee, noting that its "never break userspace" doctrine is hedged in the kernel's own documentation and has been publicly pushed back on.

Referenced sources

- [ABI stable symbols — The Linux Kernel documentation](#) — The Linux Kernel Organization **HIGH**
- [j00ru/windows-syscalls — Windows System Call Tables](#) — Mateusz "j00ru" Jurczyk **HIGH**
- [Windows X86-64 System Call Table \(XP/2003/Vista/7/8/10/11 and Server\)](#) — Mateusz "j00ru" Jurczyk **HIGH**
- [NtQuerySystemInformation function \(winternl.h\)](#) — Win32 apps — Microsoft **HIGH**
- [Calling Internal APIs — Win32 apps](#) — Microsoft **HIGH**
- [arch/x86/entry/syscalls/syscall_64.tbl](#) — Linux kernel source tree **HIGH**
- [include/uapi/asm-generic/unistd.h](#) — Linux kernel source tree **HIGH**
- [Do other Unix-like kernels have stable syscall ABIs?](#) — Unix & Linux Stack Exchange **MEDIUM**
- [FreeBSD Vendor Information](#) — The FreeBSD Project **HIGH**
- [openbsd-tech: system call origin verification](#) — OpenBSD project mailing list **HIGH**
- [all: stop using non-libc system calls on OpenBSD \(golang/go#36435\)](#) — The Go Project **HIGH**
- [syscall\(2\) — NetBSD Manual Pages](#) — The NetBSD Project **HIGH**
- [port-solaris mailing list, November 2010: syscall number reservation](#) — port-solaris list archive (Carnegie Mellon mirror) **HIGH**
- [NtTrace — Native API tracing for Windows](#) — Roger Orr **HIGH**
- [strace — linux syscall tracer](#) — The strace project **HIGH**
- [DTrace on Windows — Windows drivers](#) — Microsoft **HIGH**
- [DTrace — Windows Commands](#) — Microsoft **HIGH**
- [Inside the Native API](#) — Mark Russinovich, Sysinternals **HIGH**
- [NT OS/2 Subsystem Design Rationale \(NT Design Workbook\)](#) — Mark H. Lucovsky, Microsoft **HIGH**
- [The Sad History of the Microsoft POSIX Subsystem](#) — Brian Reiter (2010) **MEDIUM**

- [Syscall ABI compatibility: Linux vs Windows/macOS](#) — Mark Mossberg (2023)

MEDIUM

- [Never break userspace](#) — LWN.net HIGH
- [Updating Syscall Tables](#) — SysWhispers4 documentation — SysWhispers4 project documentation LOW
- [CRT-free in 2023: tips and tricks](#) — discussion — Lobsters discussion thread MEDIUM
- [Architecture of Windows NT](#) — Wikipedia MEDIUM
- [U.S. v. Microsoft Corporation — Information on the Settlement](#) — U.S. Department of Justice, Antitrust Division HIGH
- [Syscalls on Windows 11](#) — hammertux MEDIUM

10.38 V39 — The Trade-off Between Observation Cost and Blind Spots in Tracing Mechanisms

The note's central claim is that a tracer is an edit to the program it observes, not a window onto it: wherever a mechanism attaches — kernel syscall boundary, a breakpoint in a user-mode DLL, every instruction via a JIT'd code cache, a verified in-kernel probe, or a provider that agreed in advance to emit events — that single choice simultaneously fixes what the observation costs and what it structurally cannot see. The two are read from the same decision, not set independently. The note backs this with a comparison across six mechanisms. `ptrace` / `strace` stops the traced process twice per syscall and costs one to two orders of magnitude (442× on a syscall-saturated benchmark, ~3× on a realistic build), and cannot see anything that does not cross the syscall boundary, including `io_uring` traffic. NtTrace breakpoints the Windows Native API in user mode, avoiding administrative privileges at the cost of missing driver-originated kernel calls and serialising concurrent threads. DynamoRIO instruments every instruction via a code cache at 2–10× overhead for real clients, and can alter timing enough to manufacture bugs that do not occur natively. eBPF and DTrace attach in-kernel at nanosecond-to-low-microsecond cost and are treated as production-safe, but silently drop events under recursion or overload. ETW is a provider/consumer architecture with kernel-managed buffering, cheap and crash-immune, but shows only what a provider was written to emit in advance. Record-and-replay (`rr`) and statistical sampling are offered as alternatives that trade the classic cost/blind-spot pair for different limitations — serialised execution and resolution/rarity, respectively.

The note is explicit about where this claim does and does not bind. It states two boundary conditions under which the trade stops mattering: when event rates are low enough that cost becomes irrelevant, and when a bug is both inside a mechanism's field

of view and not timing-sensitive, so perturbation cannot change the answer. It flags one point as genuinely contested rather than settled — whether in-kernel tracing (eBPF/DTrace) is cheap enough to run continuously at fleet scale, where Brendan Gregg and Dan Luu are read as answering different questions rather than being reconciled. It also lists open gaps: no published per-call overhead measurement exists for NtTrace, ETW's headline cycle-cost figures trace to Pentium III-era Microsoft benchmarking of unstated modern validity, and no source quantifies how often eBPF's silent event drops actually occur under a characterized workload. The note treats this synthesis as evidence-backed (27 of 30 claims verified against primary sources) rather than speculative, but is careful to mark which specific claims rest on vendor self-assessment, a single practitioner blog, or unmeasured structural argument rather than independent benchmarking.

Referenced sources

- [strace Wow Much Syscall](#) — Brendan Gregg (2014) **HIGH**
- [Ptrace, Utrace, Uprobes: Lightweight, Dynamic Tracing of User Apps](#) — Keniston, Panchamukhi, Hiramatsu, Ottawa Linux Symposium (2007) **HIGH**
- [Introducing strace --seccomp-bpf](#) — Paul Chaignon (2019) **HIGH**
- [strace: fight for performance \(FOSDEM 2020 slides\)](#) — strace maintainers, FOSDEM (2020) **HIGH**
- [Sysdig vs DTrace vs Strace: A technical discussion](#) — Sysdig (n.d.) **MEDIUM**
- [NtTrace — Native API tracing for Windows](#) — Roger Orr (n.d.) **HIGH**
- [Notes and Thoughts on Strace for Windows](#) — Jarek Przygodzki (n.d.) **MEDIUM**
- [Transparent Dynamic Instrumentation](#) — Derek Bruening, Qin Zhao, Saman Amarasinghe, VEE (2012) **HIGH**
- [DynamoRIO Release Notes](#) — DynamoRIO project (n.d.) **HIGH**
- [Dynamic binary instrumentation \(DBI\) with DynamoRio](#) — Cisco Talos Intelligence (2025) **MEDIUM**
- [Tracing a Subset of Execution](#) — DynamoRIO project (n.d.) **HIGH**
- [DTrace for Linux 2016](#) — Brendan Gregg (2016) **HIGH**
- [Benchmarking Performance Overhead of DTrace on FreeBSD and eBPF on Linux](#) — Mateusz Piotrowski, AsiaBSDCon (2024) **HIGH**
- [ebpf_exporter eBPF overhead benchmark](#) — Cloudflare (n.d.) **HIGH**
- [Pitfalls of relying on eBPF for security monitoring \(and some solutions\)](#) — Artem Dinaburg, Trail of Bits (2023) **HIGH**
- [eBPF Hookpoints Gotchas \(FOSDEM 2026 slides\)](#) — FOSDEM (2026) **HIGH**
- [Instrumenting Your Code with ETW](#) — Microsoft (n.d.) **HIGH**

- [Event Tracing for Windows \(ETW\)](#) — Ryan Myers, Microsoft (archived blog, n.d.)
MEDIUM
- [Performance Analysis with ETW: Event Tracing for Windows](#) — Atomic Object (n.d.) **MEDIUM**
- [Heisenbug](#) — Wikipedia (2026) **MEDIUM**
- [Engineering Record And Replay For Deployability](#) — O'Callahan, Jones, Froyd, Huey, Noll, Partush, USENIX ATC (2017) **HIGH**
- [rr: lightweight recording & deterministic debugging](#) — rr project (2024) **HIGH**
- [Sampling v. tracing](#) — Dan Luu (n.d.) **HIGH**
- [eBPF monitoring overhead: measure and reduce it](#) — groundcover (n.d.) **MEDIUM**
- [strace of io_uring events? \(strace-devel mailing list\)](#) — strace-devel (2020) **HIGH**
- [strace\(1\)](#) — Linux manual page — man7.org (n.d.) **HIGH**
- [DTrace pid Provider Overhead](#) — Brendan Gregg (2011) **HIGH**
- [Portable, Low-Overhead Observability for io_uring](#) — arXiv preprint (2026)
MEDIUM
- [io_uring War Stories](#) — kernel-internals.org (n.d.) **MEDIUM**
- [Transparently Integrating Debugging and Dynamic Binary Instrumentation](#) — Rensselaer Polytechnic Institute (n.d.) **MEDIUM**
- [Event Tracing for Windows \(ETW\) — architecture and providers](#) — Salucci (n.d.)
MEDIUM
- [Bypassing ETW For Fun and Profit](#) — White Knight Labs (2021) **MEDIUM**
- [Support io_uring](#) — rr issue #2613 — Robert O'Callahan, rr project (2020) **MEDIUM**
- [Sampling vs deterministic profilers: which should I use?](#) — Python Developer Tooling Handbook (n.d.) **MEDIUM**

10.39 V40 — PostgreSQL's fsync() Surprise

This note establishes that Linux's `fsync()` reports a writeback error to the caller only once: the failure is recorded as a flag on the file's page-cache state, and the act of reporting it clears that flag. A retried `fsync()` therefore returns success — truthfully, by its own narrow semantics, because the kernel has already marked the failed page clean and discarded it, so there is genuinely nothing left to write. PostgreSQL's checkpointer relied on a retried, successful `fsync()` as license to recycle its write-ahead log, so a missed error meant committed data could be lost with no error ever surfacing to the operator. The note treats this as well-established rather than merely reported: the mechanism is confirmed by the affected developers, reproduced under `strace` with a fault-injection tool in the same discussion thread, and independently verified by

a later peer-reviewed study that fault-injected three separate filesystems. The response was uniform — PostgreSQL, MySQL's InnoDB, and MongoDB's WiredTiger each independently replaced `fsync` retry with a hard crash (PANIC/abort) during 2018, converging on the same underlying principle: once a process's in-memory belief about what has reached disk may be wrong, the only sound move is to discard that belief and rebuild it from the write-ahead log, the one artefact whose durability is independently guaranteed.

The note is explicit about what remains unresolved. The crash-and-replay fix narrows the exposure but does not close it: PostgreSQL's own commit message names a residual inode-eviction race, and a later systematic study found that no application — PostgreSQL included — handles `fsync()` failure perfectly even after the 2018 fixes. Whether the kernel or the applications were "in the wrong" is treated as genuinely unsettled rather than decided: POSIX specifies the success case for `fsync()` but explicitly declines to constrain what happens after a failure, so the note's own critical assessment frames the episode as a mismatch between two internally coherent failure models meeting across a layering boundary, not a defect on either side. It also flags that the long-term fix every party endorsed — moving databases to direct I/O to bypass the page cache entirely — has still not shipped as a production path years later, so the crash-based fix should be read as a durable stopgap rather than a closed question.

Referenced sources

- [PostgreSQL's fsync\(\) surprise](#) — Jonathan Corbet / LWN.net (2018) **HIGH**
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (archive of the pgsql-hackers thread, 2018) **HIGH**
- [PostgreSQL commit 9ccdd7f66e3324d2b6d3dec282cfa9ff084083f1 — "PANIC on fsync\(\) failure"](#) — Craig Ringer / Thomas Munro, PostgreSQL Global Development Group (2018) **HIGH**
- [PostgreSQL documentation — data_sync_retry](#) — PostgreSQL Global Development Group **HIGH**
- [MySQL Bug #90296 — "Hard error should be report when fsync\(\) return EIO"](#) — Oracle / MySQL bug tracker (2018) **HIGH**
- [mysql-server commit 8590c8e12a3374eeccb547359750a9d2a128fa6a — Bug#27805553](#) — Oracle / MySQL Server (2018) **HIGH**
- [WiredTiger commit ae8bccce3d8a8248afa0e4e0cf67674a43dede96 — WT-4045 "Don't retry fsync calls after EIO failure"](#) — Keith Bostic / Alex Gorrod, MongoDB WiredTiger (2018) **HIGH**
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau, USENIX ATC '20 (2020); extended in ACM Transactions on Storage 17 (2021) **HIGH**

- [fs: introduce new writeback error reporting and convert existing API as a wrapper around it](#) — Jeff Layton, linux-fsdevel (2017) **HIGH**
- [PATCH v2 3/5 vfs: track per-sb writeback errors and report them to syncfs](#) — Jeff Layton, LKML (2018) **HIGH**
- [Linux Filesystems API summary](#) — [filemap_set_wb_err](#) / [filemap_check_wb_err](#) — The Linux Kernel documentation **HIGH**
- [PostgreSQL 18 and beyond: From AIO to Direct IO?](#) — Cybertec PostgreSQL International (2025) **MEDIUM**
- [debug_io_direct](#) — pgPedia — pgPedia, a PostgreSQL Encyclopedia **MEDIUM**
- [All Your GUCs in a Row: data_sync_retry](#) — The Build (PostgreSQL Experts) **MEDIUM**
- [fsync](#) — POSIX Programmer's Manual — IEEE Std 1003.1 / The Open Group Base Specifications **HIGH**

10.40 V41 — PostgreSQL's fsync() Error-Reporting Failure

This note summarises reporting and follow-up primary sources establishing that Linux's fsync() reports a writeback failure only once: the kernel marks a failed page's error flag, and the act of returning that error to a caller clears it, discarding the page as "clean" even though its data was never written. A process that retries fsync() after seeing a failure therefore does not re-attempt the failed I/O — it queries an already-reset flag over a page cache the data has already left, and gets a false success. The note documents that this was not hypothetical: PostgreSQL's checkpointer retried after an EIO, saw success, and recycled its write-ahead log, permanently losing committed data. Evidence spans the original 2018 LWN report, a reproducible strace trace from the pgsql-hackers thread, and a later peer-reviewed fault-injection study across ext4, XFS, and Btrfs. In direct response, PostgreSQL, InnoDB, and WiredTiger — three independently built storage engines — each replaced retry with an immediate crash/panic during 2018, so an unreported failure forces recovery from the write-ahead log rather than trusting a possibly-corrupted in-memory model.

The note is explicit about what it does not settle. Whether the kernel's one-shot error semantics are a defect or a defensible, if undocumented, response to an unspecified failure path is presented as an unresolved dispute, since POSIX constrains only the success case and is silent on what a caller may assume after failure. A later kernel-side fix (per-descriptor error tracking, merged in Linux 4.13) is described as making error *reporting* more reliable without making the *data* recoverable, and the note flags residual gaps — for instance, a process that opens a file only after an error occurred can still miss it entirely. Direct I/O is named as the long-term fix both kernel and database developers

endorsed, but the note observes that, years on, it remains an unfinished, non-production feature. Drawing on a later academic study, the note treats the broader question as still open: no examined application-level failure-handling strategy is judged fully sufficient, so the 2018 fixes are framed as converting silent data loss into a loud, recoverable crash rather than as a complete solution.

Referenced sources

- PostgreSQL's fsync() surprise — Jonathan Corbet / LWN.net (2018) — <https://lwn.net/Articles/752063/> **HIGH**
- Fsyncgate: errors on fsync are unrecoverable — Dan Luu — <https://danluu.com/fsyncgate/> **HIGH**
- PostgreSQL commit 9ccdd7f66e3324d2b6d3dec282cfa9ff084083f1 — "PANIC on fsync() failure" — Craig Ringer / Thomas Munro — <https://git.postgresql.org/git-web/?p=postgresql.git;a=commit;h=9ccdd7f66e3324d2b6d3dec282cfa9ff084083f1> **HIGH**
- PostgreSQL documentation — `data_sync_retry` — PostgreSQL Global Development Group — <https://www.postgresql.org/docs/current/runtime-config-error-handling.html> **HIGH**
- MySQL Bug #90296 — "Hard error should be report when fsync() return EIO" — Oracle / MySQL bug tracker — <https://bugs.mysql.com/bug.php?id=90296> **HIGH**
- mysql-server commit 8590c8e12a3374eecb547359750a9d2a128fa6a — Bug#27805553 — Oracle / MySQL Server — <https://github.com/mysql/mysql-server/commit/8590c8e12a3374eecb547359750a9d2a128fa6a> **HIGH**
- WiredTiger commit ae8bccce3d8a8248afa0e4e0cf67674a43dede96 — WT-4045 "Don't retry fsync calls after EIO failure" — Keith Bostic / Alex Gorrod, MongoDB WiredTiger — <https://github.com/wiredtiger/wiredtiger/commit/ae8bccce3d8a8248afa0e4e0cf67674a43dede96> **HIGH**
- Can Applications Recover from fsync Failures? — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau, USENIX ATC '20 — <https://www.usenix.org/system/files/atc20-rebello.pdf> **HIGH**
- fs: introduce new writeback error reporting and convert existing API as a wrapper around it — Jeff Layton, linux-fsdevel (2017) — <https://lwn.net/Articles/722250/> **HIGH**
- PATCH v2 3/5 vfs: track per-sb writeback errors and report them to syncfs — Jeff Layton, LKML (2018) — <https://lkml.org/lkml/2018/5/31/354> **HIGH**
- Linux Filesystems API summary — `filemap_set_wb_err` / `filemap_check_wb_err` — The Linux Kernel documentation — <https://www.kernel.org/doc/html/v5.1/filesystems/api-summary.html> **HIGH**

- PostgreSQL 18 and beyond: From AIO to Direct IO? — Cybertec PostgreSQL International (2025) — <https://www.cybertec-postgresql.com/en/postgresql-18-and-beyond-from-aio-to-direct-io/> **MEDIUM**
- `debug_io_direct` — pgPedia — https://pgpedia.info/i/debug_io_direct.html **MEDIUM**
- All Your GUCs in a Row: `data_sync_retry` — The Build (PostgreSQL Experts) — <https://thebuild.com/blog/all-your-gucs-in-a-row-datasyncretry/> **MEDIUM**
- `fsync` — POSIX Programmer's Manual — IEEE Std 1003.1 / The Open Group Base Specifications — https://www.unix.com/man_page/linux/3/fsync/ **HIGH**

10.41 V43 — The differing layer at which Windows and Linux fix the stable syscall boundary

This note's central claim is that Windows and Linux both promise a stable interface to the kernel, but nominate different layers for that promise — Linux at the syscall trap itself, a numbered, append-only table, and Windows one layer up at Win32, with everything below it (`ntdll`, and the raw kernel transition beneath that) explicitly disclaimed by Microsoft as subject to change. The quoted span backs the note's correction to how this difference is usually framed: it is tempting to call Windows the odd platform out, but the note argues this is backwards. Surveying the rest of the Unix family, it finds each system explicitly declining the Linux-style number guarantee: FreeBSD's own vendor documentation promises no ABI compatibility across major releases; OpenBSD mechanically enforces syscall origin verification, refusing traps that do not originate in the libc stub; NetBSD's manual page tells developers not to use the raw call in normal applications; Solaris/illumos engineers call syscall numbers "undocumented/uncommitted"; and Apple guarantees compatibility only at the `libSystem` layer, not the kernel transition. The Go runtime supplies corroborating evidence, having been forced off direct syscalls on Solaris, then macOS, then OpenBSD in turn. The note's conclusion is that Linux, not Windows, is the exception among mainstream systems in treating the raw syscall interface as a public contract — a consequence, it argues, of Linux historically not controlling its own userspace C library the way Windows and macOS control theirs.

The note is careful about the limits of this correction. It does not claim Windows syscall numbers are documented anywhere — they remain undocumented, demonstrably renumbered between builds and even between concurrently shipping SKUs — and it leaves open whether that renumbering is purely mechanical (a side effect of an alphabetically ordered service table) or partly deliberate, as well as whether any Windows-side artifact plays the role of Linux's in-tree table. Nor does it treat the five corroborat-

ing systems as uniformly non-guaranteeing: FreeBSD's place on the list is flagged as contested, since a former core-team contributor argues the compiled-in `COMPAT_FREEBSD*` mechanism amounts to a practical stability guarantee even without a formal cross-release promise, and the note reports that disagreement rather than resolving it. Two further questions adjacent to this span — whether Microsoft's `winternl.h` disclosure was compelled by the 2002 antitrust settlement, and whether Windows' current DTrace-based syscall tracer will ever become default-on — are marked explicitly as unsettled.

Referenced sources

- [ABI stable symbols — The Linux Kernel documentation](#) — The Linux Kernel Organization **HIGH**
- [j00ru/windows-syscalls — Windows System Call Tables](#) — Mateusz "j00ru" Jurczyk **HIGH**
- [Windows X86-64 System Call Table \(XP/2003/Vista/7/8/10/11 and Server\)](#) — Mateusz "j00ru" Jurczyk **HIGH**
- [NtQuerySystemInformation function \(winternl.h\)](#) — Win32 apps — Microsoft **HIGH**
- [Calling Internal APIs — Win32 apps](#) — Microsoft **HIGH**
- [arch/x86/entry/syscalls/syscall_64.tbl](#) — Linux kernel source tree **HIGH**
- [include/uapi/asm-generic/unistd.h](#) — Linux kernel source tree **HIGH**
- [Do other Unix-like kernels have stable syscall ABIs?](#) — Unix & Linux Stack Exchange **MEDIUM**
- [FreeBSD Vendor Information](#) — The FreeBSD Project **HIGH**
- [openbsd-tech: system call origin verification](#) — OpenBSD project mailing list **HIGH**
- [all: stop using non-libc system calls on OpenBSD \(golang/go#36435\)](#) — The Go Project **HIGH**
- [syscall\(2\) — NetBSD Manual Pages](#) — The NetBSD Project **HIGH**
- [port-solaris mailing list, November 2010: syscall number reservation](#) — port-solaris list archive (Carnegie Mellon mirror) **HIGH**
- [NtTrace — Native API tracing for Windows](#) — Roger Orr **HIGH**
- [strace — linux syscall tracer](#) — The strace project **HIGH**
- [DTrace on Windows — Windows drivers](#) — Microsoft **HIGH**
- [DTrace — Windows Commands](#) — Microsoft **HIGH**
- [Inside the Native API](#) — Mark Russinovich, Sysinternals **HIGH**

- [NT OS/2 Subsystem Design Rationale \(NT Design Workbook\)](#) — Mark H. Lucovsky, Microsoft **HIGH**
- [The Sad History of the Microsoft POSIX Subsystem](#) — Brian Reiter (2010) **MEDIUM**
- [Syscall ABI compatibility: Linux vs Windows/macOS](#) — Mark Mossberg (2023) **MEDIUM**
- [Never break userspace](#) — LWN.net **HIGH**
- [Updating Syscall Tables](#) — SysWhispers4 documentation — SysWhispers4 project documentation **LOW**
- [CRT-free in 2023: tips and tricks — discussion](#) — Lobsters discussion thread **MEDIUM**
- [Architecture of Windows NT](#) — Wikipedia **MEDIUM**
- [U.S. v. Microsoft Corporation — Information on the Settlement](#) — U.S. Department of Justice, Antitrust Division **HIGH**
- [Syscalls on Windows 11](#) — hammertux **MEDIUM**

10.42 V44 — The dated hardware behind the FlexSC eviction measurements

The note's central evidence for its core claim — that a system call's dominant cost is paid after the call returns, in evicted cache and TLB state, rather than in the crossing itself — is Soares and Stumm's FlexSC paper (USENIX OSDI 2010). FlexSC's Table 1 measured the processor-structure footprint of individual Linux system calls: a single `stat` call retires 4,972 instructions in 13,585 cycles (IPC 0.37) while evicting 32 L1 instruction-cache lines, 186 L1 data-cache lines, 660 L2 lines, 2,559 L3 lines, and 21 data-TLB entries; `open+write+close` reaches 9,921 instructions and evicts 5,105 L3 lines. The paper states directly that at moderate call frequencies (once per 2,000 to 100,000 instructions) this indirect eviction cost, not the transition itself, is the dominant source of user-mode IPC degradation. The note flags, in its Knowledge Gaps, that these specific figures come from 2010-era hardware — cache hierarchies, TLB sizes, and mitigation state have all changed materially since, most concretely through the later introduction of Kernel Page-Table Isolation, which the note documents elsewhere as adding a mandatory CR3 reload on every crossing. It nonetheless treats the qualitative shape of the finding — indirect cost dominating direct cost at moderate frequency — as likely to still hold, on the grounds that the underlying mechanism, a crossing evicting working-set state that user code must later refill, has not changed even though its magnitude plausibly has.

The note is explicit that this is a judgment call about durability, not a re-measurement, and it draws a firm boundary around what it will still assert. It does not claim that the specific eviction counts — 32 i-cache lines, 2,559 L3 lines, and so on — hold on current hardware, stating outright that "the specific eviction counts should not be quoted as current." It also leaves open exactly where the frequency threshold sits at which direct cost (the exception and pipeline-flush cost) overtakes indirect cost on modern hardware, noting elsewhere that this crossover point is unquantified and has to be measured per workload rather than derived from the sources it cites. The note further scopes the FlexSC evidence to Linux alone: no comparable published cache/TLB-eviction footprint exists for Windows, so any claim about Windows' indirect-cost magnitude is inferred from shared x86-64 hardware behavior, not measured directly. Architectures other than x86-64, including ARM64, are outside what these sources cover at all.

Referenced sources

- [SYSCALL — Fast System Call \(Intel 64 and IA-32 Architectures Software Developer's Manual\) — Intel Corporation](#) **HIGH**
- [The Definitive Guide to Linux System Calls — packagecloud](#) **MEDIUM**
- [FlexSC: Flexible System Call Scheduling with Exception-Less System Calls — Livio Soares and Michael Stumm, University of Toronto — USENIX OSDI \(2010\)](#) **HIGH**
- [System calls on Windows x64 — n4r1b \(2019\)](#) **MEDIUM**
- [Page Table Isolation \(PTI\) — The Linux Kernel documentation — Linux kernel maintainers](#) **HIGH**
- [KPTI/KAISER Meltdown Initial Performance Regressions — Brendan Gregg, Netflix \(2018\)](#) **MEDIUM**
- [Kernel page-table isolation — Wikipedia](#) **LOW**
- [What Makes System Calls Expensive: A Linux Internals Deep Dive — Abhinav Upadhyay \(2025\)](#) **MEDIUM**
- [Two frequently used system calls are ~77% slower on AWS EC2 — packagecloud](#) **MEDIUM**
- [Randomizing the KUSER_SHARED_DATA Structure on Windows — Microsoft Security Response Center \(2022\)](#) **HIGH**
- [io_uring for High-Performance DBMSs: When and How to Use It — Jasny et al. \(2025\)](#) **MEDIUM**
- [SubmitIoRing function \(io_uring.h\) — Microsoft Learn](#) **HIGH**
- [One I/O Ring to Rule Them All: A Full Read/Write Exploit Primitive on Windows 11 — Yarden Shafir, Winsider Seminars & Solutions](#) **MEDIUM**

- [io](#) — Core tools for working with streams (Python documentation) — Python Software Foundation **HIGH**
- [FileStreamOptions.BufferSize Property \(System.IO\)](#) — Microsoft Learn **HIGH**
- [Buffered I/O \(stdio library\) vs. Unbuffered I/O \(Sys. Calls\)](#) — circuitlabs (2025) **LOW**
- [15-213 Lecture 16 Activity Solutions: Unix I/O and Standard I/O](#) — Carnegie Mellon University, [15-213 Introduction to Computer Systems](#) **HIGH**
- [Meltdown/Spectre Benchmarks](#) — Open Telekom Cloud Image Factory — T-Systems / Open Telekom Cloud **MEDIUM**
- [On the Costs of Syscalls \(2021\)](#) — Hacker News discussion — Hacker News commenters **LOW**
- [On the Costs of Syscalls](#) — Georg Sauthoff (2021) **MEDIUM**
- [Inspection of I/O Operations from System Call Traces using Directly-Follows-Graph](#) — Aravind Sankaran, Ilya Zhukov, Wolfgang Frings, Paolo Bientinesi (2024) **MEDIUM**
- [A Deep Dive into Zero-Copy Networking and io_uring](#) — Jatin Umamtora, Medium **LOW**
- [Comment thread on syscall cost and out-of-order execution](#) — Hacker News — commenter "anarazel" (2021) **LOW**

10.43 V45 — The system call as a per-crossing cost mechanism, not a per-byte one

This note argues that a system call is not simply an expensive function call, and the claim this section backs is its sharpest practical consequence: the transition cost and the cache/TLB eviction footprint of a kernel crossing are paid per crossing and are largely independent of how much data the call moves. Because of that, total I/O cost is better modelled as a fixed cost per crossing plus a marginal cost per byte than as a function of bytes moved alone — for small payloads, the fixed term dominates. The evidence behind this is a measured processor-footprint study of individual Linux system calls: a single `stat` call retires under 5,000 instructions yet evicts thousands of cache lines and dozens of TLB entries, and that lost state, not the instruction count, is what user-mode code pays for afterward. The note extends the same point to demonstrated practice — buffering and syscalls that combine multiple operations into one crossing both reduce cost by cutting the number of crossings while moving identical numbers of bytes, and default buffer sizes in both Python and .NET are shown to silently set that crossing count.

The note is explicit about where this stops holding. When a call is served through Linux's vDSO or the equivalent Windows shared page, there is no crossing at all, and the call behaves like an ordinary function call. When crossings are batched, as with `io_uring` or Windows I/O Rings, the fixed cost is amortized toward the marginal cost, though batching can itself be slower than the crossing it replaces at very small batch sizes. At extremely high call frequency, the direct cost of the transition overtakes the indirect eviction cost as the dominant term. And when a workload is bound by device latency rather than CPU, neither crossing count nor bytes moved is the binding constraint. The note also flags that absolute per-crossing cost figures vary by orders of magnitude across sources depending on mitigation state and hardware, so it treats "count crossings, not bytes" as a qualitatively robust rule rather than a claim backed by any single fixed number.

Referenced sources

- [SYSCALL — Fast System Call \(Intel 64 and IA-32 Architectures Software Developer's Manual\)](#) — Intel Corporation **HIGH**
- [The Definitive Guide to Linux System Calls](#) — packagecloud **MEDIUM**
- [FlexSC: Flexible System Call Scheduling with Exception-Less System Calls](#) — Livio Soares and Michael Stumm, University of Toronto — USENIX OSDI (2010) **HIGH**
- [System calls on Windows x64](#) — n4r1b (2019) **MEDIUM**
- [Page Table Isolation \(PTI\)](#) — The Linux Kernel documentation — Linux kernel maintainers **HIGH**
- [KPTI/KAISER Meltdown Initial Performance Regressions](#) — Brendan Gregg, Netflix (2018) **MEDIUM**
- [Kernel page-table isolation](#) — Wikipedia **LOW**
- [What Makes System Calls Expensive: A Linux Internals Deep Dive](#) — Abhinav Upadhyay (2025) **MEDIUM**
- [Two frequently used system calls are ~77% slower on AWS EC2](#) — packagecloud **MEDIUM**
- [Randomizing the KUSER_SHARED_DATA Structure on Windows](#) — Microsoft Security Response Center (2022) **HIGH**
- [io_uring for High-Performance DBMSs: When and How to Use It](#) — Jasny et al. (2025) **MEDIUM**
- [SubmitIoRing function \(ioringapi.h\)](#) — Microsoft Learn **HIGH**
- [One I/O Ring to Rule Them All: A Full Read/Write Exploit Primitive on Windows 11](#) — Yarden Shafir, Winsider Seminars & Solutions **MEDIUM**
- [io](#) — Core tools for working with streams (Python documentation) — Python Software Foundation **HIGH**

- [FileStreamOptions.BufferSize Property \(System.IO\)](#) — Microsoft Learn **HIGH**
- [Buffered I/O \(stdio library\) vs. Unbuffered I/O \(Sys. Calls\)](#) — circuitlabs (2025) **LOW**
- [15-213 Lecture 16 Activity Solutions: Unix I/O and Standard I/O](#) — Carnegie Mellon University, 15-213 Introduction to Computer Systems **HIGH**
- [Meltdown/Spectre Benchmarks](#) — Open Telekom Cloud Image Factory — T-Systems / Open Telekom Cloud **MEDIUM**
- [On the Costs of Syscalls \(2021\)](#) — Hacker News discussion — Hacker News commenters **LOW**
- [On the Costs of Syscalls](#) — Georg Sauthoff (2021) **MEDIUM**
- [Inspection of I/O Operations from System Call Traces using Directly-Follows-Graph](#) — Aravind Sankaran, Ilya Zhukov, Wolfgang Frings, Paolo Bientinesi (2024) **MEDIUM**
- [A Deep Dive into Zero-Copy Networking and io_uring](#) — Jatin Umamtora, Medium **LOW**
- [Comment thread on syscall cost and out-of-order execution](#) — Hacker News — commenter "anarazel" (2021) **LOW**

10.44 V46 — Why a system call's per-crossing cost cannot be quoted as one number

The note argues that a system call's cost cannot be captured as one fixed number, and offers a cluster of practitioner-collected measurements as evidence. Reported per-crossing figures span roughly two orders of magnitude — around 45 ns unmitigated rising toward 200 ns with kernel page-table isolation enabled on a 4.14 kernel, about 80 ns for the most trivial calls, and, at finer grain, "a 76–668 cycle spread for `close` and raw `syscall` invocations measured across fifteen different hosts," drawn from a single-author cross-CPU microbenchmark table. The note treats this spread as direct support for its central claim rather than as noise: a kernel crossing's cost is not a property of the call itself but of the caller's context — CPU generation, kernel mitigation state, and working-set size. It backs that framing with mechanism-level evidence elsewhere in the note: kernel page-table isolation forces a mandatory CR3 reload documented at "on the order of a hundred cycles" on every entry and exit, and a peer-reviewed measurement of a single `stat` call shows it retiring 4,972 instructions in 13,585 cycles while evicting 2,559 L3 cache lines — the eviction, not the instruction count, being what later user-mode code pays for.

The note is careful not to let this figure carry more weight than its source supports. It grades the entire cluster of absolute per-crossing numbers, this spread included, as only "Partially verified," adding plainly that "the methodology behind several of these is not fully documented." It draws no optimization threshold from the number — only the more modest conclusion that no single reported cost should be quoted as *the* syscall cost. Nor does it claim the variability is universal: elsewhere the note documents regimes where the crossing is avoided entirely (Linux's vDSO, Windows' KUSER_SHARED_DATA), amortized across a batch (io_uring, Windows I/O Rings), or dominated instead by the direct cost of the exception itself at very high call frequencies. Finally, essentially all of the note's cycle-level evidence, including this spread, is drawn from x86-64 measurements; the note states outright that no source it consulted measures the equivalent cost decomposition on ARM64.

Referenced sources

- [SYSCALL — Fast System Call \(Intel 64 and IA-32 Architectures Software Developer's Manual\) — Intel Corporation](#) **HIGH**
- [The Definitive Guide to Linux System Calls — packagecloud](#) **MEDIUM**
- [FlexSC: Flexible System Call Scheduling with Exception-Less System Calls — Livio Soares and Michael Stumm, University of Toronto — USENIX OSDI \(2010\)](#) **HIGH**
- [System calls on Windows x64 — n4r1b \(2019\)](#) **MEDIUM**
- [Page Table Isolation \(PTI\) — The Linux Kernel documentation — Linux kernel maintainers](#) **HIGH**
- [KPTI/KAISER Meltdown Initial Performance Regressions — Brendan Gregg, Netflix \(2018\)](#) **MEDIUM**
- [Kernel page-table isolation — Wikipedia](#) **LOW**
- [What Makes System Calls Expensive: A Linux Internals Deep Dive — Abhinav Upadhyay \(2025\)](#) **MEDIUM**
- [Two frequently used system calls are ~77% slower on AWS EC2 — packagecloud](#) **MEDIUM**
- [Randomizing the KUSER_SHARED_DATA Structure on Windows — Microsoft Security Response Center \(2022\)](#) **HIGH**
- [io_uring for High-Performance DBMSs: When and How to Use It — Jasny et al. \(2025\)](#) **MEDIUM**
- [SubmitIoRing function \(ioringapi.h\) — Microsoft Learn](#) **HIGH**
- [One I/O Ring to Rule Them All: A Full Read/Write Exploit Primitive on Windows 11 — Yarden Shafir, Winsider Seminars & Solutions](#) **MEDIUM**
- [io — Core tools for working with streams \(Python documentation\) — Python Software Foundation](#) **HIGH**

- [FileStreamOptions.BufferSize Property \(System.IO\)](#) — *Microsoft Learn* **HIGH**
- [Buffered I/O \(stdio library\) vs. Unbuffered I/O \(Sys. Calls\)](#) — *circuitlabs (2025)* **LOW**
- [15-213 Lecture 16 Activity Solutions: Unix I/O and Standard I/O](#) — *Carnegie Mellon University, 15-213 Introduction to Computer Systems* **HIGH**
- [Meltdown/Spectre Benchmarks](#) — *Open Telekom Cloud Image Factory* — *T-Systems / Open Telekom Cloud* **MEDIUM**
- [On the Costs of Syscalls \(2021\)](#) — *Hacker News discussion* — *Hacker News commenters* **LOW**
- [On the Costs of Syscalls](#) — *Georg Sauthoff (2021)* **MEDIUM**
- [Inspection of I/O Operations from System Call Traces using Directly-Follows-Graph](#) — *Aravind Sankaran, Ilya Zhukov, Wolfgang Frings, Paolo Bientinesi (2024)* **MEDIUM**
- [A Deep Dive into Zero-Copy Networking and io_uring](#) — *Jatin Umamtora, Medium* **LOW**
- [Comment thread on syscall cost and out-of-order execution](#) — *Hacker News* — *commenter "anarazel" (2021)* **LOW**

10.45 V47 — NtTrace, a user-mode Windows native-API tracer

NtTrace is a Windows-side counterpart to `strace`: a user-mode tracer that reports on calls into `ntdll.dll`, the DLL sitting between application code and the Windows kernel. It attaches to the target process as a debugger, through the Windows debug interface, and places software breakpoints in `ntdll.dll` itself — a design chosen deliberately so the tool needs no device driver and no administrator rights. That design decision has a direct, structural consequence: driver-originated kernel calls and any direct `int 2e` / `syscall` / `sysenter` stub are invisible, because such calls never execute the `ntdll` code the tool has instrumented. The author's own project documentation states this limitation explicitly, as one of two blind spots that follow from staying in user mode. The note treats the gap as more than a technical footnote — it ties the direct-syscall half of the pair to established security literature describing direct-syscall invocation as the standard technique attackers use to evade exactly this kind of user-mode monitoring, so the blind spot the quote describes is not hypothetical but the documented basis of a real evasion technique.

The note is careful not to overstate what this costs. NtTrace's own documentation states the blind spot as a bullet point rather than treating it as undermining the tool, and the note's Critical Assessment weighs two readings without settling flatly on either: one, that NtTrace therefore traces a convention rather than the true kernel interface; the other — the reading the note judges stronger — that `ntdll` is the boundary Windows actually maintains, and that direct-syscall evasion is a violation of that convention rather

than evidence the convention isn't real. The note does not quantify how often driver-originated calls or direct-syscall stubs occur in practice, and it explicitly restricts NtTrace's reliable scope to cooperative programs, leaving adversarial or self-obfuscating code as a case the trace cannot speak to. No benchmark or measurement of the blind spot's practical frequency appears anywhere in the source material — the direction of the limitation is well established, its real-world magnitude is not.

Referenced sources

- [NtTrace — Native API tracing for Windows](#) — Roger Orr (continuously maintained) **HIGH**
- [rogerorr/NtTrace — repository README](#) — Roger Orr (2015–, actively maintained) **HIGH**
- [Home · rogerorr/NtTrace Wiki](#) — Roger Orr **HIGH**
- [Notes and Thoughts on Strace for Windows](#) — Jarek Przygodzki **MEDIUM**
- [AWStrace: Another Windows Strace](#) — downwithup (2026) **MEDIUM**
- [StraceNT – System Call Tracer for Windows NT](#) — Pankaj Garg **MEDIUM**
- [System Call Tracer for Windows \(drstrace\)](#) — Dr. Memory / DynamoRIO project **HIGH**
- [mxmssh/drltrace — README](#) — Maksim Shudrak **MEDIUM**
- [Introducing QTrace, a "zero knowledge" system call tracer](#) — Roberto Paleari (2014) **MEDIUM**
- [strace Wow Much Syscall](#) — Brendan Gregg (2014) **HIGH**
- [Introducing strace --seccomp-bpf](#) — Paul Chaignon (2019) **HIGH**
- [Do other Unix-like kernels have stable syscall ABIs?](#) — Unix & Linux Stack Exchange (2018–) **MEDIUM**
- [A 10-minute guide to the Linux ABI](#) — Alison Chaiken, opensource.com (2022) **HIGH**
- [A Deep Dive Into Malicious Direct Syscall Detection](#) — Or Chechik and Ofir Ozer, Palo Alto Networks (2024) **HIGH**
- [Overload Content by Article — ACCU](#) — ACCU (Association of C and C++ Users) **HIGH**
- [Windows System Calls & Native API: A Guide to Assembly Calling Conventions Explained](#) — linuxvox.com **LOW**
- [mandiant/STrace — A DTrace on Windows Reimplementation](#) — Mandiant **MEDIUM**

10.46 V48 — NtTrace, a user-mode Windows native-API tracer

NtTrace is a tool that traces the calls a Windows process makes into `ntdll.dll`, the DLL that sits between application code and the NT kernel. It works by attaching to the target process as a debugger, through the Windows debug interface (the same `WaitForDebugEvent / ContinueDebugEvent` machinery a symbolic debugger uses), and placing software breakpoints around the native call stubs inside `ntdll`. When a breakpoint fires, the debug interface delivers an exception event, NtTrace suspends the thread, reads its registers and memory to recover arguments and return values, prints a decoded line, and resumes it. The claim this appendix centres on — that simultaneous calls from multiple threads are serialised in the trace because the debug interface is event-based — follows directly from that mechanism: the debugger receives one exception event at a time, so it processes concurrent breakpoint hits sequentially rather than in parallel, regardless of how the traced threads actually interleaved. The claim is stated as a single design-consequence sentence in the tool's own repository README, written by the tool's author, and it is corroborated by the general architecture the documentation describes elsewhere (event-based debug attachment, one thread suspended and resumed per traced call).

The note is explicit that this is a cost of the design, not a neutral side effect: the observed ordering in a trace is an artefact of the tracer, not of the program, so a trace can make a racy program look deterministic or make a particular interleaving look canonical when it was manufactured by the tracer's own serialisation. What the source does not do is quantify the effect — it states the serialisation in one sentence, with no measurement of how many threads, how much lock contention, or what workload shape makes the distortion severe versus negligible. It also does not claim this is unique to NtTrace; the note treats it as one instance of a more general and, on Windows, unsettled question — namely, how much timing and ordering data taken under any breakpoint-based tracer can be trusted at all. The note treats that broader question as open rather than resolved by this source.

Referenced sources

- [NtTrace — Native API tracing for Windows](#), Roger Orr **HIGH**
- [rogerorr/NtTrace — repository README](#), Roger Orr **HIGH**
- [Home · rogerorr/NtTrace Wiki](#), Roger Orr **HIGH**
- [Notes and Thoughts on Strace for Windows](#), Jarek Przygodzki **MEDIUM**
- [AWStrace: Another Windows Strace](#), downwithup **MEDIUM**
- [StraceNT – System Call Tracer for Windows NT](#), Pankaj Garg **MEDIUM**
- [System Call Tracer for Windows \(drstrace\)](#), Dr. Memory / DynamoRIO project **HIGH**

- [mxmssh/drltrace — README](#), Maksim Shudrak **MEDIUM**
- [Introducing QTrace, a "zero knowledge" system call tracer](#), Roberto Paleari **MEDIUM**
- [strace Wow Much Syscall](#), Brendan Gregg **HIGH**
- [Introducing strace --seccomp-bpf](#), Paul Chaignon **HIGH**
- [Do other Unix-like kernels have stable syscall ABIs?](#), Unix & Linux Stack Exchange **MEDIUM**
- [A 10-minute guide to the Linux ABI](#), Alison Chaiken, opensource.com **HIGH**
- [A Deep Dive Into Malicious Direct Syscall Detection](#), Or Chechik and Ofir Ozer, Palo Alto Networks **HIGH**
- [Overload Content by Article — ACCU](#), ACCU (Association of C and C++ Users) **HIGH**
- [Windows System Calls & Native API: A Guide to Assembly Calling Conventions Explained](#), linuxvox.com **LOW**
- [mandiant/STrace — A DTrace on Windows Reimplementation](#), Mandiant **MEDIUM**

10.47 V49 — NtTrace, a debugger-based native API tracer for Windows

This note documents NtTrace, a Windows tool that reports the calls a process makes into `ntdll.dll`, the user-mode boundary between an application and the NT kernel. It attaches as a debugger, using the Windows debug interface (`WaitForDebugEvent / ContinueDebugEvent`) to place breakpoints around the native call stubs in `ntdll`, then reads arguments and return values when a breakpoint fires. That attachment choice is deliberate: because NtTrace is a debugger rather than a kernel driver, it needs no administrator rights and no driver installation, at the cost of two structural blind spots — it cannot see kernel calls made on a process's behalf by device drivers, and it cannot see any call that bypasses `ntdll` via a direct `int 2e`, `syscall`, or `sysenter` instruction. The claim this appendix backs follows directly from the same mechanism: because the debug interface is event-based, calls from multiple threads that occur simultaneously in the real program get delivered to the tracer one at a time and so appear serialized in the trace output. The note states this as: "The observed interleaving is an artefact of the tracer, not of the program." The evidence for this is the author's own description of the debug-interface mechanism, corroborated by two independent developers who built the same breakpoint-based design and separately concluded it does not scale to concurrent or GUI workloads.

The note is explicit about what this claim does not establish. It offers no benchmark or magnitude for how badly a given workload's true interleaving is distorted — the serialization caveat is stated as a single sentence in the tool's own documentation, with no

guidance on severity, and the note's critical review flags this as a gap a reader could easily fail to internalize. Nor does the note claim NtTrace is unique in this respect: it treats thread serialization as one instance of a broader "observer effect," alongside the separate and better-evidenced claim that breakpoint tracing also distorts timing generally. The note does not claim to know whether the true interleaving could be recovered by other means, and it treats the unreliability of trace-derived concurrency evidence as a settled methodological caution for students rather than an open research question.

Referenced sources

- [NtTrace — Native API tracing for Windows](#) — Roger Orr **HIGH**
- [rogerorr/NtTrace — repository README](#) — Roger Orr **HIGH**
- [Home · rogerorr/NtTrace Wiki](#) — Roger Orr **HIGH**
- [Notes and Thoughts on Strace for Windows](#) — Jarek Przygodzki **MEDIUM**
- [AWStrace: Another Windows Strace](#) — downwithup (2026) **MEDIUM**
- [StraceNT - System Call Tracer for Windows NT](#) — Pankaj Garg **MEDIUM**
- [System Call Tracer for Windows \(drstrace\)](#) — Dr. Memory / DynamoRIO project **HIGH**
- [mxmssh/drltrace — README](#) — Maksim Shudrak **MEDIUM**
- [Introducing QTrace, a "zero knowledge" system call tracer](#) — Roberto Paleari (2014) **MEDIUM**
- [strace Wow Much Syscall](#) — Brendan Gregg (2014) **HIGH**
- [Introducing strace --seccomp-bpf](#) — Paul Chaignon (2019) **HIGH**
- [Do other Unix-like kernels have stable syscall ABIs?](#) — Unix & Linux Stack Exchange (2018-) **MEDIUM**
- [A 10-minute guide to the Linux ABI](#) — Alison Chaiken, opensource.com (2022) **HIGH**
- [A Deep Dive Into Malicious Direct Syscall Detection](#) — Or Chechik and Ofir Ozer, Palo Alto Networks (2024) **HIGH**
- [Overload Content by Article — ACCU](#) — ACCU (Association of C and C++ Users) **HIGH**
- [Windows System Calls & Native API: A Guide to Assembly Calling Conventions Explained](#) — linuxvox.com **LOW**
- [mandiant/STrace — A DTrace on Windows Reimplementation](#) — Mandiant **MEDIUM**